

Digital Logic and Design

Lecture : 01

Department of Computer Science
GPGC SWABI (Khyber Pakhtunkhwa)

Spring 2025 Semester
BS Course: DLD

Course Instructor :
Mr. Ihtisham Ul Haq
Lecturer (CS)
GPGC Swabi

Objectives

- ❑ At the end of course the students will be able to:
- 1. Understand Digital Systems and Number Systems** – Learn about digital vs. analog systems and number system conversions.
- 2. Learn Boolean Algebra and Logic Gates** – Understand logic gates and simplify Boolean expressions.
- 3. Design Combinational Circuits** – Create circuits like adders, multiplexers, and decoders.
- 4. Understand Sequential Circuits** – Learn about flip-flops, counters, and state machines.
- 5. Apply Digital Logic in Real-World Systems** – Use digital logic concepts in practical applications.

References

1. **"Digital Logic and Computer Design"** by Muhammad Rafique
2. **"Digital Design"** by M. Morris Mano & Michael D. Ciletti
3. **"Fundamentals of Digital Logic with Verilog Design"** by Stephen Brown & Zvonko Vranesic

Subject Marks Breakdown

- ❑ Midterm Marks : 20
- ❑ Internal Marks : 15 (Quiz, Assignment, Presentation, Attendance)
- ❑ Internal Viva Marks : 30
- ❑ Final Term Marks : 35

Contents

- ☐ Introduction to Digital Logic
- ☐ Analog Data vs. Digital Data
- ☐ Analog Systems vs. Digital Systems
- ☐ How Digital Systems Work
- ☐ Advantages of Digital Over Analog Systems

Introduction to Digital Logic

- ❑ Digital logic is the foundation of modern electronics and computing.
- ❑ It deals with the principles of representing and processing information in binary form (0s and 1s).
- ❑ Everything from computers, smartphones, and digital watches to smart appliances and robots relies on digital logic.
- ❑ **Why Study Digital Logic?**
 - ❑ It is the **fundamental building block** of computing and electronic circuits.
 - ❑ It helps us understand **how data is stored, processed, and transmitted** in digital devices.
 - ❑ It is **used in microprocessors, memory devices, and networking equipment**.

Introduction to Digital Logic

- ❑ **What is Data?**
- ❑ Data refers to any information that can be **measured, processed, or transmitted**.
- ❑ It can be in **analog** or **digital** form.

Introduction to Digital Logic

☐ **Analog Data**

☐ Analog data is continuous and can take an infinite number of values within a range.

☐ **Examples:**

☐ Human voice in air

☐ Temperature measured using a mercury thermometer

☐ Light intensity in a room

☐ Voltage signals in traditional radio and television

☐ **Graphical Representation:**

☐ Analog signals are represented as smooth, continuous waveforms (like sine waves).

☐ Example: A **sound wave** is an analog signal because it changes smoothly over time.

Introduction to Digital Logic

☐ **Key Characteristics of Analog Data:**

- ☐ Continuous in nature (smooth flow of values).
- ☐ More susceptible to noise and distortion (e.g., in old cassette tapes).
- ☐ Difficult to process and store digitally without conversion.

Introduction to Digital Logic

- ❑ **Digital Data**

- ❑ Digital data consists of discrete values, typically represented in binary (0s and 1s).

- ❑ **Examples:**

- ❑ Text stored in a computer (ASCII code)

- ❑ Digital images (JPEG, PNG)

- ❑ MP3 audio files

- ❑ Data stored in USB flash drives

- ❑ **Graphical Representation:**

- ❑ Digital signals are represented as **step-like waveforms** with sudden transitions between high (1) and low (0) levels.

Introduction to Digital Logic

☐ **Key Characteristics of Digital Data:**

- ☐ Discrete values (binary 0s and 1s).
- ☐ More resistant to noise and interference.
- ☐ Easier to store, process, and transmit using digital circuits.

Introduction to Digital Logic

❑ Analog vs. Digital Data – Comparison Table

Feature	Analog Data	Digital Data
Nature	Continuous	Discrete (0s & 1s)
Examples	Sound waves, temperature	Text, images, MP3 audio
Storage	Stored on tapes, vinyl	Stored on hard drives, SSD
Transmission	Prone to noise (e.g., radio static)	Less affected by noise
Processing	Requires analog processing circuits	Uses microprocessors, logic gates

Introduction to Digital Logic

- ❑ **Analog System**

- ❑ An **analog system** processes **continuous signals** without converting them into discrete values.

- ❑ **Examples:**

- ❑ **Analog radio:** The sound is transmitted as continuous waveforms.

- ❑ **Mercury thermometer:** Temperature is measured as a continuous rise or fall in mercury.

- ❑ **Old televisions and cameras:** Used analog signals to transmit and display images.

Introduction to Digital Logic

- ❑ **Digital System**
- ❑ A **digital system** processes and stores information in the form of **binary numbers (0s and 1s)**.
- ❑ **Examples:**
 - ❑ **Digital watches:** Show time as discrete digits.
 - ❑ **Computers and smartphones:** Process everything in binary form.
 - ❑ **Digital communication (Wi-Fi, mobile networks):** Transmit data in digital packets.

Introduction to Digital Logic

- ❑ **How Digital Systems Work**
- ❑ **Analog-to-Digital Conversion (ADC)** – Converts real-world analog signals into digital form.
 - ❑ Example: A microphone records sound as an analog signal, which is converted into digital data.
- ❑ **Processing and Storage** – Digital circuits (microprocessors) manipulate data using logic gates.
- ❑ **Digital-to-Analog Conversion (DAC)** – Converts processed digital signals back into analog for real-world output.
 - ❑ Example: A digital music file is converted to analog sound in speakers.

Introduction to Digital Logic

❑ Analog vs. Digital Systems: Comparison Table

Feature	Analog System	Digital System
Signal Type	Continuous (waveform)	Discrete (binary 0s & 1s)
Processing	Amplifiers, filters	Microprocessors, logic gates
Noise Sensitivity	Prone to distortion	More resistant to noise
Storage	Stored as waveforms	Stored as binary data
Speed	Slower	Faster
Example	Old TV, analog watch	Computer, smartphone

Introduction to Digital Logic

❑ Advantages of Digital Over Analog Systems

1. **Accuracy:** More precise data representation
2. **Noise Resistance:** Less affected by interference
3. **Storage:** Easy to store large amounts of data
4. **Processing:** Faster and more efficient
5. **Cost:** Cheaper over time due to mass production

Any Question?

Thank you

Digital Logic and Design

Lecture : 02

Department of Computer Science
GPGC SWABI (Khyber Pakhtunkhwa)

Spring 2025 Semester
BS Course: DLD

Course Instructor :
Mr. Ihtisham Ul Haq
Lecturer (CS)
GPGC Swabi

Contents

- ☐ Number Systems
 - ☐ Decimal Number System
 - ☐ Binary Number System
 - ☐ Octal Number System
 - ☐ Hexadecimal Number System
- ☐ Number System Conversion (Base Conversion)
- ☐ Binary Coded Decimal
- ☐ Gray Code

Numbers

- ❑ The arithmetic value which is used for representing the quantity and used in making calculations are defined as NUMBERS.
- ❑ A symbol like “4, 5, 6” which represents a number is known as **numerals**.
- ❑ Without numbers, counting things is not possible, date, time, money, etc.

Number System

- ❑ A number system, also known as a numeral system or base, is a mathematical notation for representing numbers using a set of symbols or digits.
- ❑ There are four commonly used number systems:
 - ❑ **Decimal Number System**
 - ❑ **Binary Number System**
 - ❑ **Octal Number System**
 - ❑ **Hexadecimal Number System**

Decimal Number System

- ❑ Number system with a base value of 10 is termed a Decimal number system.
- ❑ It uses 10 digits i.e. 0-9 for the creation of numbers.
- ❑ Here, each digit in the number is at a specific place with place value a product of different powers of 10.
- ❑ First place value is called units, second to the left is Tens, so on Hundreds, Thousands, etc.

...	100,000	10,000	1,000	100	10	1
...	10^5	10^4	10^3	10^2	10^1	10^0
...	6	5	4	3	2	1
	Sixth digit	Fifth digit	Fourth digit	Third digit	Second digit	First digit

Decimal Number System

❑ Here, units have the place value as 10^0 , tens have the place value as 10^1 , hundreds as 10^2 , thousands as 10^3 , and so on.

❑ For example, 10264 has place values as,
$$= (1 \times 10^4) + (0 \times 10^3) + (2 \times 10^2) + (6 \times 10^1) + (4 \times 10^0)$$

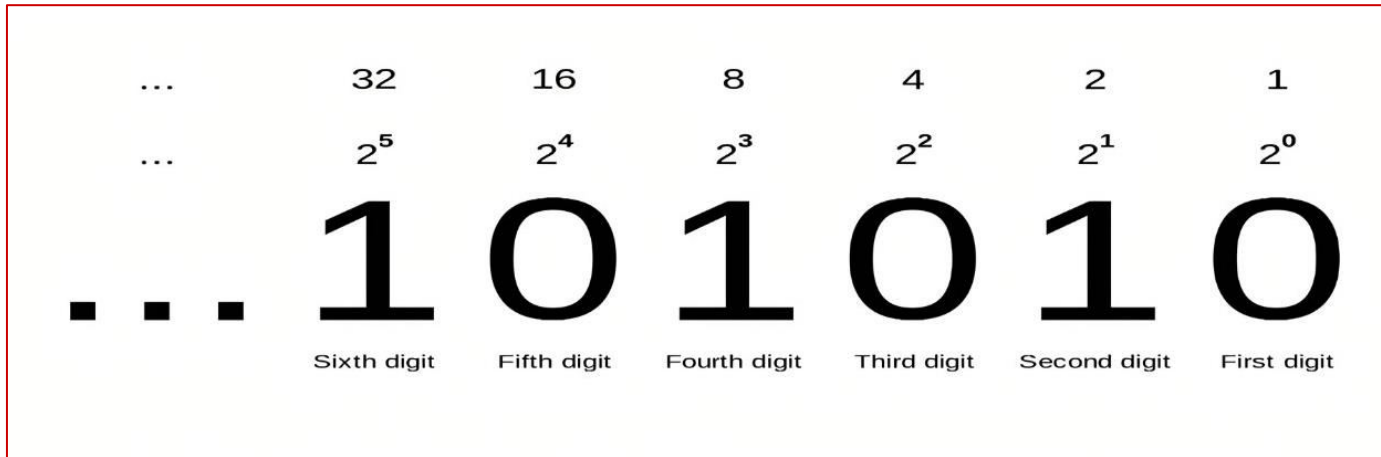
$$= (1 \times 10000) + (0 \times 1000) + (2 \times 100) + (6 \times 10) + (4 \times 1)$$

$$= 10000 + 0 + 200 + 60 + 4$$

$$= (10264)_{10}$$

Binary Number System

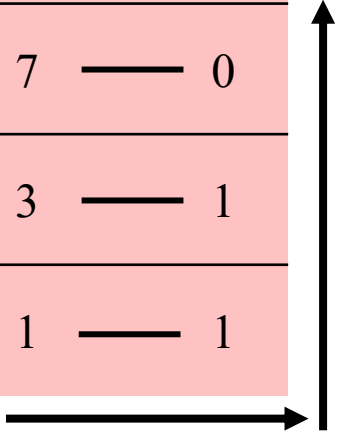
- ❑ Number System with base value 2 is termed as Binary number system.
- ❑ It uses 2 digits i.e. 0 and 1 for the creation of numbers.
- ❑ The numbers formed using these two digits are termed Binary Numbers.
- ❑ The binary number system is used in electronic devices and computer systems to represent the two states ON and OFF.



Binary Number System

- ❑ **Decimal To Binary Conversion:**
- ❑ Recursively divide the decimal number by 2.
- ❑ If there is a remainder write it separately.
- ❑ When the decimal number reaches to 1 stop the division.
- ❑ Write the remainders of each step in reverse order which will give you the binary equivalent.
- ❑ E.g. Convert the decimal number $(14)_{10}$ to its binary equivalent.
- ❑ **Binary Equivalent : $(1110)_2$**

2	14
2	7 — 0
2	3 — 1
	1 — 1



Binary Number System

- ❑ **Binary To Decimal Conversion:**
- ❑ Multiply each bit with its place values.
- ❑ Add all the numbers, this will give you the decimal equivalent.
- ❑ For example, the binary number **(11001)₂** in decimal would be equal to **(25)₁₀**.

$$= (1 \times 2^4) + (1 \times 2^3) + (0 \times 2^2) + (0 \times 2^1) + (1 \times 2^0)$$

$$= (1 \times 16) + (1 \times 8) + (0 \times 4) + (0 \times 2) + (1 \times 1)$$

$$= 16 + 8 + 0 + 0 + 1$$

$$= (25)_{10}$$

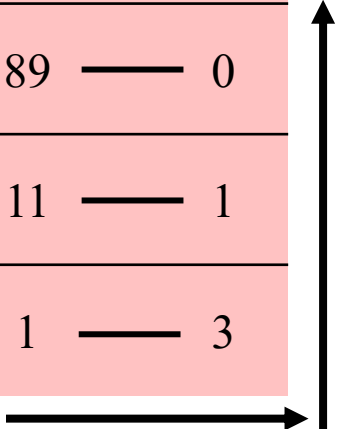
Octal Number System

- ❑ Octal Number System is one in which the base value is 8.
- ❑ It uses 8 digits i.e. 0-7 for the creation of Octal Numbers.
- ❑ Here the place values are 8^0 , 8^1 , and 8^2 .
- ❑ Octal Numbers are useful for the representation of UTF8 Numbers.

Octal Number System

- ❑ **Decimal To Octal Conversion:**
- ❑ Recursively divide the decimal number by 8.
- ❑ If there is a remainder write it separately.
- ❑ When the decimal number becomes less than 8 stop the division.
- ❑ Write the remainders of each step in reverse order which will give you the octal equivalent.
- ❑ E.g. Convert the decimal number $(712)_{10}$ to its octal equivalent.
- ❑ **Octal Equivalent : $(1310)_8$**

8	712
8	89 — 0
8	11 — 1
	1 — 3



Octal Number System

- ❑ **Octal To Decimal Conversion:**
- ❑ Multiply each digit with its place values.
- ❑ Add all the numbers, this will give you the decimal equivalent.
- ❑ For example, the octal number $(1731)_8$ in decimal would be equal to $(985)_{10}$.
$$= (1 \times 8^3) + (7 \times 8^2) + (3 \times 8^1) + (1 \times 8^0)$$
$$= (1 \times 512) + (7 \times 64) + (3 \times 8) + (1 \times 1)$$
$$= 512 + 448 + 24 + 1$$
$$= (985)_{10}$$

Octal Number System

❑ Binary to Octal Conversion:

❑ To convert from binary to octal, start grouping the bits in groups of 3 from the right end.

❑ Write the equivalent octal for the 3-bit binary.

❑ Add 0's on the left to adjust the groups.

❑ Convert binary number $(100110)_2$ to octal number.

$$= (100110)_2$$

$$= (100 \ 110)_2$$

$$= (4 \quad 6)_8$$

$$= (46)_8$$

❑ Answer : $(46)_8$

Binary Triplets	Octal Equivalent
000	0
001	1
010	2
011	3
100	4
101	5
110	6
111	7

Octal Number System

❑ Octal to Binary Conversion:

❑ To convert from octal to binary, for each octal digit assign the equivalent 3-bit binary.

❑ Convert binary number $(46)_8$ to octal number.

$$= (46)_8$$

$$= (4 \quad 6)_8$$

$$= (100 \quad 110)_2$$

$$= (100110)_2$$

❑ Answer : $(100110)_2$

Binary Triplets	Octal Equivalent
000	0
001	1
010	2
011	3
100	4
101	5
110	6
111	7

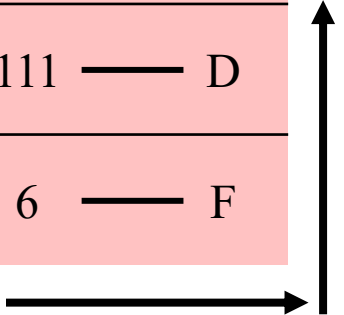
Hexadecimal Number System

- ❑ Number System with base value 16 is termed as Hexadecimal Number System.
- ❑ It uses 16 digits for the creation of its numbers.
- ❑ Digits from 0-9 are taken like the digits in the decimal number system but the digits from 10-15 are represented as A-F i.e. 10 is represented as A, 11 as B, 12 as C, 13 as D, 14 as E, and 15 as F.
- ❑ Hexadecimal Numbers are useful for handling memory address locations.

Hexadecimal Number System

- ❑ **Decimal To Hexadecimal Conversion:**
- ❑ Recursively divide the decimal number by the number 16/F.
- ❑ If there is a remainder write it separately.
- ❑ When the decimal number becomes less than 16/F stop the division.
- ❑ Write the remainders of each step in reverse order which will give you the hexadecimal equivalent.
- ❑ E.g. Convert the decimal number $(1789)_{10}$ to its hexadecimal equivalent.
- ❑ **Hexadecimal Equivalent : $(6FD)_{16}$**

16	1789
16	111 — D
	6 — F



Hexadecimal Number System

- ❑ **Hexadecimal To Decimal Conversion:**

- ❑ Multiply each digit with its place values.

- ❑ Add all the numbers, this will give you the decimal equivalent.

- ❑ For example, the hexadecimal number $(9E1B)_{16}$ in decimal would be equal to $(40475)_{10}$.

$$= (9 \times 16^3) + (E \times 16^2) + (1 \times 16^1) + (B \times 16^0)$$

$$= (9 \times 4096) + (14 \times 256) + (1 \times 16) + (11 \times 1)$$

$$= 36864 + 3584 + 16 + 11$$

$$= (40475)_{10}$$

Hexadecimal Number System

- ❑ **Binary to Hexadecimal Conversion:**
- ❑ To convert from Binary to Hexadecimal, start grouping the bits in groups of 4 from the right-end.
- ❑ Write the equivalent hexadecimal for the 4-bit binary.
- ❑ Add extra 0's on the left to adjust the groups.
- ❑ Convert binary number $(10110110)_2$ to hexadecimal.
= $(10110110)_2$
= $(1011 \quad 0110)_2$
= $(B \quad 6)_{16}$
= $(B6)_{16}$
- ❑ Answer : $(B6)_{16}$

Binary	Hexadecimal Equivalent
0000	0
0001	1
0010	2
0011	3
0100	4
0101	5
0110	6
0111	7
1000	8
1001	9
1010	A
1011	B
1100	C
1101	D
1110	E
1111	F

Hexadecimal Number System

- ❑ **Hexadecimal to Binary Conversion:**
- ❑ To convert from Hexadecimal to Binary, for each hexadecimal digit write its equivalent 4-bit binary.
- ❑ Convert hexadecimal number $(B6)_{16}$ to binary.
= $(B6)_{16}$
= $(B \quad 6)_{16}$
= $(1011 \quad 0110)_2$
= $(10110110)_2$
- ❑ Answer : $(10110110)_2$

Binary	Hexadecimal Equivalent
0000	0
0001	1
0010	2
0011	3
0100	4
0101	5
0110	6
0111	7
1000	8
1001	9
1010	A
1011	B
1100	C
1101	D
1110	E
1111	F

Binary Coded Decimal

- ❑ Binary Coded Decimal, or BCD, is another process for converting decimal numbers into their binary equivalents.
- ❑ It is a form of binary encoding where each digit in a decimal number is represented in the form of bits.
- ❑ This encoding can be done in either 4-bit or 8-bit (usually 4-bit is preferred).
- ❑ BCD code is more precisely known as 8421 BCD code.

Decimal Number	BCD Equivalent
0	0000
1	0001
2	0010
3	0011
4	0100
5	0101
6	0110
7	0111
8	1000
9	1001

Binary Coded Decimal

❑ **Example 1: Convert $(123)_{10}$ to BCD Code.**

$$= (123)_{10}$$

$$= (1 \quad 2 \quad 3)_{10}$$

$$= (0001 \ 0010 \ 0011)_2$$

$$= (000100100011)_2 \quad \text{(BCD Equivalent)}$$

❑ **Example 2: Convert $(469)_{10}$ to BCD Code.**

$$= (469)_{10}$$

$$= (4 \quad 6 \quad 9)_{10}$$

$$= (0100 \ 0110 \ 1001)_2$$

$$= (010001101001)_2 \quad \text{(BCD Equivalent)}$$

Gray Code

- ❑ Gray Code is a binary numeral system where two successive values differ by only one bit.
- ❑ It is also known as Reflected Binary Code/Unit distance code.
- ❑ Why Use Gray Code?
 - ❑ **Error Minimization** – Since only one bit changes at a time, errors in digital systems (like encoders) are reduced.
 - ❑ **Hardware Applications** – Used in analog-to-digital converters (ADCs), rotary encoders, and Karnaugh maps (K-maps).

Decimal	Binary Code	Gray Code
0	0000	0000
1	0001	0001
2	0010	0011
3	0011	0010
4	0100	0110
5	0101	0111
6	0110	0101
7	0111	0100
8	1000	1100
9	1001	1101
10	1010	1111

Gray Code

❑ Binary to Gray Code Conversion

❑ To convert binary to Gray code, follow these steps:

1. Keep the first bit same.
2. For the remaining bits: XOR the previous binary bit with the current bit.

Gray Code

☐ **Gray Code to Binary Conversion**

☐ To convert Gray code to binary:

1. Keep the first bit same.
2. For the next bits: XOR the previous binary bit with the current Gray code bit.

Any Question?

Thank you

Digital Logic and Design

Lecture : 03

Department of Computer Science
GPGC SWABI (Khyber Pakhtunkhwa)

Spring 2025 Semester
BS Course: DLD

Course Instructor :
Mr. Ihtisham Ul Haq
Lecturer (CS)
GPGC Swabi

Contents

- ❑ Complements
 - ❑ 1's Complement
 - ❑ 2's Complement
- ❑ Binary Arithmetic's
 - ❑ Binary Addition
 - ❑ Binary Subtraction
 - ❑ Binary Subtraction (Using 1's Complement)
 - ❑ Binary Subtraction (Using 2's Complement)
 - ❑ Binary Multiplication
 - ❑ Binary Division

Complement's

- ❑ In digital systems, complements are used for:
 - ❑ **Simplifying subtraction** using addition.
 - ❑ **Error detection** in digital circuits.
 - ❑ **Representing negative numbers** in binary systems.
- ❑ There are two types of complements:
 - ❑ 1's Complement
 - ❑ 2's Complement

Complement's

❑ 1's Complement

- ❑ Obtained by **flipping** (inverting) all bits ($0 \rightarrow 1$, $1 \rightarrow 0$).
- ❑ Used in **subtraction** and **error detection**.

❑ Example 1: Finding 1's Complement

- ❑ Find the 1's complement of $(101010)_2$.
- ❑ Original: **1 0 1 0 1 0**
1's Complement: **0 1 0 1 0 1**

Complement

☐ 2's Complement

- ☐ Obtained by adding 1 to the 1's complement.
- ☐ Used in **signed number representation**.
- ☐ **More efficient** for subtraction.

☐ Example 2: Finding 2's Complement

- ☐ Find the 2's complement of $(101010)_2$.

Find 1's complement: **010101**

Add 1: **$010101 + 1 = 010110$**

2's complement: 010110

Binary Arithmetic's

- ❑ Binary arithmetic is one of the fundamental concepts in the field of digital electronics and computer engineering.
- ❑ It is basically the mathematics of binary numbers that allows us to perform various arithmetic operations on binary numbers.
- ❑ We will discuss the following four main binary arithmetic operations –
 - ❑ Binary Addition
 - ❑ Binary Subtraction
 - ❑ Binary Multiplication
 - ❑ Binary Division

Binary Arithmetic's

❑ Binary Addition

❑ Binary addition follows four basic rules:

❑ **Ex 1:** $(100)_2$
 $+ \underline{(011)_2}$
 $(111)_2$

Binary Digits	Sum	Carry
$0 + 0$	0	x
$0 + 1$	1	x
$1 + 0$	1	x
$1 + 1$	0	1

❑ **Ex 2:** $(1011)_2$
 $+ \underline{(0101)_2}$
 $(10000)_2$

Binary Arithmetic's

❑ Binary Subtraction

❑ Binary subtraction follows four basic rules:

❑ **Ex 1:** $(1101)_2$
 $- (0101)_2$
 $\hline$
 $(1000)_2$

Binary Digits	Difference	Borrow
$0 - 0$	0	x
$1 - 0$	1	x
$1 - 1$	0	x
$0 - 1$	1 (borrow 1)	1

❑ **Ex 2:** $(1010)_2$
 $- (0011)_2$
 $\hline$
 $(0111)_2$

Binary Arithmetic's

❑ Binary Subtraction (Using 1's Complement)

❑ Steps for 1's Complement Subtraction

1. Find the 1's complement of the subtrahend.
2. Add the result to the minuend.
3. If there is a carry, add it to the result (for positive results).
4. If no carry occurs, take the 1's complement of the result and make it negative.

Binary Arithmetic's

❑ Binary Subtraction (Using 1's Complement)

❑ Ex 1: $1010_2 - 0101_2$ (10 - 5 in decimal)

1. Find 1's Complement of the Subtrahend $(0101)_2$.

1's complement of $0101_2 \rightarrow 1010_2$ (Invert all bits)

2. Add to the Minuend $(1010)_2$.

$$\begin{array}{r} (1010)_2 \\ + \underline{(1010)_2} \\ (10100)_2 \end{array}$$

3. Add Carry (If Any). Since there is a carry (1), add it back to the result.

❑ Final Answer: $(0101)_2$ (5 in decimal)

Binary Arithmetic's

❑ Binary Subtraction (Using 1's Complement)

❑ Ex 2: $0100_2 - 1010_2$ (4 - 10 in decimal)

1. Find 1's Complement of the Subtrahend $(1010)_2$.

1's complement of $1010_2 \rightarrow 0101_2$ (Invert all bits)

2. Add to the Minuend $(0100)_2$.

$$\begin{array}{r} (0100)_2 \\ + \underline{(0101)_2} \\ \hline (1001)_2 \quad (\text{No carry}) \end{array}$$

3. Since there is no carry, the result is negative, so take 1's complement of 1001_2 .

❑ Final Answer: $(0110)_2$ (-6 in decimal)

Binary Arithmetic's

❑ Binary Subtraction (Using 2's Complement)

❑ Steps for 2's Complement Subtraction

1. Find the 2's complement of the subtrahend.
2. Add the result to the minuend.
3. If there is a carry, discard it (for positive results).
4. If no carry occurs, take the 2's complement of the result and make it negative.

Binary Arithmetic's

❑ Binary Subtraction (Using 2's Complement)

❑ Ex 1: $1010_2 - 0101_2$ (10 - 5 in decimal)

1. Find 2's Complement of the Subtrahend $(0101)_2$.

2's complement of $0101_2 \rightarrow 1011_2$ (Invert all bits and add 1)

2. Add to the Minuend $(1010)_2$.

$$\begin{array}{r} (1010)_2 \\ + \underline{(1011)_2} \\ \hline (10101)_2 \quad (\text{Ignore leftmost carry}) \end{array}$$

3. If there is a carry, discard it (for positive results).

❑ Final Answer: $(0101)_2$ (5 in decimal)

Binary Arithmetic's

❑ Binary Subtraction (Using 2's Complement)

❑ Ex 2: $0100_2 - 1010_2$ (4 - 10 in decimal)

1. Find 2's Complement of the Subtrahend $(1010)_2$.

2's complement of $1010_2 \rightarrow 0110_2$ (Invert all bits and add 1)

2. Add to the Minuend $(0100)_2$.

$$\begin{array}{r} (0100)_2 \\ + \underline{(0110)_2} \\ \hline (1010)_2 \quad (\text{No carry}) \end{array}$$

3. Since there is no carry, the result is negative, so take 2's complement of 1010_2 .

❑ Final Answer: $(0110)_2$ (-6 in decimal)

Binary Arithmetic's

❑ Binary Multiplication

❑ Binary multiplication follows four basic rules:

❑ **Ex 1:**

$$\begin{array}{r} (1\ 0\ 1\ 1)_2 \\ \times \quad (1\ 1)_2 \\ \hline (1\ 0\ 1\ 1)_2 \\ + (1\ 0\ 1\ 1)_2 \quad \text{(Shifted left)} \\ \hline (1\ 0\ 0\ 0\ 0\ 1)_2 \end{array}$$

Binary Digits	Product
0 x 0	0
0 x 1	0
1 x 0	0
1 x 1	1

Binary Arithmetic's

❑ Binary Division

❑ Binary division follows the same principles as decimal division, using subtraction and shifting.

❑ **Ex 1:** $(1\ 1\ 0\ 0\ 1\ 1)_2 \div (1\ 1)_2$

$$\begin{array}{r} 1\ 0\ 0\ 0\ 1 \\ \overline{11 \sqrt{1\ 1\ 0\ 0\ 1\ 1}} \\ \underline{1\ 1} \\ 0\ 0\ 0\ 1\ 1 \\ \underline{1\ 1} \\ 0 \end{array}$$

❑ **Quotient:** $(1\ 0\ 0\ 0\ 1)_2$

❑ **Remainder:** $(0)_2$

Any Question?

Thank you

Digital Logic and Design

Lecture : 04

Department of Computer Science
GPGC SWABI (Khyber Pakhtunkhwa)

Spring 2025 Semester
BS Course: DLD

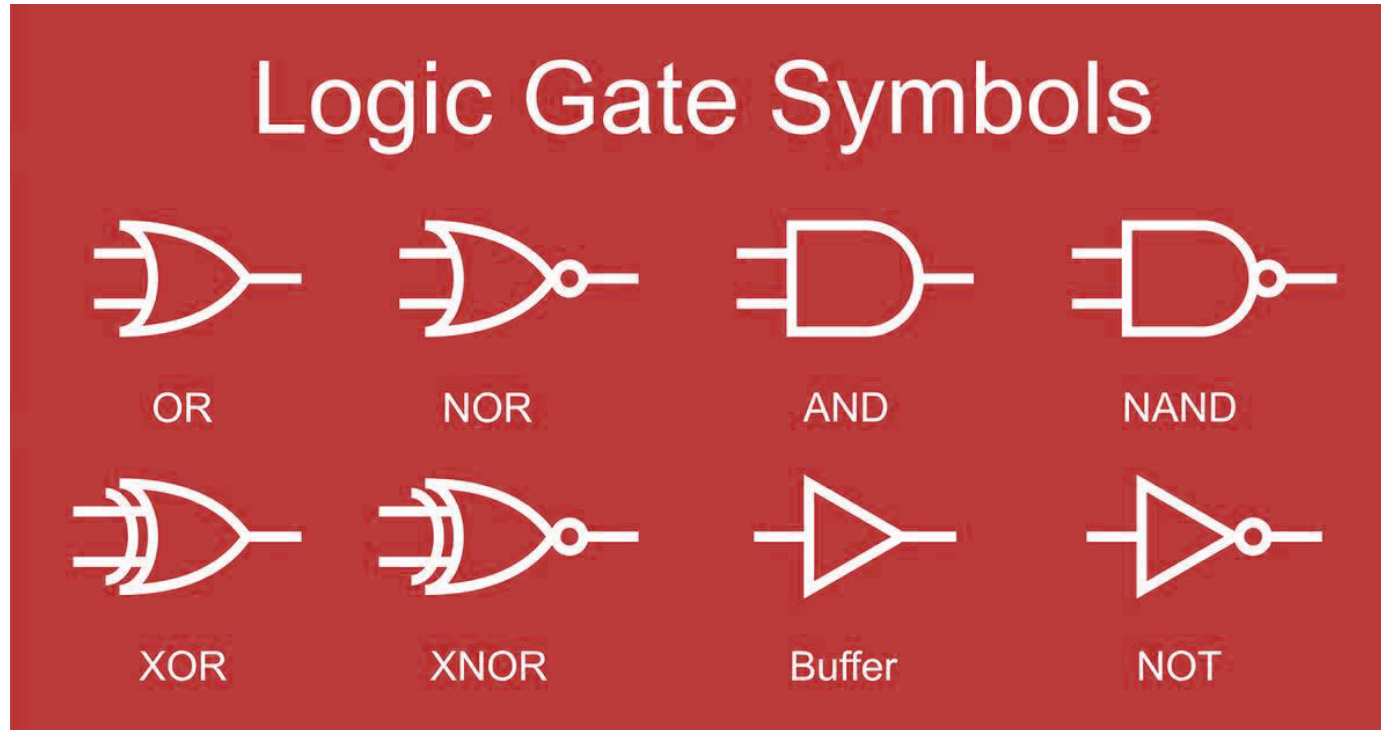
Course Instructor :
Mr. Ihtisham Ul Haq
Lecturer (CS)
GPGC Swabi

Contents

- ❑ Logic Gates
 - ❑ Classification of Logic Gates
 - ❑ Basic Gates (AND, OR, NOT)
 - ❑ Universal Gates (NAND, NOR)
 - ❑ Special Purpose Gates (XOR and XNOR)

Logic Gates

- ❑ Logic gates are the fundamental building blocks of digital circuits.
- ❑ They perform logical operations on one or more binary inputs to produce a single output.

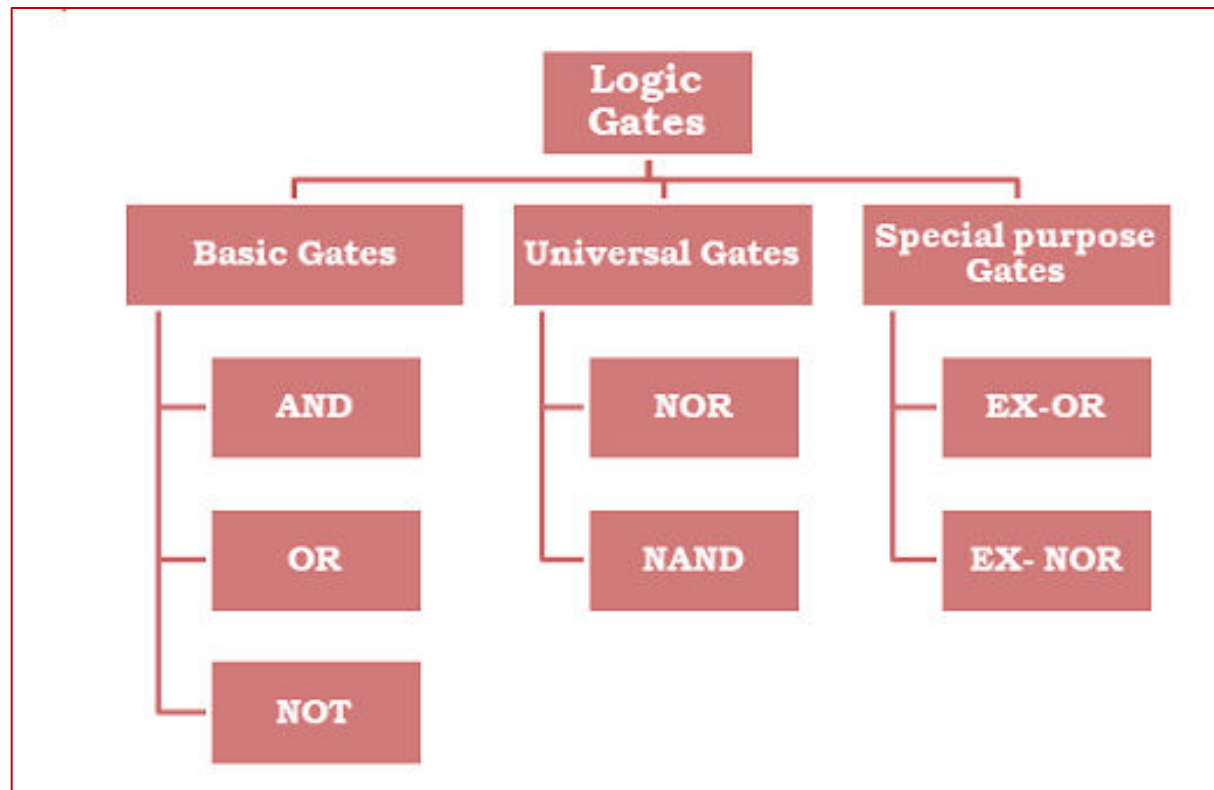


Logic Gates

- ❑ **Truth Table in Boolean Algebra**
- ❑ A **Truth Table** is a mathematical table used to define the output of a **Logic Gate/Boolean function** for all possible combinations of input values.
- ❑ It helps in analyzing and designing logic circuits.
- ❑ For n inputs there are 2^n possible input combinations.

Logic Gates

- ❑ There are three basic classes of logic gates as illustrated in the diagram.



Basic Gates

- ❑ The three fundamental logic gates are:
 - ❑ **AND Gate**
 - ❑ **OR Gate**
 - ❑ **NOT Gate**
- ❑ Each gate performs a unique Boolean operation and follows a specific **truth table**.
- ❑ 0 shows low voltage level called logic low
- ❑ 1 show high voltage level called logic high

Basic Gates

- ❑ **AND Gate**
- ❑ The **AND** gate outputs **1** only if **all inputs are 1**; otherwise, it outputs **0**.

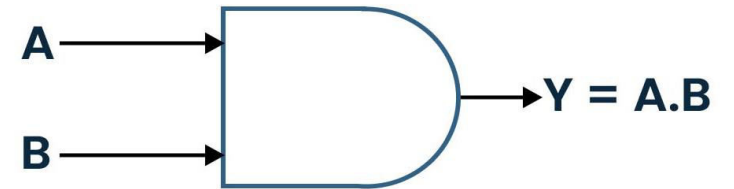


Figure: AND Gate

- ❑ **Boolean Expression**

$$Y = A \cdot B$$

OR

(where " $\cdot$ " represents the AND operation)

$$Y = AB$$

Truth Table (AND Gate)

Input A	Input B	Output $Y = A \cdot B$
0	0	0
0	1	0
1	0	0
1	1	1

Basic Gates

- ❑ **OR Gate**
- ❑ The **OR** gate outputs **1** if **at least one input is 1**; otherwise, it outputs **0**.

- ❑ **Boolean Expression**

$Y=A+B$ (where "+" represents the OR operation)

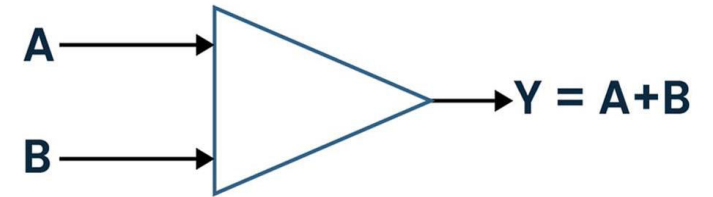


Figure: OR Gate

Truth Table (OR Gate)

Input A	Input B	Output $Y = A+B$
0	0	0
0	1	1
1	0	1
1	1	1

Basic Gates

❑ NOT Gate

❑ The **NOT gate** (also called an **Inverter**) **flips** the input:

- ❑ If the input is **1**, the output is **0**.
- ❑ If the input is **0**, the output is **1**.

❑ Boolean Expression

$Y=A'$ (where " ' " represents the NOT operation)

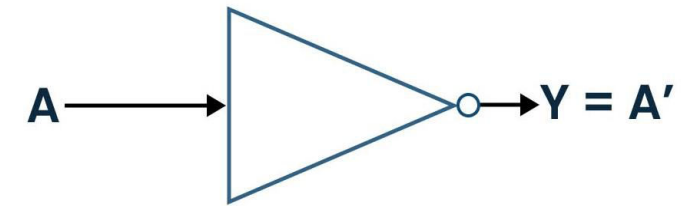


Figure: NOT Gate

Truth Table (NOT Gate)

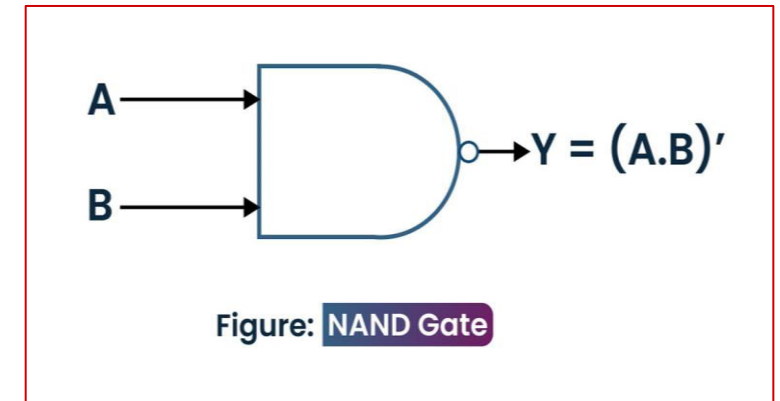
Input A	Output $Y=A'$
0	1
1	0

Universal Gates

- ❑ Universal gates are **logic gates that can be used to implement any Boolean function or logic circuit.**
- ❑ The two universal gates are:
 - ❑ **NAND Gate** (NOT AND)
 - ❑ **NOR Gate** (NOT OR)
- ❑ These gates are called **universal** because any logic gate (AND, OR, NOT, XOR, XNOR) can be designed using only NAND or only NOR gates.

Universal Gates

- ❑ **NAND Gate (NOT AND Gate)**
- ❑ The NAND gate is the **inverted** version of the AND gate.
- ❑ It gives **0** only when all inputs are **1**; otherwise, the output is **1**.



- ❑ **Boolean Expression**
 $Y = (A \cdot B)'$ (The output is the **complement** of AND operation)

Truth Table (NAND Gate)		
Input A	Input B	Output $Y = (A \cdot B)'$
0	0	1
0	1	1
1	0	1
1	1	0

Universal Gates

- ❑ **NOR Gate (NOT OR Gate)**
- ❑ The NOR gate is the **inverted** version of the OR gate.
- ❑ It gives **1** only when all inputs are **0**; otherwise, the output is **0**.
- ❑ **Boolean Expression**
 $Y = (A+B)'$ (The output is the **complement** of OR operation)

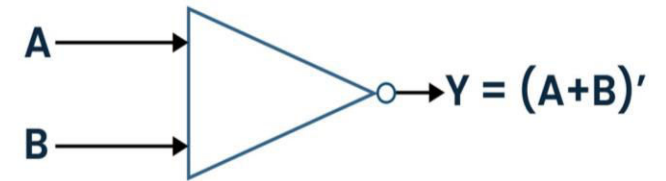


Figure: NOR Gate

Truth Table (NOR Gate)

Input A	Input B	Output $Y = (A+B)'$
0	0	1
0	1	0
1	0	0
1	1	0

Special Purpose Gates

- ❑ Special purpose gates are logic gates that perform **specific Boolean functions** beyond the basic AND, OR, and NOT operations.
- ❑ The two most commonly used special purpose gates are:
 - ❑ **XOR (Exclusive OR) Gate**
 - ❑ **XNOR (Exclusive NOR) Gate**
- ❑ These gates are widely used in **arithmetic circuits, error detection, data processing, and encryption.**

Special Purpose Gates

- ❑ **XOR Gate (Exclusive OR Gate)**
- ❑ The **XOR gate** outputs **1** if the number of **1s in the input is odd**.
- ❑ If both inputs are the same (either 0,0 or 1,1), the output is **0**.

- ❑ **Boolean Expression**

$$Y = A \oplus B$$

OR

$$Y = (A \cdot B') + (A' \cdot B) \quad \text{The XOR operation is represented by } \oplus$$

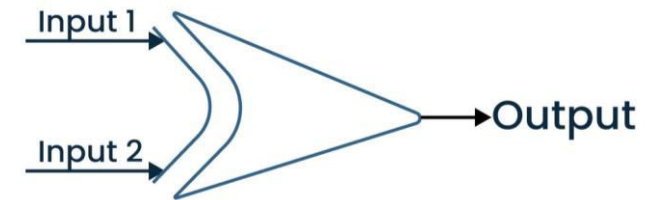


Figure: X-OR Gate

Truth Table (XOR Gate)

Input A	Input B	Output $Y = A \oplus B$
0	0	0
0	1	1
1	0	1
1	1	0

Special Purpose Gates

- ❑ **XNOR Gate (Exclusive NOR Gate)**
- ❑ The **XNOR** gate is the **inverted XOR** gate.
- ❑ It outputs **1** if the inputs are the same (both 0 or both 1) and **0** if the inputs are different.

- ❑ **Boolean Expression**

$$Y=(A\oplus B)'$$

OR

$$Y=A\cdot B+A'\cdot B'$$

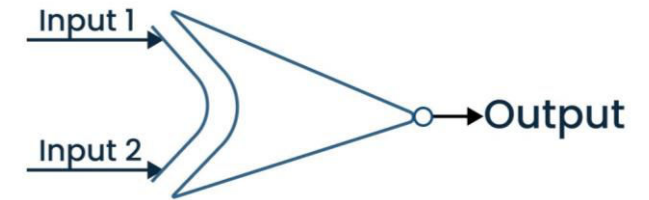


Figure: X-NOR Gate

Truth Table (XNOR Gate)

Input A	Input B	Output $Y=(A\oplus B)'$
0	0	1
0	1	0
1	0	0
1	1	1

Any Question?

Thank you

Digital Logic and Design

Lecture : 05

Department of Computer Science
GPGC SWABI (Khyber Pakhtunkhwa)

Spring 2025 Semester
BS Course: DLD

Course Instructor :
Mr. Ihtisham Ul Haq
Lecturer (CS)
GPGC Swabi

Contents

- ❑ Boolean Algebra
 - ❑ Boolean Algebra Laws and Theorems
 - ❑ De Morgan's Theorem
 - ❑ Duality Theorem
 - ❑ Boolean Expression Simplification
 - ❑ Applications of Boolean Algebra

Boolean Algebra

- ❑ Boolean Algebra is a mathematical framework developed by **George Boole** in the mid-19th century.
- ❑ Boolean Algebra is a branch of algebra that deals with binary variables (0 and 1) and logical operations.
- ❑ It is widely used in **digital logic design, computer science, and circuit simplification.**
- ❑ **Why Boolean Algebra?**
 - ❑ It simplifies the design of **digital circuits.**
 - ❑ Operations include **AND, OR, NOT.**
 - ❑ It uses **Boolean laws and theorems** to simplify expressions.
 - ❑ It reduces the number of logic gates required in a circuit.
 - ❑ It helps in creating **efficient and optimized logical expressions.**

Boolean Algebra

- ❑ Boolean Algebra operates on **binary variables** that can take only two values:
 - ❑ **0 (LOW or FALSE)**
 - ❑ **1 (HIGH or TRUE)**

Boolean Algebra Laws and Theorems

❑ Boolean expressions can be simplified using fundamental laws and theorems.

❑ **Identity Law**

$$A + 0 = A$$

$$A \cdot 1 = A$$

❑ **Null (Dominance) Law**

$$A + 1 = 1$$

$$A \cdot 0 = 0$$

❑ **Idempotent Law**

$$A + A = A$$

$$A \cdot A = A$$

Boolean Algebra Laws and Theorems

☐ Inverse (Complement) Law

$$A + A' = 1$$

$$A \cdot A' = 0$$

☐ Double Inverse (Complement) Law

$$(A')' = A$$

☐ Absorption Law:

$$A + A.B = A$$

$$A.(A + B) = A$$

Boolean Algebra Laws and Theorems

❑ Commutative Laws

$$A+B=B+A$$

$$A \cdot B = B \cdot A$$

❑ Associative Laws

$$(A+B)+C=A+(B+C)$$

$$(A \cdot B) \cdot C = A \cdot (B \cdot C)$$

❑ Distributive Law:

$$A \cdot (B + C) = A \cdot B + A \cdot C$$

$$A + B \cdot C = (A + B) \cdot (A + C)$$

Boolean Algebra Laws and Theorems

□ De Morgan's Theorem

- These theorems help in transforming logic expressions, which is useful in circuit design and simplification.

1. First Theorem:

$$(A \cdot B)' = A' + B'$$

NAND = Inverted OR

2. Second Theorem:

$$(A + B)' = A' \cdot B'$$

NOR = Inverted AND

Boolean Algebra Laws and Theorems

1. Find the complement of the function $F1 = x' yz' + x' y' z$

☐ $F = (x' yz' + x' y' z)'$

☐ $F = (x' + y + z)' \cdot (x' + y' + z)'$ [using de morgan's law $(A+B)' = A' \cdot B'$]

☐ $F = (x + y' + z') \cdot (x + y + z')$

Boolean Algebra Laws and Theorems

2. Find the complement of the function $F2 = x(y'z' + yz)$.

☐ $F = x(y'z' + yz)$

☐ $F = [x.(y'z' + yz)]'$

☐ $F = x' + (y'z' + yz)'$

[using de morgan's law $(A \cdot B)' = A' + B'$]

☐ $F = x' + (y'z')' . (yz)'$

[using de morgan's law $(A + B)' = A' \cdot B'$]

☐ $F = x' + (y + z)(y' + z')$

[using de morgan's law $(A \cdot B)' = A' + B'$]

Boolean Algebra Laws and Theorems

☐ **Duality Theorem**

- ☐ This theorem states that the dual of the Boolean function is obtained by interchanging the logical AND operator with logical OR operator and zeros with ones.
- ☐ For every Boolean function, there will be a corresponding Dual function.
- ☐ Let us make the Boolean equations (relations) that we discussed in the section of Boolean postulates and basic laws into two groups.
- ☐ The following table shows these two groups

Boolean Algebra Laws and Theorems

Expression	Dual
$1.0 = 0$	$0 + 1 = 1$
$0.1 = 0$	$1 + 0 = 1$
$1 + 0 = 1$	$0.1 = 0$
$0 + 1 = 1$	$1.0 = 0$
$A.0 = 0$	$A + 1 = 1$
$0.A = 0$	$1 + A = 1$
$A.1 = A$	$A + 0 = A$
$1.A = A$	$0 + A = A$
$A.A = A$	$A + A = A$
$A.B = B.A$	$A + B = B + A$
$A.(B.C) = (A.B).C$	$A + (B + C) = (A + B) + C$
$A.(A + B) = A$	$A + A.B = A$
$AB + C + BCA = 0$	$(A + B).C.(B + C + A) = 1$

Boolean Expression Simplification

- ❑ **Example 1:**
- ❑ Simplify: $Y = A \cdot A + A \cdot B$
- ❑ **Solution:**
- ❑ Apply **Idempotent Law**: $Y = A + A \cdot B$
- ❑ Apply **Absorption Law**: $Y = A$ i.e. $A + A \cdot B = A$
- ❑ **Final Simplified Expression:** $Y = A$

Boolean Expression Simplification

- ❑ **Example 2:**
- ❑ Simplify: $Y = AB + A'B + ABC'$
- ❑ **Solution:**
- ❑ Apply **Absorption Law**: $Y = AB + A'B$
- ❑ Apply **Distributive Law**: $Y = B(A + A')$
- ❑ Apply **Complement Law**: $Y = B(1)$
- ❑ Apply **Identity Law**: $Y = B$
- ❑ **Final Simplified Expression**: $Y = B$

Boolean Expression Simplification

- ❑ **Example 3:**
- ❑ Simplify: $Y=(A+B)(A+B')$
- ❑ **Solution:**
- ❑ Apply **Distributive Law**: $Y=A \cdot A + A \cdot B' + B \cdot A + B \cdot B'$
- ❑ Apply **Idempotent Law**: $Y=A + A \cdot B' + B \cdot A + B \cdot B'$
- ❑ Apply **Absorption Law**: $Y=A + B \cdot B'$
- ❑ Apply **Complement Law**: $Y=A + 0$
- ❑ Apply **Identity Law**: $Y=A$
- ❑ **Final Simplified Expression**: $Y=A$

Applications of Boolean Algebra

- ❑ **Applications of Boolean Algebra in Digital Circuits**
- ❑ Boolean Algebra is used to **design and optimize digital circuits**. Some applications include:
 - ❑ **Simplifying Circuit Designs:** Reducing the number of logic gates.
 - ❑ **Developing Arithmetic Circuits:** Adders, subtractors, multipliers.
 - ❑ **Designing Control Circuits:** Traffic lights, microprocessors, ALUs.
 - ❑ **Error Detection and Correction:** Parity bit generation.

Any Question?

Thank you

Digital Logic and Design

Lecture : 06

Department of Computer Science
GPGC SWABI (Khyber Pakhtunkhwa)

Spring 2025 Semester
BS Course: DLD

Course Instructor :
Mr. Ihtisham Ul Haq
Lecturer (CS)
GPGC Swabi

Contents

- ☐ Minterms
- ☐ Maxterms
- ☐ Boolean Function Representation
 - ☐ SOP (Sum of Products)
 - ☐ POS (Product of Sums)
 - ☐ Conversion from SOP to POS
 - ☐ Conversion from POS to SOP
 - ☐ Canonical Form
 - ☐ Standard Form

Boolean Algebra

- ❑ In digital logic design, understanding the concepts of **minterms**, **maxterms**, **Sum of Products (SOP)**, and **Product of Sums (POS)** is crucial for simplifying and implementing Boolean functions efficiently.
- ❑ These standard forms facilitate the systematic analysis and synthesis of logical expressions, leading to optimized digital circuits.

Minterms

- ❑ A **minterm** is a specific type of **product (AND) term** in a Boolean function where each variable appears exactly once, either in its **true form or complemented form**.
- ❑ A minterm evaluates to **1** for a unique combination of variable states and to **0** for all other combinations.
- ❑ Each minterm is represented by m (small letter).
- ❑ For 0 input literal is represented in complemented form.
- ❑ For n variables there must be 2^n possible minterms.
- ❑ **Example**
- ❑ Consider a Boolean function with two variables, A and B :
 - ❑ The minterm for $A=0$ and $B=1$ is $A'B$
 - ❑ The minterm for $A=1$ and $B=0$ is AB'

Minterms

- ❑ **Truth Table with Minterms**
- ❑ Each minterm corresponds to a row in the truth table where the given function output is **1**.

A	B	Minterm	Minterm Notation
0	0	$A'B'$	m0
0	1	$A'B$	m1
1	0	AB'	m2
1	1	AB	m3

Maxterms

- ❑ A **maxterm** is a specific type of **sum (OR) term** in a Boolean function where each variable appears exactly once, either in its **true form or complemented form**.
- ❑ A maxterm evaluates to **0** for a unique combination of variable states and to **1** for all other combinations.
- ❑ Each maxterm is represented by M (capital letters).
- ❑ For 1 input literal is represented in complemented form.
- ❑ For n variables there must be 2^n possible Maxterms.
- ❑ **Example**
- ❑ Consider a Boolean function with two variables, A and B:
 - ❑ The maxterm for A=0 and B=1 is $A+B'$
 - ❑ The maxterm for A=1 and B=0 is $A'+B$

Maxterms

- ❑ **Truth Table with Maxterms**
- ❑ Each maxterm corresponds to a row in the truth table where the given function output is 0.

A	B	Maxterm	Minterm Notation
0	0	$A+B$	M0
0	1	$A+B'$	M1
1	0	$A'+B$	M2
1	1	$A'+B'$	M3

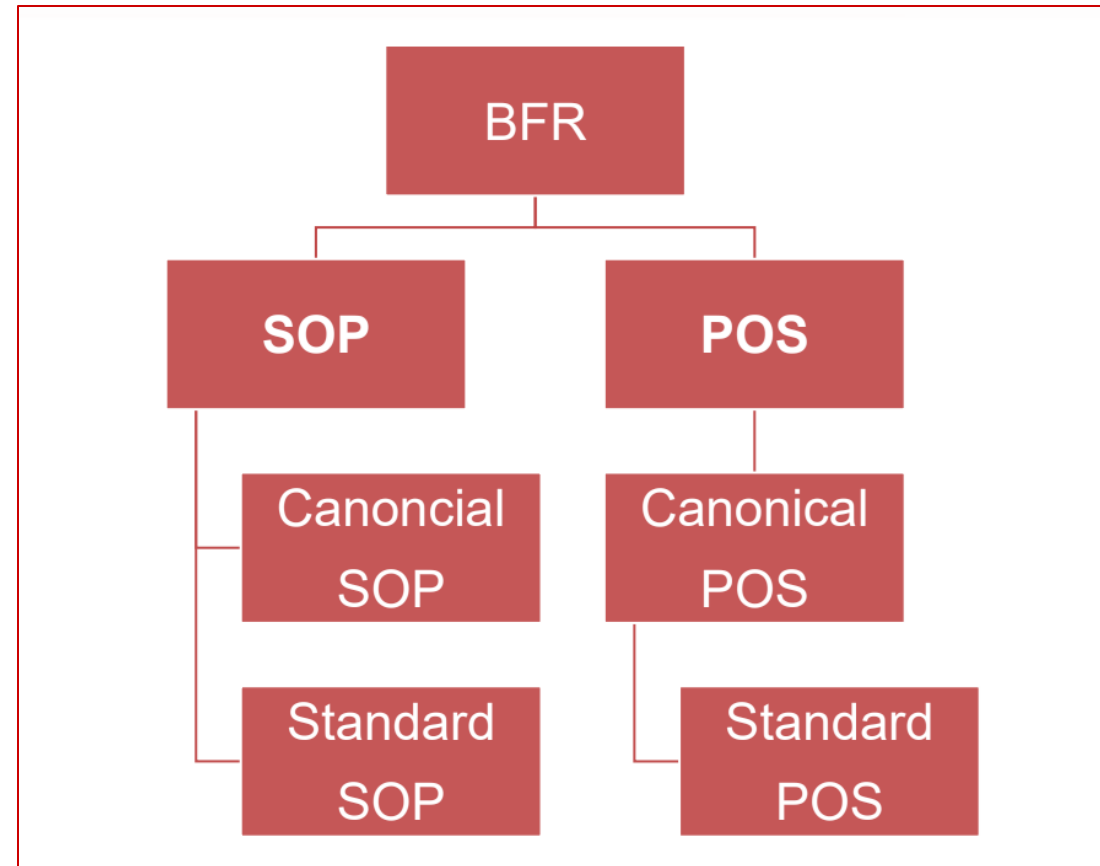
Minterms Vs Maxterms

□ Each minterm is the complement of its corresponding maxterm.

1) $m_i = (M_i)'$

2) $M_i = (m_i)'$

Boolean Function Representation



Sum of Products/Sum of minterms

- ❑ The **Sum of Products (SOP)** form is a Boolean expression format where multiple product (AND) terms are summed (ORed) together.
- ❑ It is also called Disjunctive Normal form.

Product of Sums/ Product of maxterms

- ❑ The **Product of Sums (POS)** form is a Boolean expression format where multiple sum (OR) terms are multiplied (ANDed) together.
- ❑ It is also called Conjunctive Normal form.

Canonical Form

- ☐ In canonical representation each term contain all the literals.
- ☐ It is directly noted from the truth table.
- ☐ **Types**
 - ☐ Canonical SOP
 - ☐ Canonical POS
- ☐ Both canonical SOP and canonical POS forms are Dual to each other.

Sum of Products (Canonical SOP)

- ❑ The **Sum of Products (SOP)** form is a Boolean expression format where multiple product (AND) terms are summed (ORed) together.
- ❑ Each product term represents a minterm corresponding to a combination of variables that make the function true.
- ❑ In this form, each product term contains all the literals.
- ❑ Directly read from the truth Table.
- ❑ So, these product terms are nothing but the min terms.
- ❑ Hence, canonical SoP form is also called as sum of min terms form.

Sum of Products (Canonical SOP)

☐ Procedure

- ☐ First, identify the min terms for which, the output variable is 1.
- ☐ Then do the logical OR of those min terms in order to get the Boolean expression (function) corresponding to that output variable.
- ☐ This Boolean function will be in the form of sum of min terms.

☐ Example

- ☐ Given a function $F(A,B,C)$ that is true for minterms m_1 , m_4 , and m_6 :
- ☐ $F(A,B,C) = A'BC + AB'C' + ABC'$
- ☐ $F(A,B,C) = m_1 + m_4 + m_6$
- ☐ $F(A,B,C) = \sum(m_1, m_4, m_6)$

Sum of Products (Canonical SOP)

❑ Truth Table for SOP

❑ In this table, the function F is 1 for the minterms m_1 , m_4 , and m_6 .

A	B	C	$F(A, B, C)$
0	0	0	0
0	0	1	1
0	1	0	0
0	1	1	0
1	0	0	1
1	0	1	0
1	1	0	1
1	1	1	0

Product of Sums (Canonical POS)

- ❑ The **Product of Sums (POS)** form is a Boolean expression format where multiple sum (OR) terms are multiplied (ANDed) together.
- ❑ Each sum term represents a maxterm corresponding to a combination of variables that make the function false.
- ❑ In this form, each sum term contains all literals.
- ❑ So, these sum terms are nothing but the Max terms.
- ❑ Hence, canonical PoS form is also called as product of Max terms form.

Product of Sums (Canonical POS)

☐ Procedure

- ☐ First, identify the Max terms for which, the output variable is zero.
- ☐ Then do the logical AND of those Max terms in order to get the Boolean expression (function) corresponding to that output variable.
- ☐ This Boolean function will be in the form of product of Max terms.

☐ Example

- ☐ For the same function $F(A,B,C)$, the POS form can be derived from the max terms where the function is false:
- ☐ $F(A,B,C)=(A+B+C)(A+B'+C)(A'+B+C)$
- ☐ $F(A,B,C)=M1 \cdot M4 \cdot M6$
- ☐ $F(A,B,C)=\prod (M1, M4, M6)$

Product of Sums (Canonical POS)

❑ Truth Table for POS

❑ In this table, the function F is 0 for the maxterms M_1 , M_4 , and M_6 .

A	B	C	F(A, B, C)
0	0	0	1
0	0	1	0
0	1	0	1
0	1	1	1
1	0	0	0
1	0	1	1
1	1	0	0
1	1	1	1

Conversion from SOP to POS

- ❑ $F(\text{SOP}) = F(\text{POS})$
- ❑ $(\text{Canonical } F(\text{SOP})')' = F(\text{POS})$

- ❑ **Steps**
 - ❑ Find the F' for given function F (which converts 0s to 1s).
 - ❑ Express F' in sum of minterms.
 - ❑ Take complement of F' using Demorgan's law.

Conversion from SOP to POS

- ❑ $F(A,B,C) = \sum(1,4,5,6,7) = m_1 + m_4 + m_5 + m_6 + m_7$
- ❑ $F'(A,B,C) = \sum(0,2,3) = m_0 + m_2 + m_3$
- ❑ Taking complement using Demorgan's Theorem
- ❑ $[F'(A,B,C)]' = (m_0 + m_2 + m_3)'$
- ❑ $F(A,B,C) = m_0' \cdot m_2' \cdot m_3'$
- ❑ $F(A,B,C) = M_0 \cdot M_2 \cdot M_3 \quad [m_i' = M_i]$
- ❑ $F(A,B,C) = \prod(0,2,3)$

A	B	C	F	F'
0	0	0	0	1
0	0	1	1	0
0	1	0	0	1
0	1	1	0	1
1	0	0	1	0
1	0	1	1	0
1	1	0	1	0
1	1	1	1	0

Conversion from POS to SOP

- ❑ $F(\text{POS}) = F(\text{SOP})$
- ❑ $(\text{canonical } F(\text{POS}))' = F(\text{SOP})$

- ❑ **Steps**
 - ❑ Find the F' for given function F (which converts 0s to 1s).
 - ❑ Express F' in product of maxterms.
 - ❑ Take complement of F' using demorgan's law.

Conversion from POS to SOP

- ❑ $F(A,B,C) = \prod(0,2,3) = M_0.M_2.M_3$
- ❑ $F'(A,B,C) = \prod(1,4,5,6,7) = M_1.M_4.M_5.M_6.M_7$
- ❑ Taking complement using Demorgan's Theorem
- ❑ $[F'(A,B,C)]' = (M_1.M_4.M_5.M_6.M_7)'$
- ❑ $F(A,B,C) = M_1' + M_4' + M_5' + M_6' + M_7'$
- ❑ $F(A,B,C) = m_1 + m_4 + m_5 + m_6 + m_7 \quad [m_i' = M_i]$
- ❑ $F(A,B,C) = \sum(1,4,5,6,7)$

A	B	C	F	F'
0	0	0	0	1
0	0	1	1	0
0	1	0	0	1
0	1	1	0	1
1	0	0	1	0
1	0	1	1	0
1	1	0	1	0
1	1	1	1	0

Standard Form

- ❑ In standard form each term does not contain all the literals.
- ❑ This is simplified form of the canonical form.
- ❑ **Advantages**
 1. The main advantage of standard forms is that the number of inputs applied to logic gates can be minimized.
 2. Sometimes, there will be reduction in the total number of logic gates required.
- ❑ **Types**
 - ❑ Standard SOP
 - ❑ Standard POS

Standard SOP

- ❑ In this form, each product term need not contain all literals.
- ❑ The Standard SOP form is the simplified form of canonical SOP form.

- ❑ **Procedure**
- ❑ Get the canonical SOP form of output variable.
- ❑ Simplify the above Boolean function, which is in canonical SOP form.

Standard SOP

- ❑ **Convert the following Boolean function into Standard SoP form.**
- ❑ $f = p'qr + pq'r + pqr' + pqr$
- ❑ The given Boolean function is in canonical SoP form. Now, we have to simplify this Boolean function in order to get standard SoP form.
- ❑ Step 1 – Use the Boolean postulate, $x + x = x$. That means, the Logical OR operation with any Boolean variable 'n' times will be equal to the same variable. So, we can write the last term pqr two more times.
- ❑ $\Rightarrow f = p'qr + pq'r + pqr' + pqr + pqr + pqr$
- ❑ Step 2 – Use Distributive law for 1st and 4th terms, 2nd and 5th terms, 3rd and 6th terms.
- ❑ $\Rightarrow f = qr(p' + p) + pr(q' + q) + pq(r' + r)$
- ❑ Step 3 – Use Boolean postulate, $x + x' = 1$ for simplifying the terms present in each parenthesis.
- ❑ $\Rightarrow f = qr(1) + pr(1) + pq(1)$
- ❑ Step 4 – Use Boolean postulate, $x.1 = x$ for simplifying above three terms.
- ❑ $\Rightarrow f = qr + pr + pq$
- ❑ $\Rightarrow f = pq + qr + pr$
- ❑ This is the simplified Boolean function. Therefore, the standard SoP form corresponding to given canonical SoP form is $f = pq + qr + pr$.

Standard POS

- ❑ In this form, each sum term need not contain all literals.
- ❑ The Standard POS form is the simplified form of canonical POS form.
- ❑ **Procedure**
- ❑ Get the canonical POS form of output variable.
- ❑ Simplify the above Boolean function, which is in canonical POS form.

Standard POS

- ❑ **Convert the following Boolean function into Standard PoS form.**
- ❑ $f = (p + q + r).(p + q + r').(p + q' + r).(p' + q + r)$
- ❑ The given Boolean function is in canonical PoS form. Now, we have to simplify this Boolean function in order to get standard PoS form.
- ❑ Step 1 – Use the Boolean postulate, $x.x = x$. That means, the Logical AND operation with any Boolean variable ‘n’ times will be equal to the same variable. So, we can write the first term $p+q+r$ two more times.
 $\Rightarrow f = (p + q + r).(p + q + r).(p + q + r).(p + q + r').(p + q' + r).(p' + q + r)$
- ❑ Step 2 – Use Distributive law, $x + (y.z) = (x + y).(x + z)$ for 1st and 4th parenthesis, 2nd and 5th parenthesis, 3rd and 6th parenthesis.
 $\Rightarrow f = (p + q + rr').(p + r + qq').(q + r + pp')$
- ❑ Step 3 – Use Boolean postulate, $x.x'=0$ for simplifying the terms present in each parenthesis.
 $\Rightarrow f = (p + q + 0).(p + r + 0).(q + r + 0)$
- ❑ Step 4 – Use Boolean postulate, $x + 0 = x$ for simplifying the terms present in each parenthesis.
 $\Rightarrow f = (p + q).(p + r).(q + r)$
- ❑ $\Rightarrow f = (p + q).(q + r).(p + r)$
- ❑ This is the simplified Boolean function. Therefore, the standard PoS form corresponding to given canonical PoS form is $f = (p + q).(q + r).(p + r)$. This is the dual of the Boolean function, $f = pq + qr + pr$.
- ❑ Therefore, both Standard SoP and Standard PoS forms are Dual to each other.

Canonical Form Vs Standard Form

Canonical Form	Standard Form
Each term contains all the literals	Each term does not contain all the literals
Directly read from Truth Table	Simplified Form of canonical form
Inefficient for circuit construction	Efficient for circuit construction
Types: Canonical SOP and Canonical POS	Types: Standard SOP and Standard POS

Any Question?

Thank you

Digital Logic and Design

Lecture : 07

Department of Computer Science
GPGC SWABI (Khyber Pakhtunkhwa)

Spring 2025 Semester
BS Course: DLD

Course Instructor :
Mr. Ihtisham Ul Haq
Lecturer (CS)
GPGC Swabi

Contents

- ☐ Karnaugh Map (K-Map)
- ☐ Simplification using K-Map
 - ☐ K-Map for SOP
 - ☐ K-Map for POS

Karnaugh Map (K-Map)

- ❑ A **Karnaugh Map (K-Map)** is a graphical method used for simplifying Boolean expressions and designing minimal logic circuits.
- ❑ It is an alternative to **algebraic simplifications** and the **Quine-McCluskey** method.

Karnaugh Map (K-Map)

- ❑ A K-Map is a **grid-based** representation of a **truth table**, where each cell represents a minterm of the Boolean function.
- ❑ The arrangement of cells ensures that adjacent cells differ by only **one** variable (Gray Code ordering).

No. of Variables	K-Map Size
2 Variables	2×2 (4 cells)
3 Variables	2×4 (8 cells)
4 Variables	4×4 (16 cells)
5 Variables	4×8 (32 cells)
6 Variables	8×8 (64 cells)

Steps to Solve Expression using K-map

- 1) Select the K-map according to the number of variables.
- 2) Identify minterms or maxterms as given in the problem.
- 3) For SOP put 1's in blocks of K-map respective to the minterms (0's elsewhere).
- 4) For POS put 0's in blocks of K-map respective to the max terms (1's elsewhere).
- 5) Make rectangular groups containing total terms in power of two like 2,4,8 ..(except 1) and try to cover as many elements as you can in one group.
- 6) From the groups made in step 5 find the product terms and sum them up for SOP form.

Karnaugh Map (K-Map)

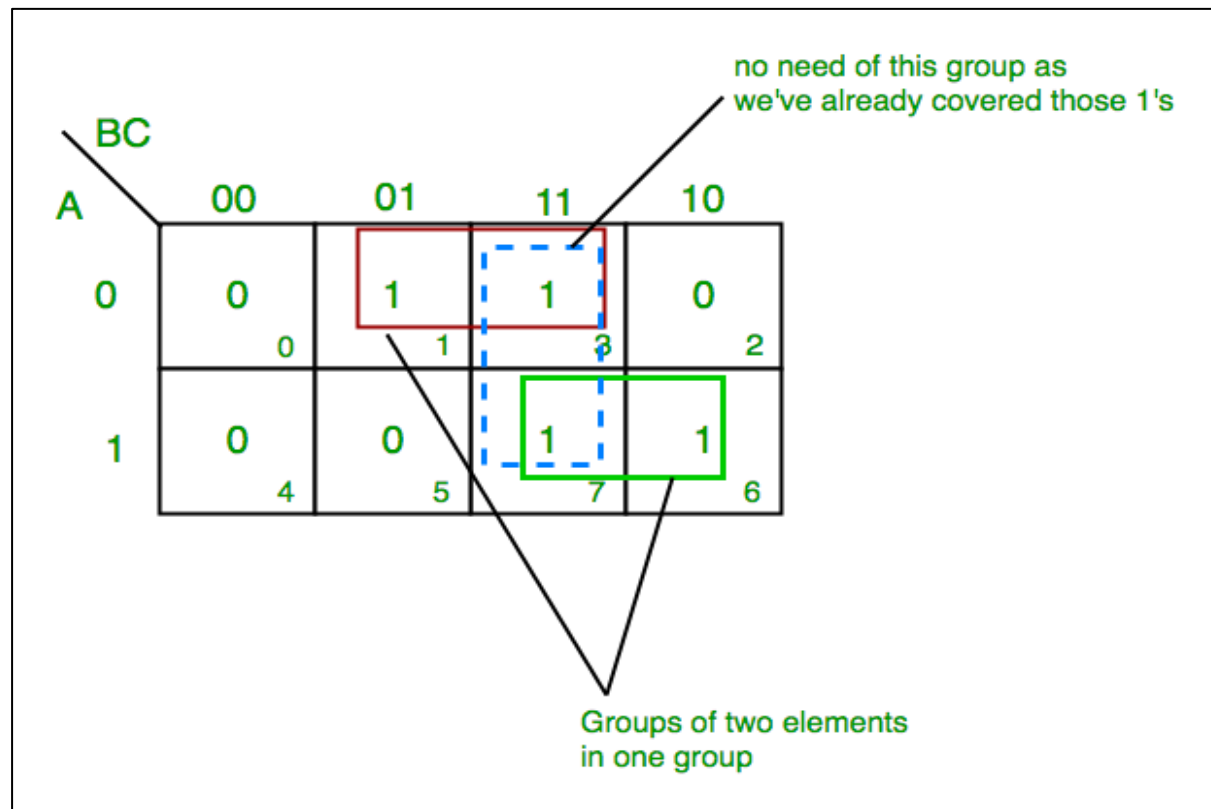
❑ K-Map SOP From (3 Variables)

		BC			
		B'C'	B'C	BC	BC'
A	0	A'B'C'	A'B'C	A'BC	A'BC'
	1	AB'C'	AB'C	ABC	ABC'

SOP(MINTERMS)

Karnaugh Map (K-Map)

□ $Z = \sum A, B, C(1, 3, 6, 7)$



Karnaugh Map (K-Map)

- ❑ From red group we get product term, $A'C$.
- ❑ From green group we get product term, AB .
- ❑ Summing these product terms we get the Final expression = $(A'C+AB)$.

Karnaugh Map (K-Map)

❑ K-Map SOP From (4 Variables)

		CD	C'D'	C'D	CD	CD'
		00	01	11	10	
AB						
A'B'	00	A'B'C'D'	A'B'C'D	A'B'CD	A'B'CD'	
A'B	01	A'BC'D'	A'BC'D	A'BCD	A'BCD'	
AB	11	ABC'D'	ABC'D	ABCD	ABCD'	
AB'	10	AB'C'D'	AB'C'D	AB'CD	AB'CD'	

SOP(MINTERMS)

Karnaugh Map (K-Map)

- ❑ Task 01:
- ❑ $F(P,Q,R,S)=\Sigma(0,2,5,7,8,10,13,15)$
- ❑ From red group we get product term, QS .
- ❑ From green group we get product term, $Q'S'$.
- ❑ Summing these product terms we get- Final expression $(QS+Q'S')$.

Karnaugh Map (K-Map)

❑ K-Map POS From (3 Variables)

		BC			
		B+C 00	B+C' 01	B'+C' 11	B'+C 10
A	0	A+B+C 0	A+B+C' 1	A+B'+C' 3	A+B'+C 2
	1	A'+B+C 4	A'+B+C' 5	A'+B'+C' 7	A'+B'+C 6

POS (MAXTERMS)

Karnaugh Map (K-Map)

□ $F(A,B,C)=?(0,3,6,7)$

		BC			
		00	01	11	10
A	0	0	1	0	1
	1	1	1	0	0

Karnaugh Map (K-Map)

- ❑ From red group we find terms, $(A' + B')$.
- ❑ From brown group we find terms, $(B' + C')$.
- ❑ From yellow group we find terms, $(A + B + C)$.
- ❑ We will take product of these three terms. Final expression is:
- ❑ $(A' + B')(B' + C')(A + B + C)$

Karnaugh Map (K-Map)

❑ K-Map POS From (4 Variables)

		CD	C+D	C+D'	C'+D'	C'+D
AB		00	01	11	10	
A = B	00	A+B+C+D 0	A+B+C+D'	A+B+C'+D'	A+B+C'+D 2	
A = B'	01	A+B'+C+D 4	A+B'+C+D'	A+B'+C'+D'	A+B'+C'+D 6	
A'+B	11	A'+B'+C+D 12	A'+B'+C+D'	A'+B'+C'+D'	A'+B'+C'+D 14	
A'+B'	10	A'+B+C+D 8	A'+B+C+D'	A'+B+C'+D'	A'+B+C'+D 10	

POS{MAXTERMS}

Karnaugh Map (K-Map)

- ❑ Task 02:
- ❑ $F(A,B,C,D)=?(3,5,7,8,10,11,12,13)$

		CD			
		00	01	11	10
AB	00	1 0	1 1	0 3	1 2
	01	1 4	0 5	0 7	1 6
	11	0 12	0 13	1 15	1 14
	10	0 8	1 9	0 11	0 10

Karnaugh Map (K-Map)

- ❑ From green group we find terms, $(B'+C+D')$.
- ❑ From red group we find terms, $(A+C'+D')$.
- ❑ From blue group we find terms, $(A'+C+D)$.
- ❑ From brown group we find terms, $(A'+B+C')$.
- ❑ Finally we express these as product:
- ❑ $(B'+C+D').(A+C'+D').(A'+C+D).(A'+B+C')$

Benefits of Karnaugh Map (K-Map)

- ❑ Simplifies Boolean Expressions – Quickly reduces Boolean functions to their minimal form.
- ❑ Minimizes Logic Gates – Leads to optimized circuit design with fewer logic gates.
- ❑ Easy to Use – Provides a visual method for simplification, making it more intuitive than algebraic methods.
- ❑ Eliminates Redundant Terms – Ensures that only essential terms remain in the final expression.
- ❑ Fast and Efficient – Significantly quicker than methods like the Quine-McCluskey for small functions.
- ❑ Useful for Fault Detection – Helps in troubleshooting and optimizing digital circuits.
- ❑ Useful for designing combinational circuits like adders, multiplexers, and encoders.
- ❑ Can be used to simplify expressions in both Sum of Products (SOP) and Product of Sums (POS) formats.

Any Question?

Thank you

Digital Logic and Design

Lecture : 08

Department of Computer Science
GPGC SWABI (Khyber Pakhtunkhwa)

Spring 2025 Semester
BS Course: DLD

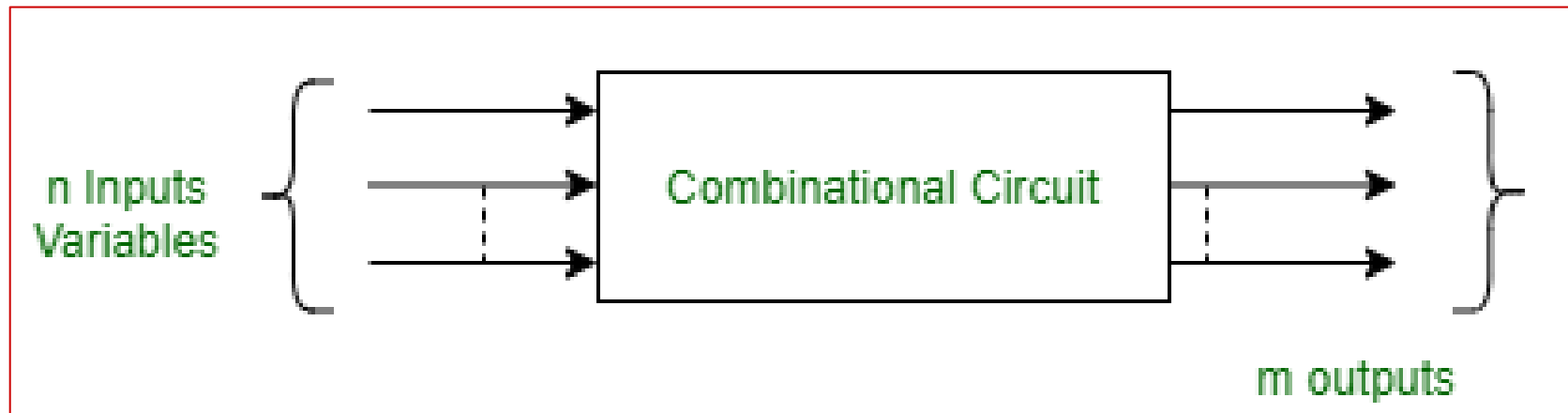
Course Instructor :
Mr. Ihtisham Ul Haq
Lecturer (CS)
GPGC Swabi

Contents

- ☐ Combinational Logic Circuits
- ☐ Adders
 - ☐ Half Adders
 - ☐ Full Adders
- ☐ Subtractors
 - ☐ Half Subtractors
 - ☐ Full Subtractors

Combinational Logic Circuits

- ❑ Combinational logic circuits are **digital circuits** where the output depends only on the present inputs and **not on past inputs** (no memory elements).
- ❑ These circuits use basic logic gates (AND, OR, NOT) to perform logical operations.



Features of Combinational Circuit

- 1) In this output depends only upon present input.
- 2) It's Speed is fast.
- 3) Easy designed.
- 4) There is no feedback between input and output.
- 5) It is time independent.
- 6) Elementary building blocks are Logic gates.
- 7) Used for both arithmetic and boolean operations.
- 8) Combinational circuits don't have the capability to store any state.

Types of Combinational Logic Circuits

- ❑ **Adders** – Perform binary addition.
 - ❑ **Half Adder** (adds two bits).
 - ❑ **Full Adder** (adds three bits including carry).
- ❑ **Subtractors** – Perform binary subtraction.
 - ❑ **Half Subtractor** (subtracts two bits).
 - ❑ **Full Subtractor** (subtracts three bits including borrow).
- ❑ **Multiplexers (MUX)** – Selects one input from multiple inputs based on a control signal.
- ❑ **Demultiplexers (DEMUX)** – Sends one input to multiple outputs based on a control signal.

Types of Combinational Logic Circuits

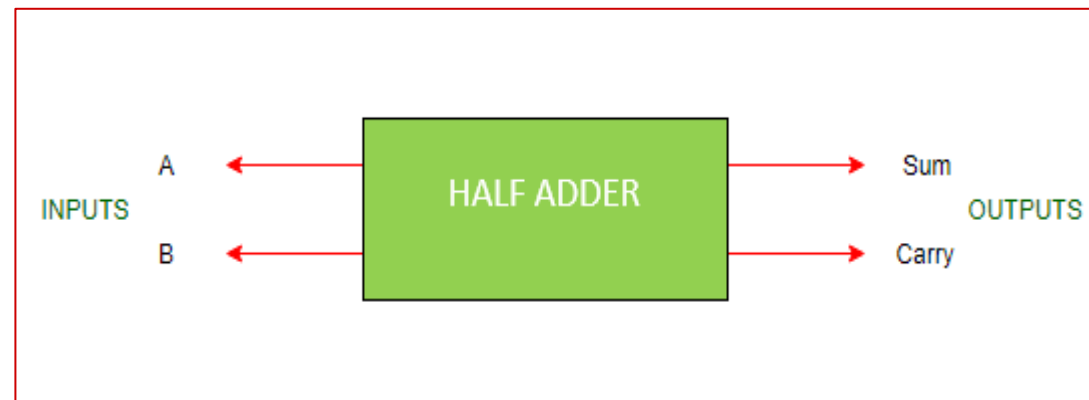
- ❑ **Encoders** – Converts multiple inputs into a smaller number of outputs (e.g., 8-to-3 encoder).
- ❑ **Decoders** – Converts smaller inputs into multiple outputs (e.g., 3-to-8 decoder).
- ❑ **Comparators** – Compares two binary numbers and outputs comparison results (greater, smaller, equal).
- ❑ **Code Converters** – Converts data from one format to another (e.g., Binary to Gray Code, BCD to Excess-3).

Adders

- ❑ Adders are combinational logic circuits used for **binary addition**.
- ❑ They are fundamental in arithmetic logic units (ALUs), processors, and digital systems.
- ❑ There are two types of adders:
 - ❑ Half Adder
 - ❑ Full Adder

Half Adder

- ❑ A half adder adds **two single-bit binary numbers** and produces a **Sum (S)** and a **Carry (C)**.
- ❑ It is an arithmetic combinational logic circuit designed to perform addition of two single bits.
- ❑ It contains two inputs and produces two outputs.
- ❑ Inputs are called Augend and Added bits and Outputs are called Sum and Carry.



Half Adder

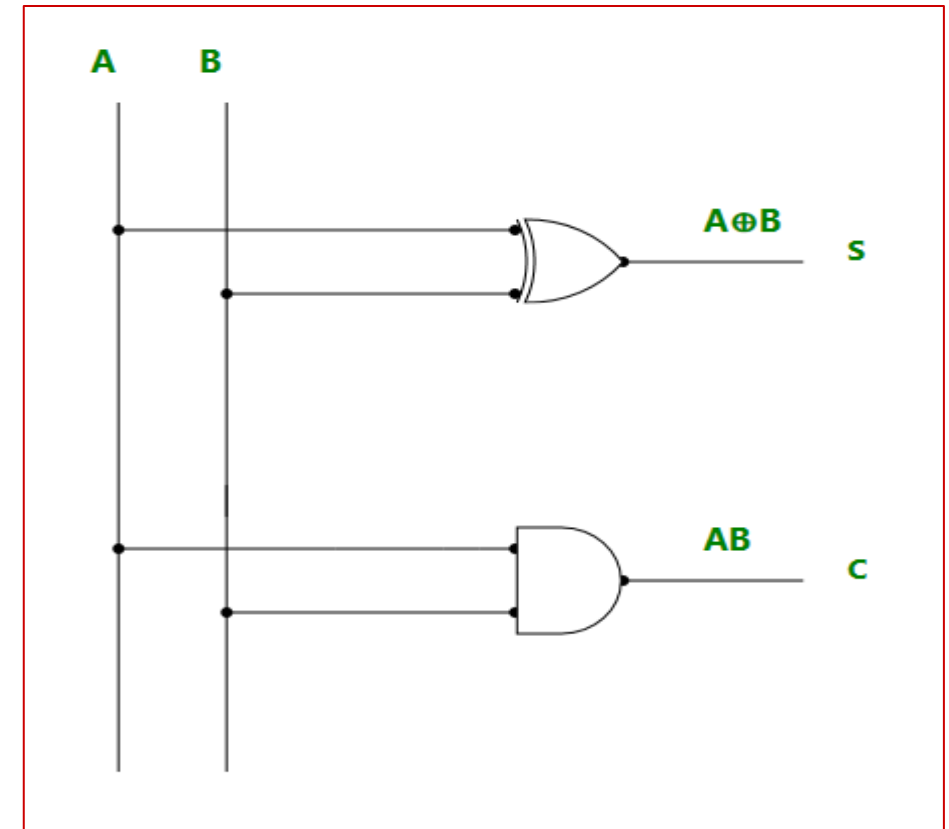
- ❑ **Boolean Equations:**
- ❑ **Sum (S) = $A \oplus B$** (XOR gate)
- ❑ **Carry (C) = $A \cdot B$** (AND gate)

Truth Table (Half Adder)

Inputs		Outputs	
A	B	Sum (S)	Carry (C)
0	0	0	0
0	1	1	0
1	0	1	0
1	1	0	1

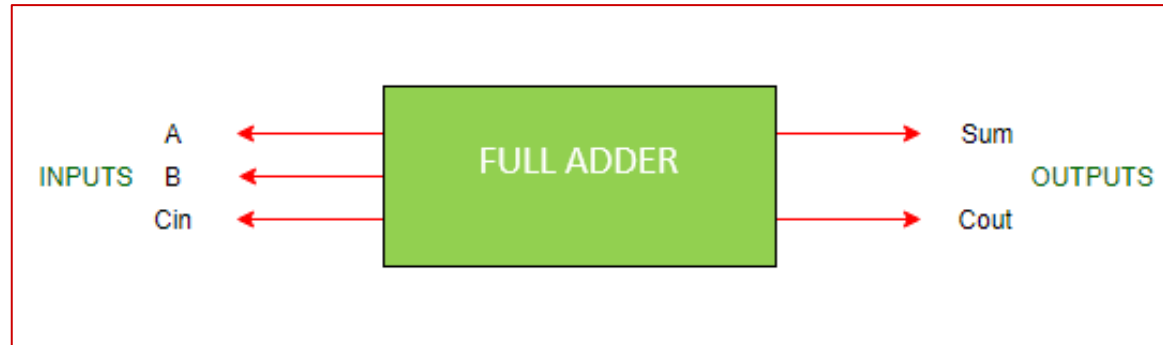
Half Adder

- ❑ **Logic Circuit:**
 - ❑ A half adder consists of:
 - ❑ XOR gate for Sum
 - ❑ AND gate for Carry
-
- ❑ **Limitations:** Adding of Carry is not possible in Half adder.



Full Adder

- ❑ To overcome the above limitation faced with Half adders, Full Adders are implemented.
- ❑ It is an arithmetic combinational logic circuit that performs **addition of three single bits**.
- ❑ It contains three **inputs (A, B, C_{in})** and produces two outputs (**Sum and C_{out}**).
- ❑ Where, C_{in} -> **Carry In** and C_{out} -> **Carry Out**.



Full Adder

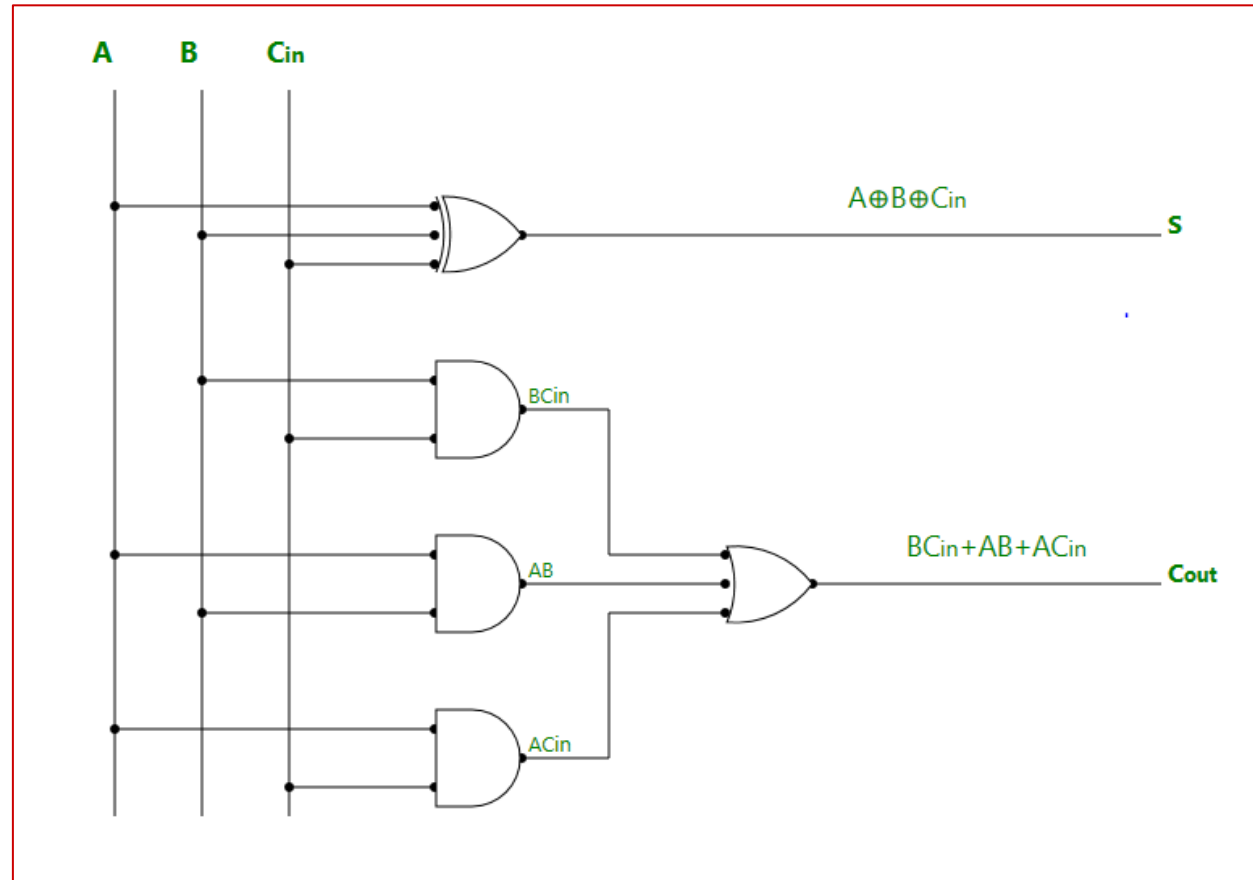
- **Boolean Equations:**
- **Sum (S) = $A \oplus B \oplus C_{in}$** (XOR gates)
- **Carry (C_{out}) = $AB + AC_{in} + BC_{in}$** (AND, OR gates)

Truth Table (Full Adder)

Inputs			Outputs	
A	B	C_{in}	Sum (S)	Carry (C_{out})
0	0	0	0	0
0	0	1	1	0
0	1	0	1	0
0	1	1	0	1
1	0	0	1	0
1	0	1	0	1
1	1	0	0	1
1	1	1	1	1

Full Adder

- ❑ **Logic Circuit:**
- ❑ A half adder consists of:
- ❑ XOR gate for Sum
- ❑ AND, OR gate for Carry

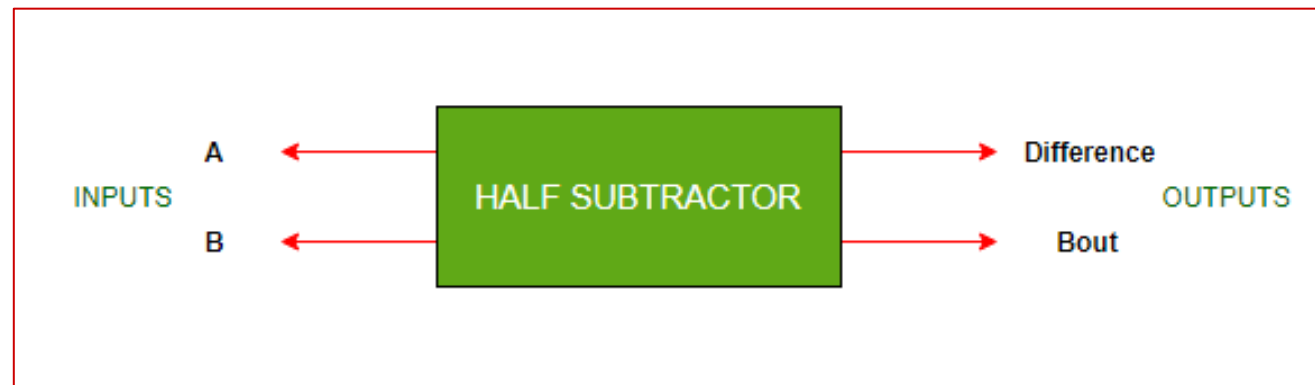


Subtractors

- ❑ A **subtractor** is a combinational logic circuit that performs binary subtraction.
- ❑ It is a fundamental arithmetic unit in processors and ALUs (Arithmetic Logic Units).
- ❑ There are two types of adders:
 - ❑ Half Subtractor
 - ❑ Full Subtractor

Half Subtractor

- ❑ A half subtractor subtracts two single-bit binary numbers and produces:
 - ❑ **Difference (D)**
 - ❑ **Borrow (B)** (Indicates if borrowing is required from the next higher bit)



Half Subtractor

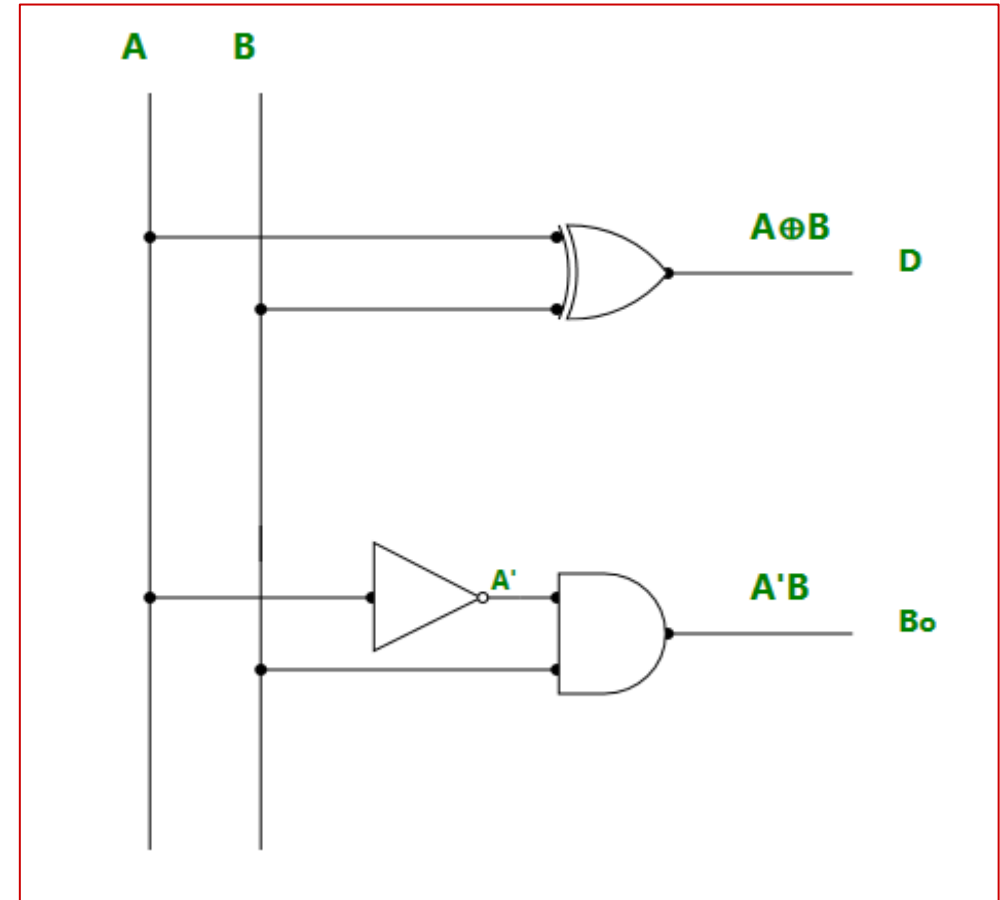
- ❑ **Boolean Expressions**
- ❑ **Difference (D) = $A \oplus B$** (XOR gate)
- ❑ **Borrow (B) = $A' \cdot B$** (NOT-AND combination)

Truth Table (Half Subtractor)

Inputs		Outputs	
A	B	Difference (D)	Borrow (B_{out})
0	0	0	0
0	1	1	1
1	0	1	0
1	1	0	0

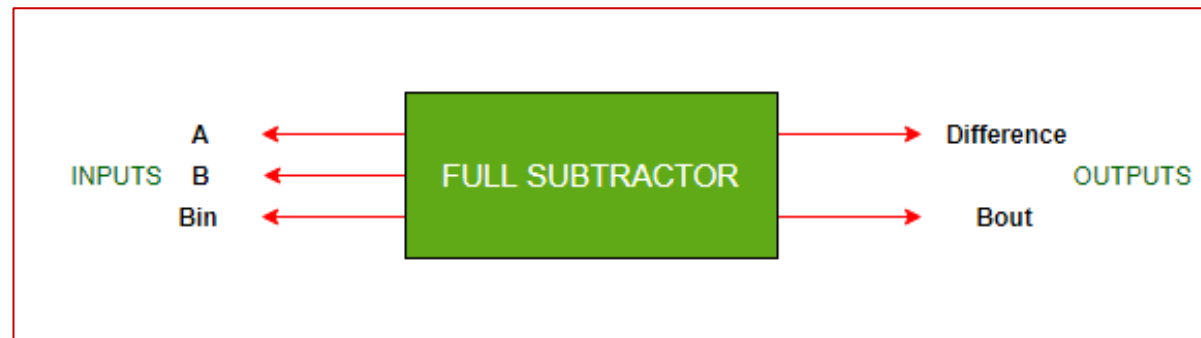
Half Subtractor

- ❑ **Logic Circuit**
- ❑ A half subtractor consists of:
- ❑ XOR gate for Difference
- ❑ NOT and AND gates for Borrow



Full Subtractor

- ❑ It is a Combinational logic circuit designed to perform subtraction of three single bits.
- ❑ It contains three inputs(A, B, B_{in}) and produces two outputs (D, B_{out}).
- ❑ Where, A and B are called **Minuend** and **Subtrahend** bits.
- ❑ And, B_{in} -> Borrow-In and B_{out} -> Borrow-Out



Full Subtractor

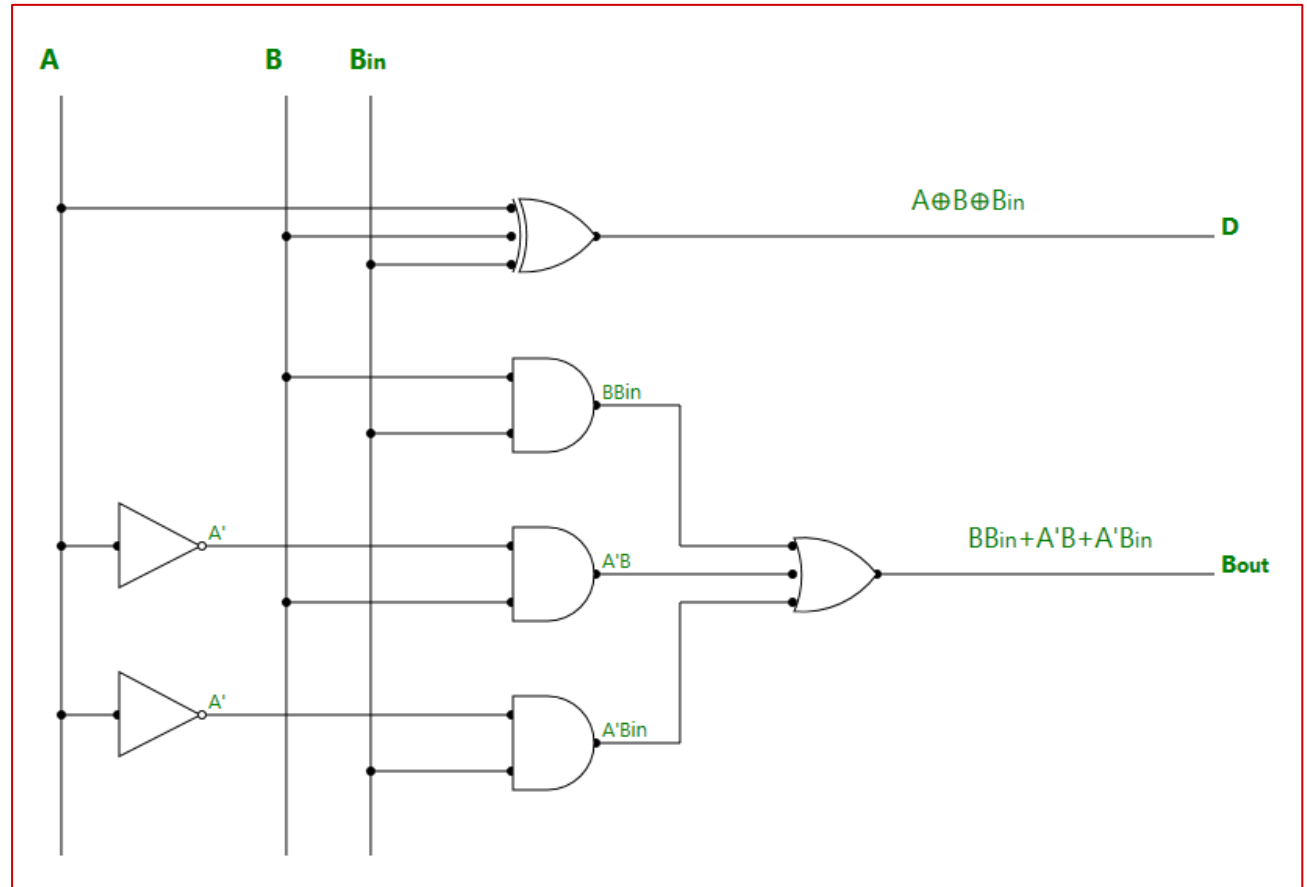
- **Boolean Expressions**
- **Difference (D)** = $A \oplus B \oplus B_{in}$ (XOR gate)
- **Borrow (B_{out})** = $A'B + A'B_{in} + BB_{in}$
(NOT-AND-OR combination)

Truth Table (Full Subtractor)

Inputs			Outputs	
A	B	B _{in}	D	B _{out}
0	0	0	0	0
0	0	1	1	0
0	1	0	1	0
0	1	1	0	1
1	0	0	1	0
1	0	1	0	1
1	1	0	0	1
1	1	1	1	1

Full Subtractor

- ❑ **Logic Circuit**
- ❑ A full subtractor consists of:
- ❑ XOR gate for Difference
- ❑ NOT, AND, and OR gates for Borrow



Applications

1. For performing arithmetic calculations in electronic calculators and other digital devices.
2. In Timers and Program Counters.
3. Useful in Digital Signal Processing.

Any Question?

Thank you

Digital Logic and Design

Lecture : 09

Department of Computer Science
GPGC SWABI (Khyber Pakhtunkhwa)

Spring 2025 Semester
BS Course: DLD

Course Instructor :
Mr. Ihtisham Ul Haq
Lecturer (CS)
GPGC Swabi

Contents

- ❑ Multiplexers (MUX)
 - ❑ Advantages and Disadvantages
- ❑ DE Multiplexers (DEMUX)
 - ❑ Advantages and Disadvantages
- ❑ Encoders
 - ❑ 3x8 Encoder (Truth Table and Implementation)
- ❑ Decoders
 - ❑ 8x3 Decoder (Truth Table and Implementation)

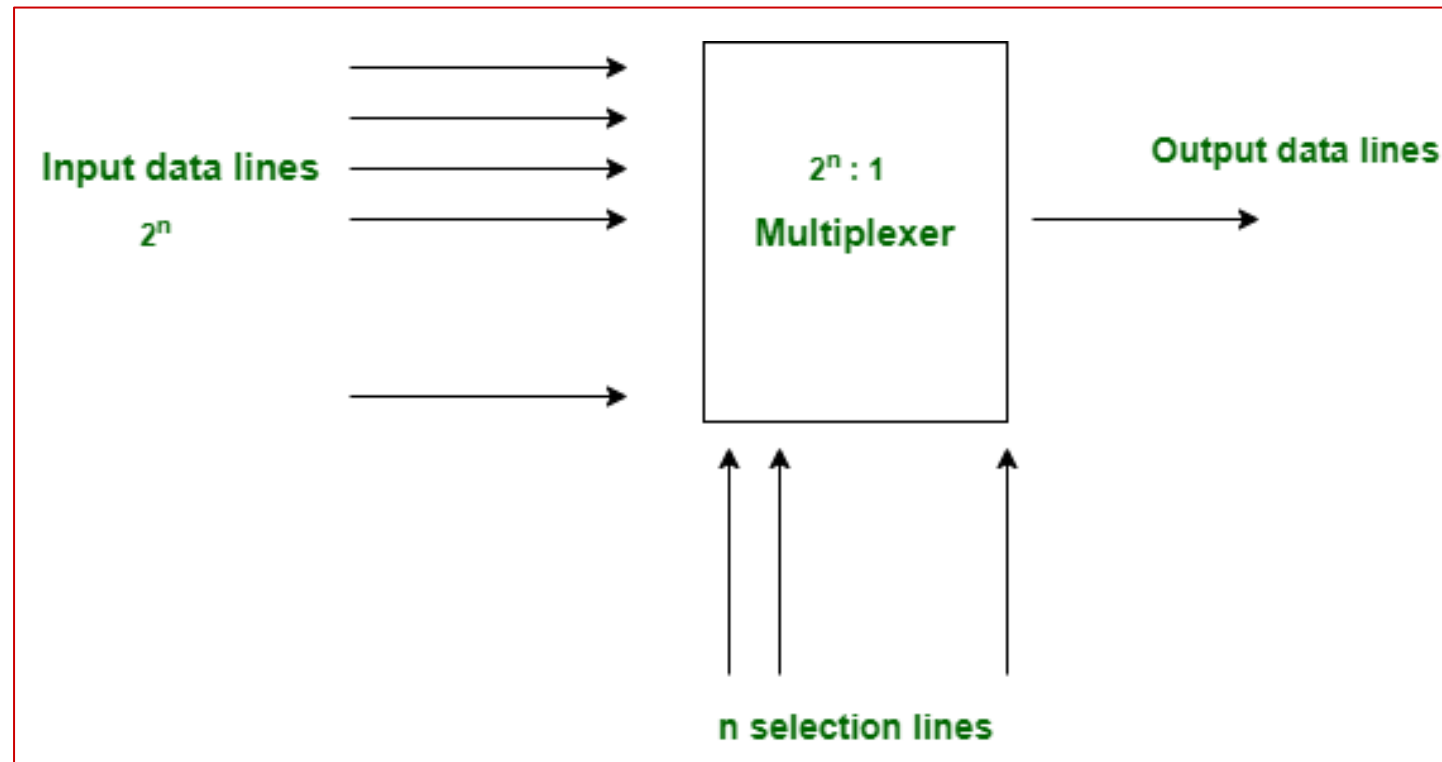
Multiplexers (MUX) and DE Multiplexers (DEMUX)

- ❑ A Multiplexer (MUX) and a Demultiplexer (DEMUX) are essential digital circuits in communication systems, performing opposite functions.
- ❑ A multiplexer combines multiple input signals into a single output, while a demultiplexer takes a single input signal and routes it to one of many output lines.

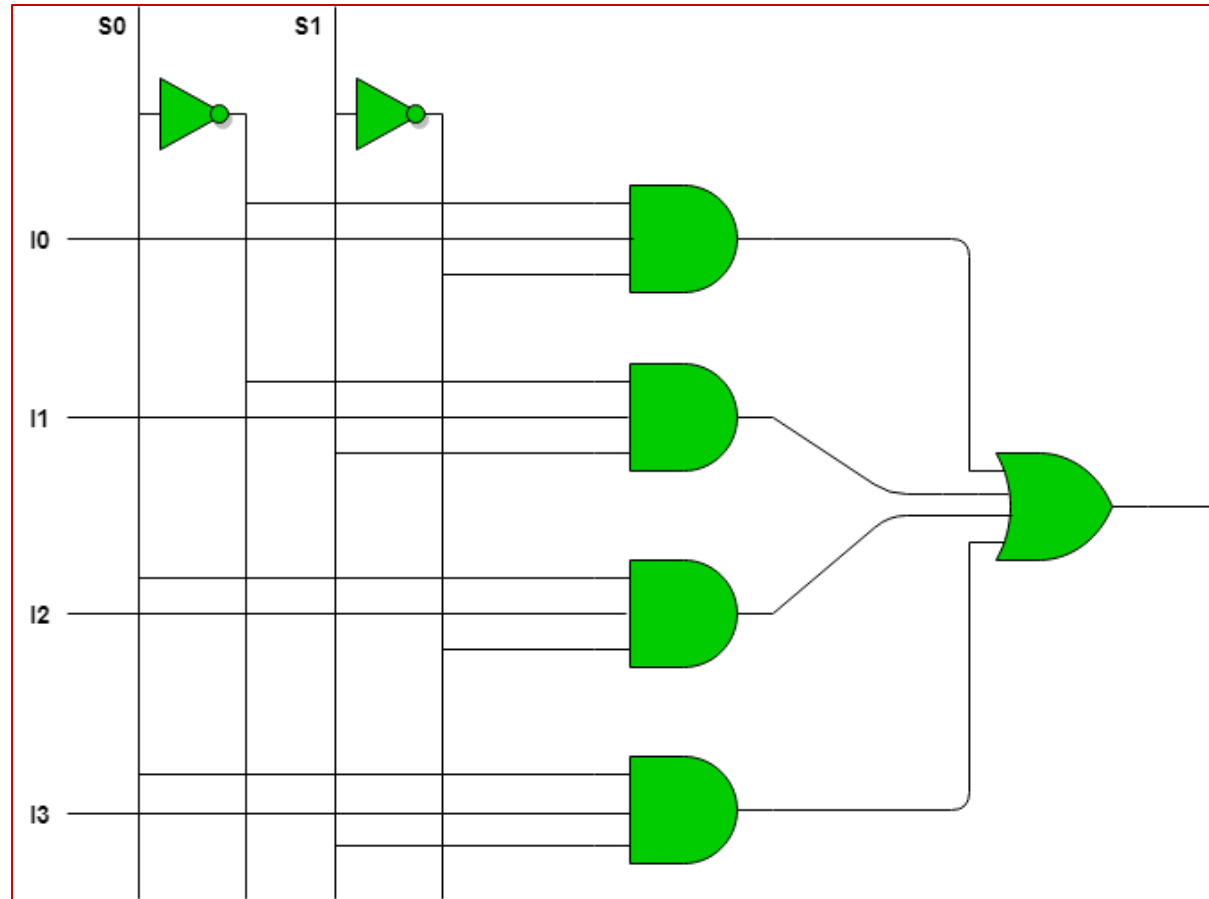
Multiplexers (MUX)

- ❑ A multiplexer is a combinational circuit with multiple data inputs and a single output, determined by control or select lines.
- ❑ Often referred to as MUX, it requires $\log_2(N)$ selection lines for N input lines, or equivalently, n selection lines for 2^n input lines.
- ❑ Below is the Block Diagram of the Multiplexer, It will have 2^n Input lines and will select output based on the Select line.

Multiplexers (MUX)



Multiplexers (MUX)



Multiplexers (MUX)

Selection Lines		Output
S_1	S_0	Y
0	0	I_0
0	1	I_1
1	0	I_2
1	1	I_3

Advantages of Multiplexers (MUX)

- ❑ **Reduces the number of data lines:** MUX allows multiple signals to share a single communication line, saving space and resources.
- ❑ **Simpler Design:** It simplifies the system by reducing the number of data channels needed.
- ❑ **Efficient Use of Resources:** MUX optimizes the use of communication channels or buses.
- ❑ **Flexible Design:** MUX can be used for different types of data and communication formats.
- ❑ **Compact Systems:** It helps make systems smaller and simpler by reducing the number of connections.
- ❑ **Reduces Errors:** Fewer data lines mean less chance of interference or noise.

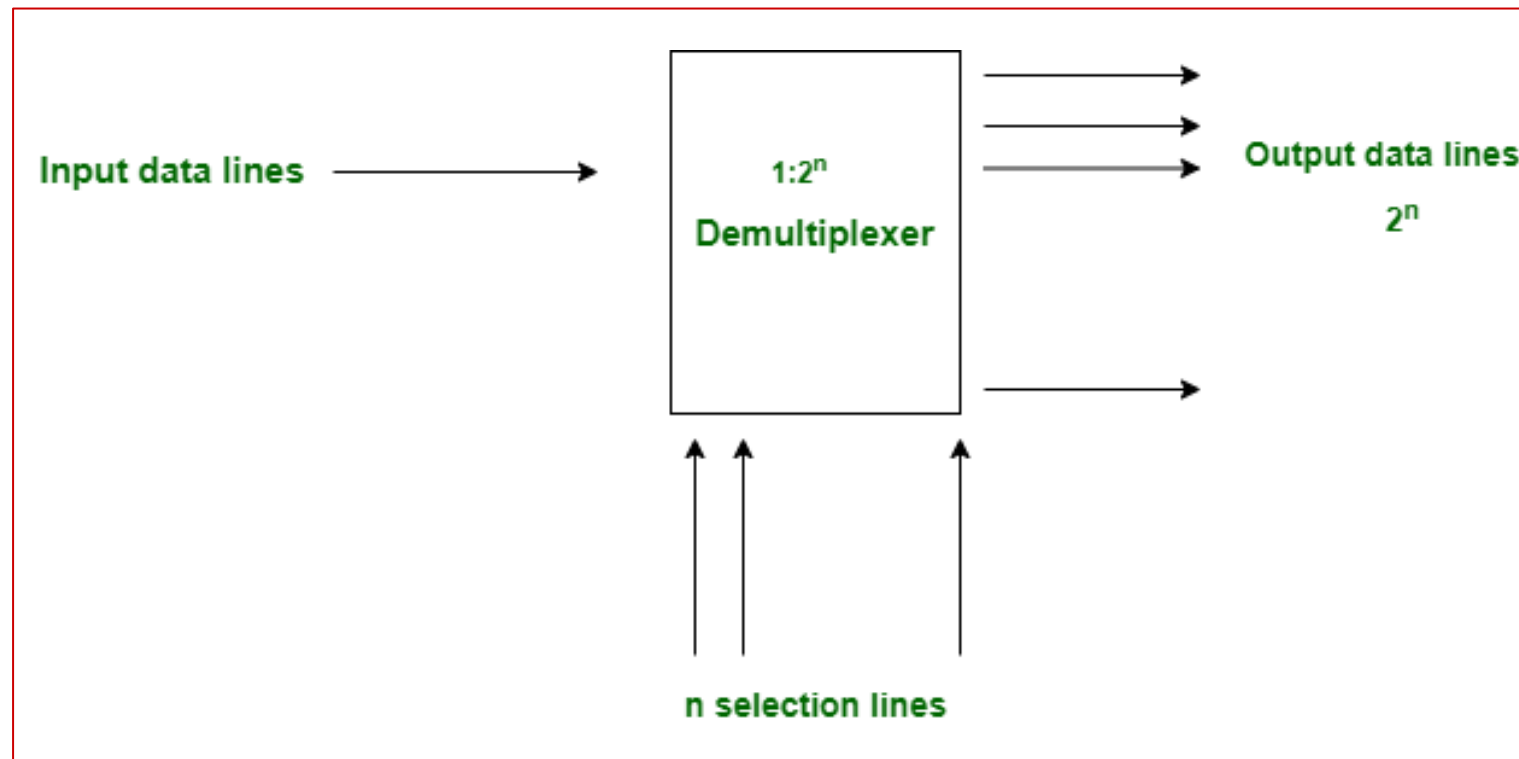
Disadvantages of Multiplexers (MUX)

- ❑ **Complex Control:** It needs extra circuits to select which input is sent, making the design more complicated.
- ❑ **Limited Inputs:** The number of inputs depends on the number of control lines (e.g., 2 control lines allow only 4 inputs).
- ❑ **Propagation Delay:** Switching between inputs can introduce delays in high-speed systems.
- ❑ **Higher Power Use:** Larger multiplexers with more inputs can consume more power.
- ❑ **Signal Loss:** Some signal degradation may occur when combining multiple signals.

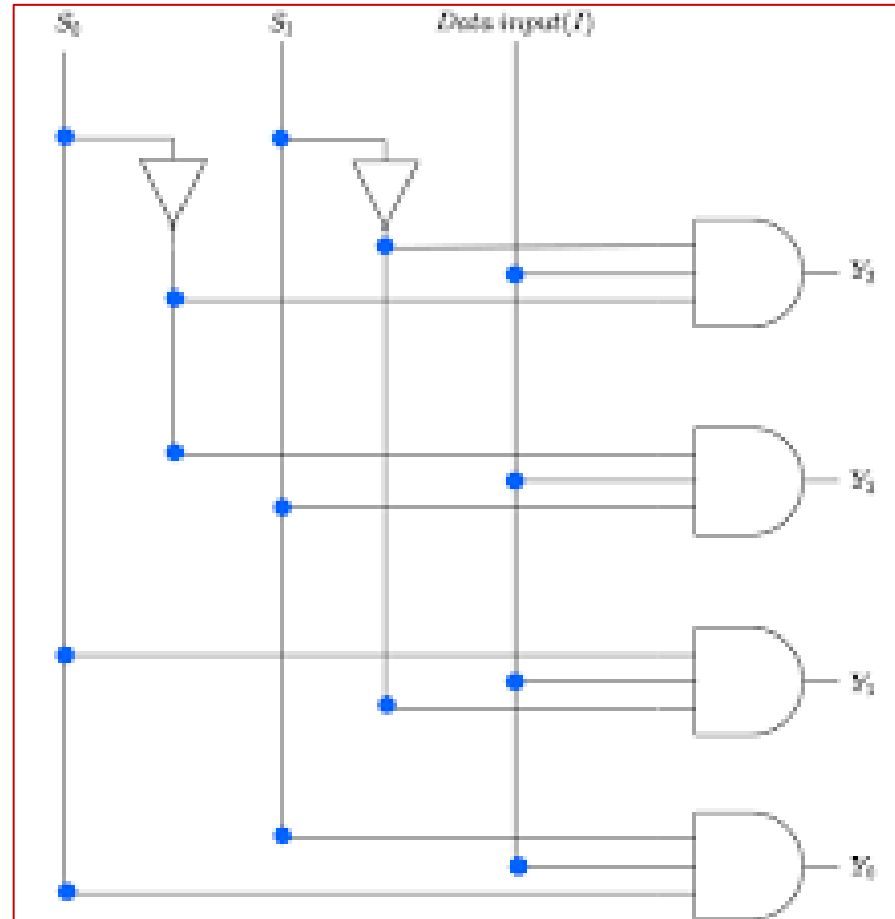
DE Multiplexers (DEMUX)

- ❑ Demultiplexer is the opposite of multiplexer. It is also termed as DEMUX.
- ❑ It takes input from one source and also converts the data to transmit towards various sources.
- ❑ The demultiplexer has one data input line and has several control lines (also known as select lines).
- ❑ These lines determine to which output the input data should be sent. The number of control lines determines the number of output lines.
- ❑ Given below is the block diagram of the Demultiplexer, It will have one Input line and will give 2^n output lines.

DE Multiplexers (DEMUX)



DE Multiplexers (DEMUX)



DE Multiplexers (DEMUX)

Select Line		Outputs			
S_1	S_0	Y_3	Y_2	Y_1	Y_0
0	0	0	0	0	I
0	1	0	0	I	0
1	0	0	I	0	0
1	1	I	0	0	0

Advantages of DE Multiplexers (DEMUX)

- ❑ **Efficient Data Distribution:** Routes one input signal to multiple outputs effectively.
- ❑ **Reduced Transmission Complexity:** Simplifies transmitter design by minimizing input lines.
- ❑ **High-Speed Data Splitting:** Ideal for applications requiring rapid data distribution.
- ❑ **Scalability:** Can handle more outputs by increasing control lines.
- ❑ **Versatility:** Widely used in TV broadcasting, communication networks, and more.

Disadvantages of DE Multiplexers (DEMUX)

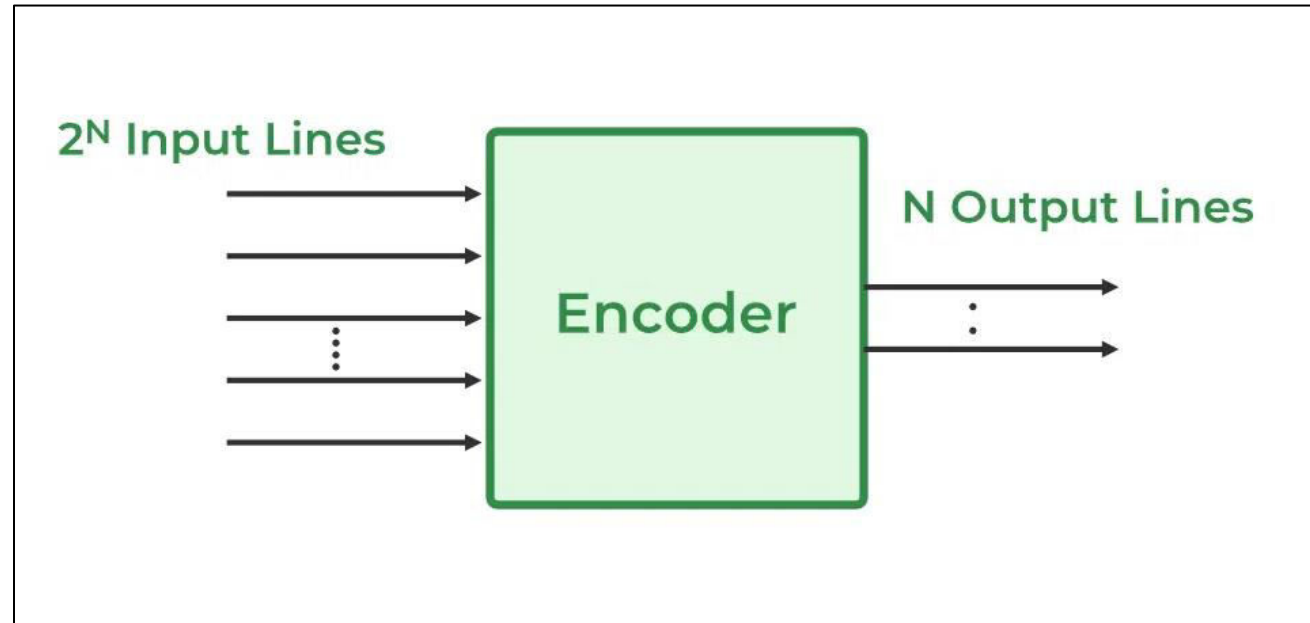
- ❑ **Control Complexity:** Additional circuits are needed for output selection.
- ❑ **Limited Outputs:** The number of outputs depends on available control lines.
- ❑ **Propagation Delay:** Routing input to outputs may introduce delays in larger systems.
- ❑ **Signal Degradation:** Signal strength may reduce when split into multiple outputs.
- ❑ **Higher Power Consumption:** Power usage increases with additional outputs.
- ❑ **Noise Susceptibility:** Splitting signals can lead to interference.

Encoders and Decoders

- ❑ Encoders and decoders are **combinational logic circuits** used in digital electronics to convert data between different formats.
- ❑ They are widely used in **communication systems, data compression, and computer memory**.
- ❑ **Encoders** convert 2^N lines of input into a code of N bits and **Decoders** decode the N bits into 2^N lines.

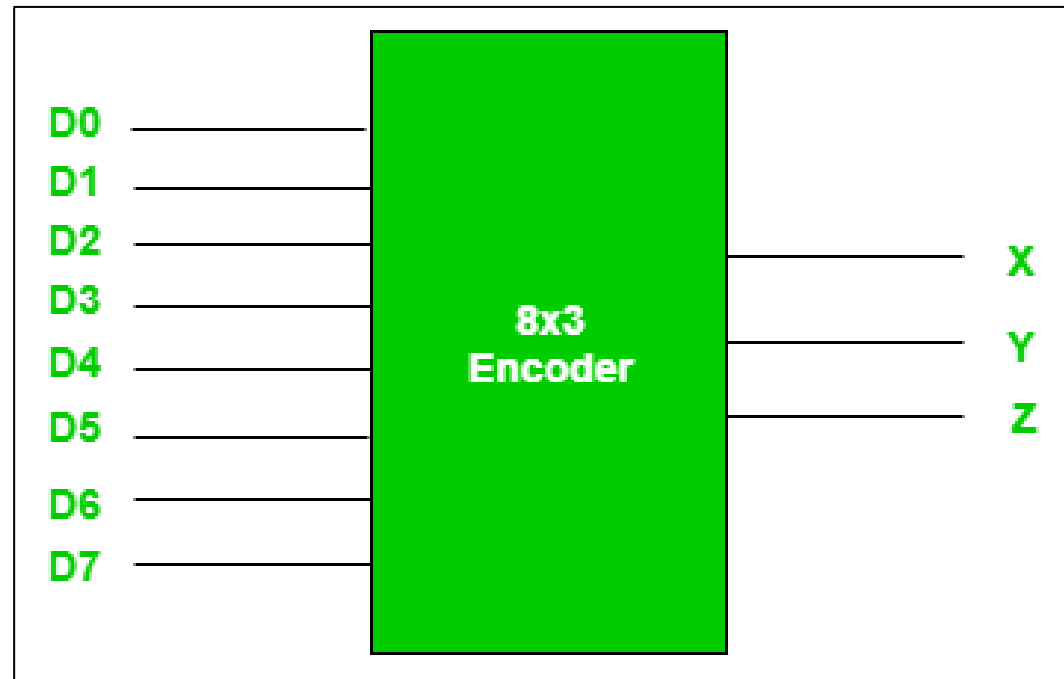
Encoders

- ❑ An encoder is a combinational circuit that converts binary information in the form of a 2^N input lines into N output lines, which represent N bit code for the input.
- ❑ For simple encoders, it is assumed that only one input line is active at a time.



Encoders

- ❑ As an example, let's consider **Octal to Binary** encoder.
- ❑ As shown in the following figure, an octal-to-binary encoder takes 8 input lines and generates 3 output lines.



Encoders

Truth Table (8x3 Encoder)

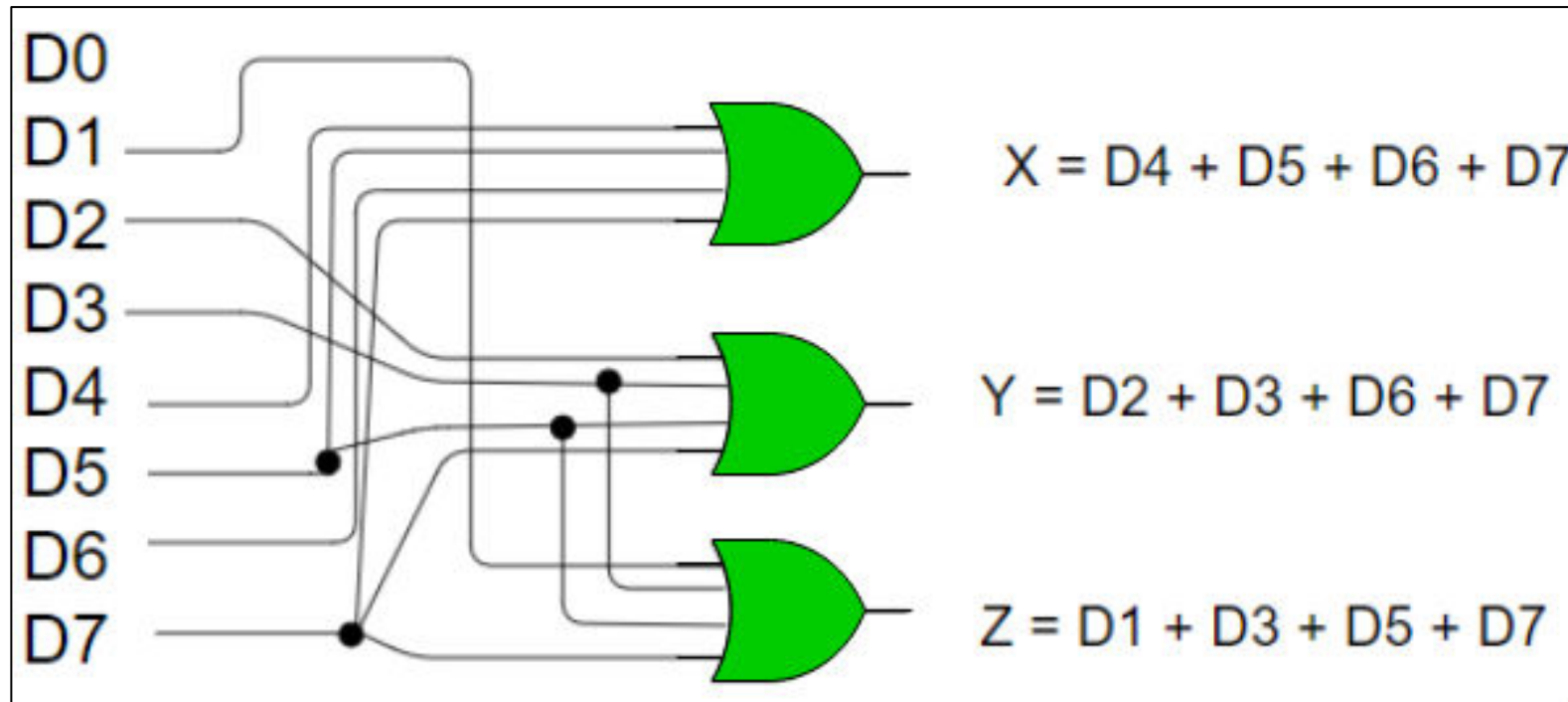
D7	D6	D5	D4	D3	D2	D1	D0	X	Y	Z
0	0	0	0	0	0	0	1	0	0	0
0	0	0	0	0	0	1	0	0	0	1
0	0	0	0	0	1	0	0	0	1	0
0	0	0	0	1	0	0	0	0	1	1
0	0	0	1	0	0	0	0	1	0	0
0	0	1	0	0	0	0	0	1	0	1
0	1	0	0	0	0	0	0	1	1	0
1	0	0	0	0	0	0	0	1	1	1

Encoders

- ❑ From the truth table, the output line Z is active when the input octal digit is 1, 3, 5 or 7.
- ❑ Similarly, Y is 1 when input octal digit is 2, 3, 6 or 7 and X is 1 for input octal digits 4, 5, 6 or 7.
- ❑ Hence, the Boolean functions would be:
 - ❑ $X = D4 + D5 + D6 + D7$
 - ❑ $Y = D2 + D3 + D6 + D7$
 - ❑ $Z = D1 + D3 + D5 + D7$

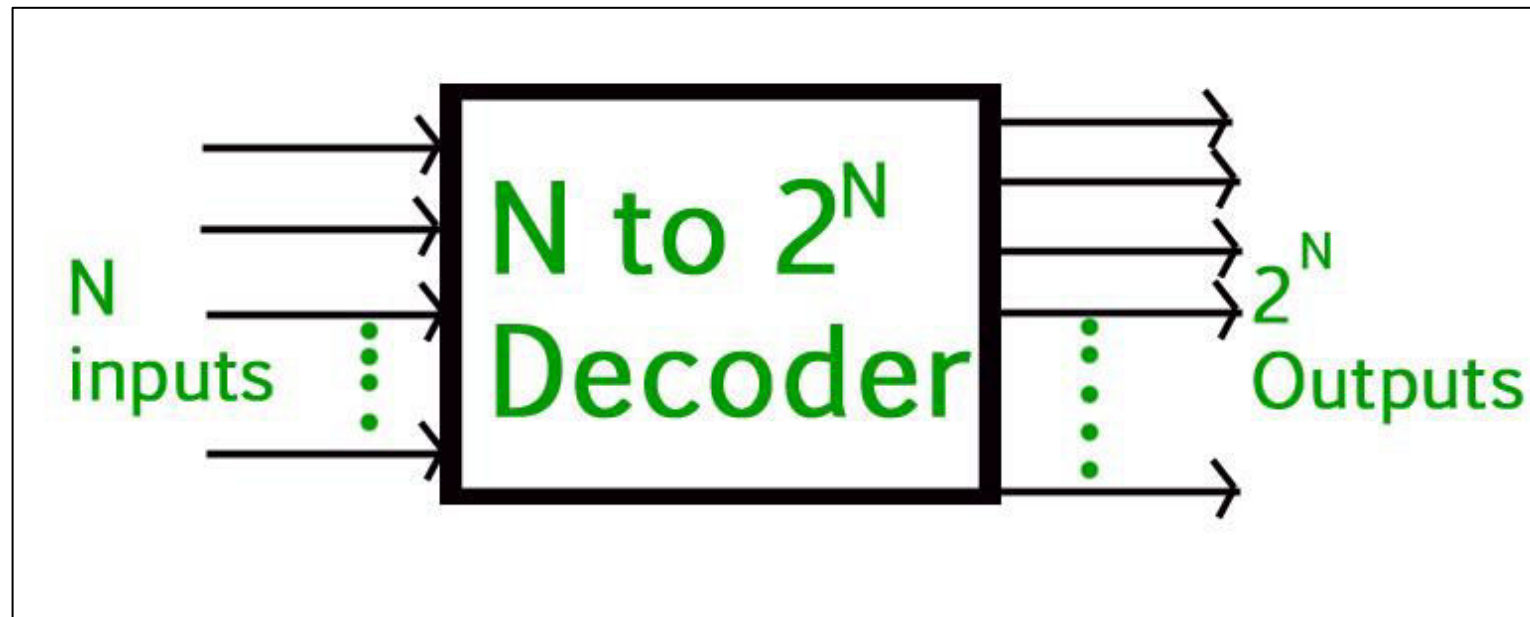
Encoders

- Hence, the encoder can be realized with OR gates as follows:



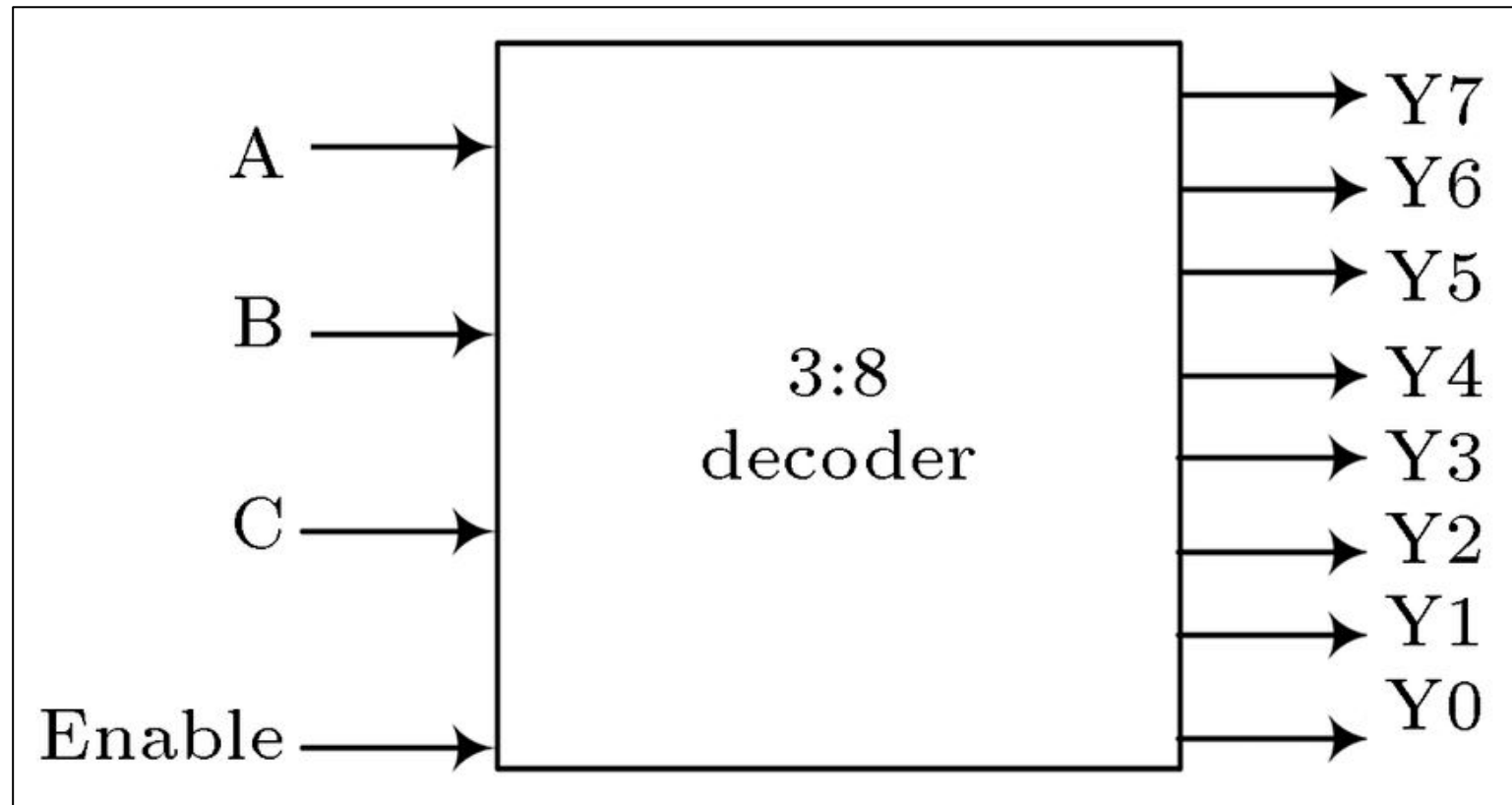
Decoders

- ❑ A decoder does the opposite job of an encoder.
- ❑ It is a combinational circuit that converts n lines of input into 2^n lines of output.



Decoders

- Let's take an example of 3-to-8 line decoder.



Decoders

Truth Table (3x8 Decoder)

X	Y	Z	D0	D1	D2	D3	D4	D5	D6	D7
0	0	0	1	0	0	0	0	0	0	0
0	0	1	0	1	0	0	0	0	0	0
0	1	0	0	0	1	0	0	0	0	0
0	1	1	0	0	0	1	0	0	0	0
1	0	0	0	0	0	0	1	0	0	0
1	0	1	0	0	0	0	0	1	0	0
1	1	0	0	0	0	0	0	0	1	0
1	1	1	0	0	0	0	0	0	0	1

Decoders

□ D0 is high, when $X = 0$, $Y = 0$ and $Z = 0$. Hence:

□ $D0 = X' Y' Z'$

□ Similarly,

□ $D1 = X' Y' Z$

□ $D2 = X' Y Z'$

□ $D3 = X' Y Z$

□ $D4 = X Y' Z'$

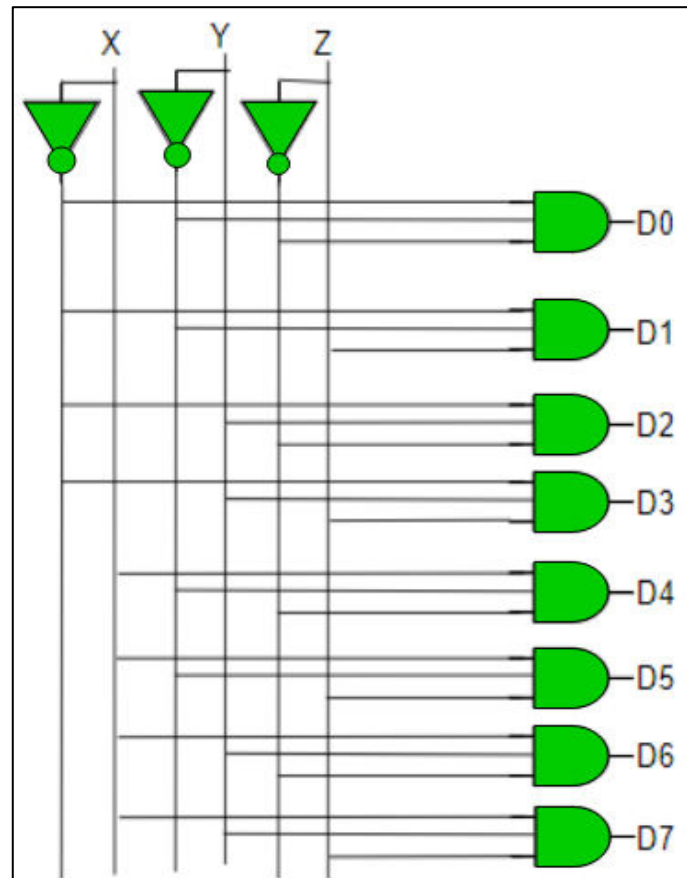
□ $D5 = X Y' Z$

□ $D6 = X Y Z'$

□ $D7 = X Y Z$

Decoders

□ Hence,



Any Question?

Thank you

Digital Logic and Design

Lecture : 10

Department of Computer Science
GPGC SWABI (Khyber Pakhtunkhwa)

Spring 2025 Semester
BS Course: DLD

Course Instructor :
Mr. Ihtisham Ul Haq
Lecturer (CS)
GPGC Swabi

Contents

- ☐ Parity Checkers
 - ☐ Working
 - ☐ Types
- ☐ Parity Generators
 - ☐ Working
 - ☐ Types
- ☐ Code Converters
 - ☐ BCD to 7-Segment
 - ☐ Binary to Gray Code

Parity Generators and Checkers

- ❑ Parity generators and checkers are essential components in digital systems used for **error detection** in data transmission.
- ❑ They help identify whether an error has occurred by adding or verifying a parity bit.

- ❑ **Parity Bit Concept?**
- ❑ A **parity bit** is an extra bit added to a group of data bits to make the number of **1s** either **even** (even parity) or **odd** (odd parity).
 - ❑ **Even Parity:** The total number of 1s (including the parity bit) is even.
 - ❑ **Odd Parity:** The total number of 1s (including the parity bit) is odd.

Parity Generators and Checkers

- ❑ The parity bit is one of the simplest forms of error-detection technique used in digital electronics.
- ❑ Example: 4-bit Data with Parity Bit

Data Bits	Even Parity Bit	Odd Parity Bit
1010	0	1
1101	1	0
1111	0	1

Parity Generators

- ❑ **Parity Generator**
- ❑ A **parity generator** takes input data and generates the required parity bit (even or odd).
- ❑ It is typically implemented using **XOR gates**.
- ❑ **Types**
 1. **Even Parity Generator**
 2. **Odd Parity Generator**
- ❑ The parity generator is used at the transmitter end and generate and add a parity bit the original code before transmission.

Parity Generators

- ❑ **Circuit for 3-bit Even Parity Generator**
- ❑ For a 3-bit input **A, B, C**, the even parity bit **P** is:
 - ❑ $P = A \oplus B \oplus C$
- ❑ Where $\oplus$ represents the XOR operation.

Parity Generators

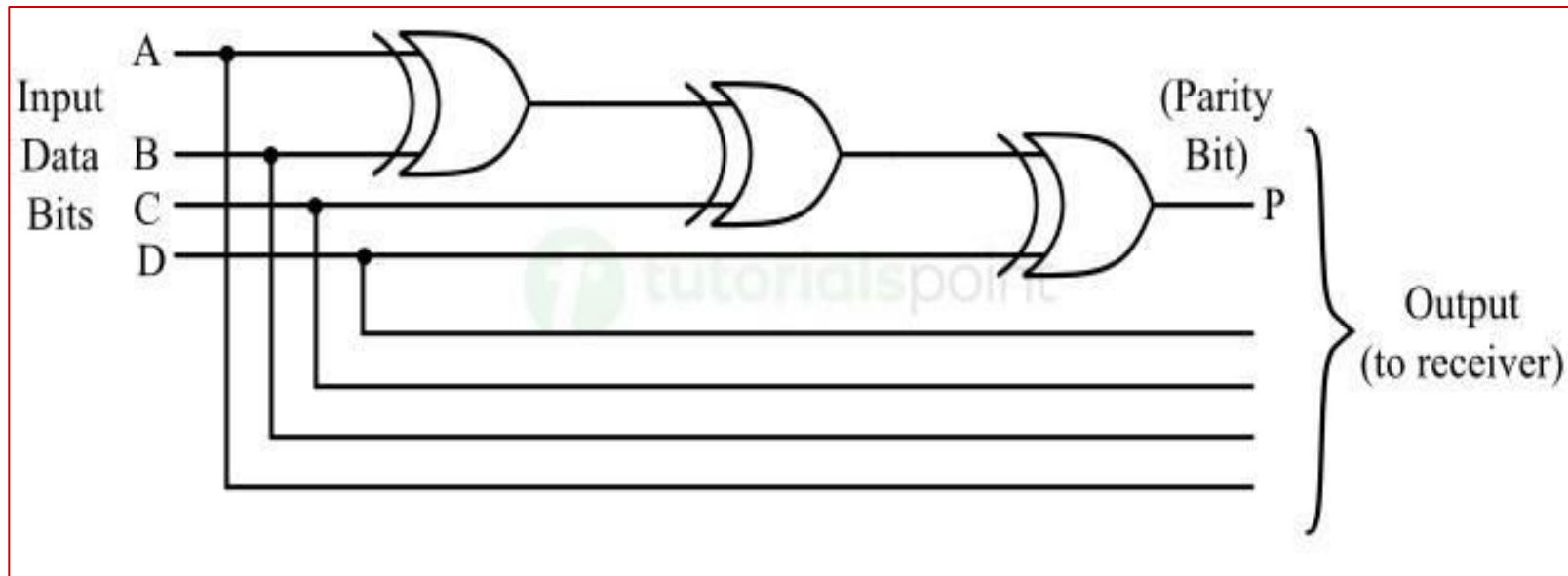
Truth Table

A	B	C	Even Parity Bit (P)
0	0	0	0
0	0	1	1
0	1	0	1
0	1	1	0
1	0	0	1
1	0	1	0
1	1	0	0
1	1	1	1

Parity Generators

- ❑ **Logic Circuit**
- ❑ A simple **3-bit even parity generator** can be implemented using:
- ❑ $P = A \oplus B \oplus C$

Parity Generators



Parity Checkers

- ❑ **Parity Checker**
- ❑ A **parity checker** verifies whether the received data has correct parity.
- ❑ It checks if the total number of 1s (including the parity bit) matches the expected parity (even or odd).
- ❑ Uses XOR gates to check parity.
- ❑ **Types**
 1. **Even Parity Checker**
 2. **Odd Parity Checker**

Parity Checkers

- ❑ **Circuit for 4-bit Even Parity Checker**
- ❑ For a **4-bit received data (A, B, C, P)**, the parity check output **X** is:
 - ❑ $X = A \oplus B \oplus C \oplus P$
 - ❑ If **X = 0**, the parity is correct.
 - ❑ If **X = 1**, an error has occurred.

Parity Checkers

Truth Table

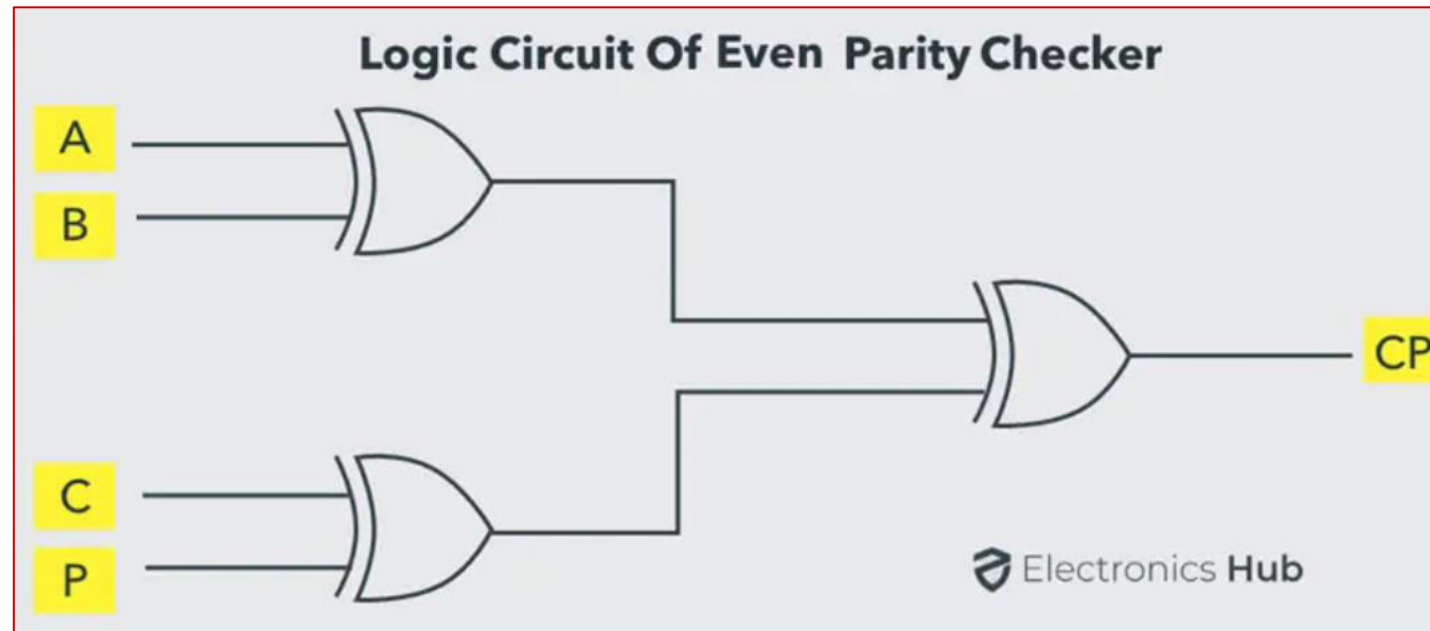
A	B	C	P	X (Error Check)
0	0	0	0	0
0	0	1	1	0
0	1	0	1	0
0	1	1	0	0
1	0	0	1	0
1	0	1	0	0
1	1	0	0	0
1	1	1	1	0
1	1	0	1	1

❑ If **X = 1**, an error is detected.

Parity Checkers

- ❑ **Logic Circuit**
- ❑ **A 4-bit even parity generator** can be implemented using:
- ❑ $P = A \oplus B \oplus C \oplus D$

Parity Checkers



Parity Check

❑ Applications of Parity Check

1. Data Transmission (Wired & Wireless Communication)
2. Error Detection in Memory (RAM, Hard Drives, SSDs)
3. Network Data Transmission (Ethernet, Wi-Fi, Serial Communication)
4. Barcode Scanning and Digital Systems

Code Converters

- ❑ Code Converters are the digital circuits or algorithms that are designed to translate data representation from one format to the other format.
- ❑ Two common types of code converters are:
 1. BCD to 7-Segment
 2. Binary to Gray Code

BCD to 7-Segment

- ❑ **Binary Coded Decimal (BCD)**

- ❑ BCD is the encoding scheme where each of the decimal numbers(0-9) is represented by its equivalent binary pattern (which is generally of 4-bits).

- ❑ **7- Segment Display**

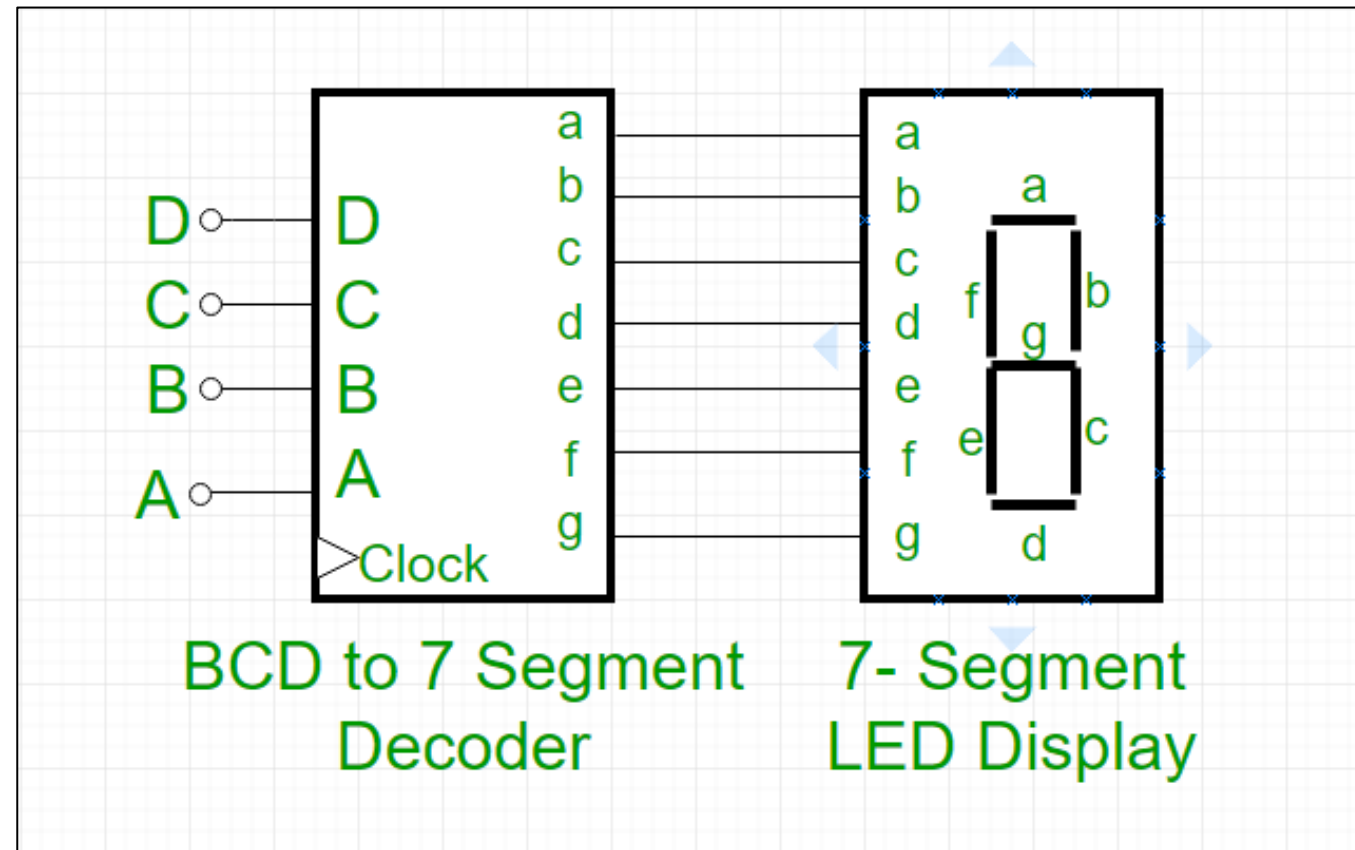
- ❑ Seven Segment display is an electronic device which consists of seven Light Emitting Diodes (LEDs) arranged in a some definite pattern (common cathode or common anode type), which is used to display decimal numbers.

BCD to 7-Segment

- ❑ First, our decimal number is changed to its BCD equivalent signal then BCD to seven segment decoder converts that signals to the form which is fed to seven segment display.
- ❑ This BCD to seven segment decoder has four input lines (A, B, C and D) and 7 output lines (a, b, c, d, e, f and g), this output is given to seven segment LED display which displays the decimal number depending upon inputs.

BCD to 7-Segment

Block Diagram



BCD to 7-Segment

Truth Table

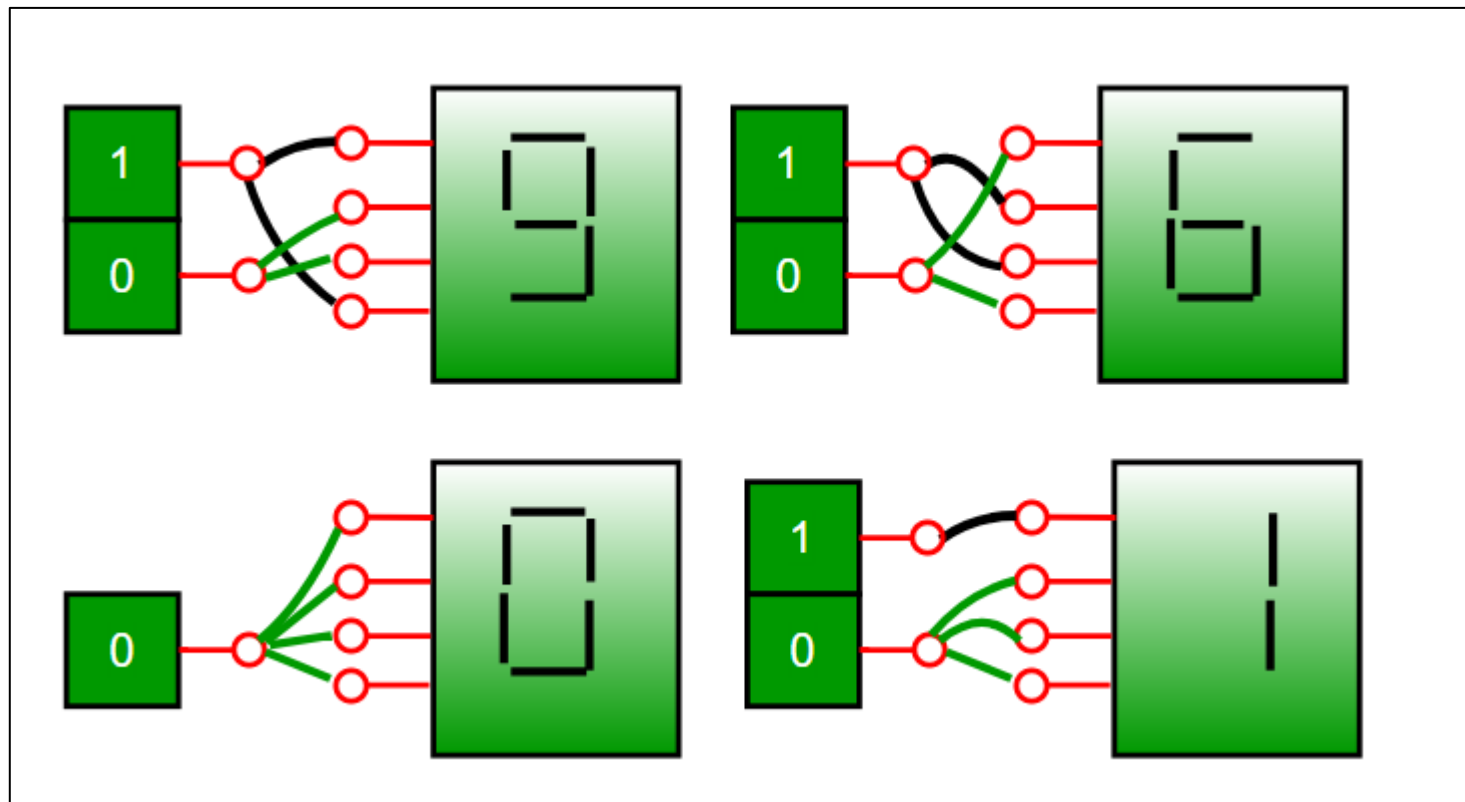
A	B	C	D	a	b	c	d	e	f	g
0	0	0	0	1	1	1	1	1	1	0
0	0	0	1	0	1	1	0	0	0	0
0	0	1	0	1	1	0	1	1	0	1
0	0	1	1	1	1	1	1	0	0	1
0	1	0	0	0	1	1	0	0	1	1
0	1	0	1	1	0	1	1	0	1	1
0	1	1	0	1	0	1	1	1	1	1
0	1	1	1	1	1	1	0	0	0	0
1	0	0	0	1	1	1	1	1	1	1
1	0	0	1	1	1	1	1	0	1	1

BCD to 7-Segment

- ❑ Output for first combination of inputs (A, B, C and D) in Truth Table corresponds to '0' and last combination corresponds to '9'.
- ❑ Similarly rest corresponds from 2 to 8 from top to bottom.

BCD to 7-Segment

□ Examples:



BCD to 7-Segment

❑ Applications of BCD to 7-Segment Converter:

1. Digital Clocks
2. Calculators
3. Electronic Meters
4. Scoreboards
5. Digital Timers
6. Fuel Price Displays

Binary to Gray Code

- ❑ **Binary Code?**
- ❑ The Binary code is the numerical system used in digital electronics.
- ❑ It only consists of two Symbols which are 0 and 1.
- ❑ In binary code each digit is represented by power of 2 with the starting bit (right most bit) as 2^0 and bit as 2^1 and so on.

Binary to Gray Code

- ❑ **Gray Code**
- ❑ Gray code is a binary numeral system in which two successive values differ by only one bit. This is also called a reflected binary code.
- ❑ **Why Use Gray Code?**
 - ❑ **Minimizes Errors:** Useful in mechanical encoders and digital systems where errors from multiple-bit changes must be avoided.
 - ❑ **Efficient in Digital Logic:** Used in analog-to-digital converters (ADCs) and communication systems.
 - ❑ **Prevents Glitches:** Reduces errors in rotary encoders and sensors.

Binary to Gray Code

- ❑ A Binary to Gray Code Converter is a digital circuit that converts a binary number into its equivalent Gray code.
- ❑ **Logic for Binary to Gray Code Conversion**
- ❑ Each **Gray code bit** is derived from the binary number using the formula:
 - ❑ $G_n = B_n \oplus B_{n-1}$
- ❑ Where:
 - ❑ G_n = Gray code bit
 - ❑ B_n = Binary bit
 - ❑ $\oplus$ = XOR operation

Binary to Gray Code

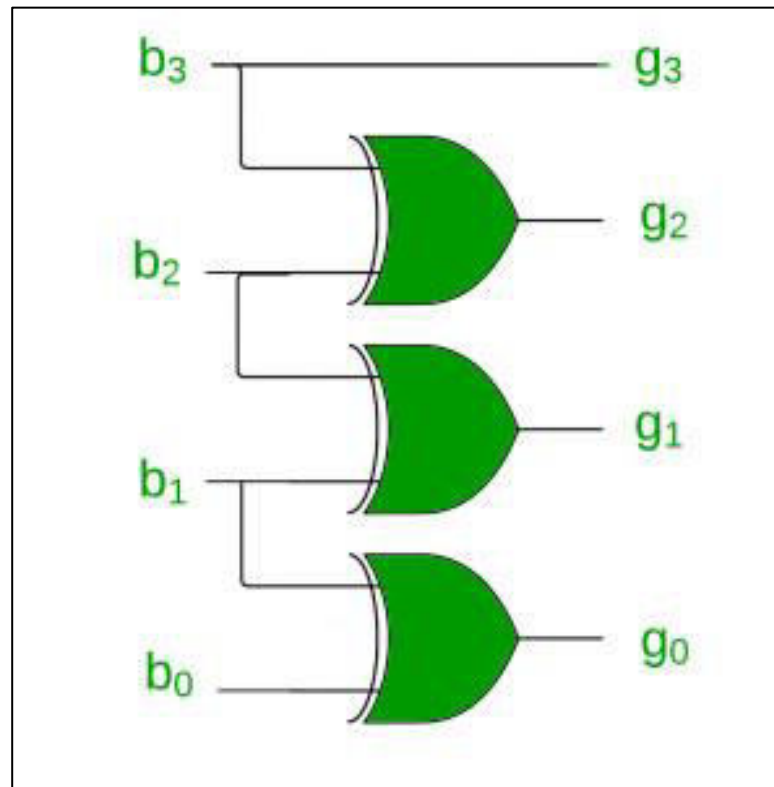
- ❑ For 4-bit binary input ($B_3B_2B_1B_0$), the corresponding Gray code output ($G_3G_2G_1G_0$) is:
 - ❑ $G_3=B_3$
 - ❑ $G_2=B_3\oplus B_2$
 - ❑ $G_1=B_2\oplus B_1$
 - ❑ $G_0=B_1\oplus B_0$

Binary to Gray Code

Binary (B ₃ B ₂ B ₁ B ₀)	Gray Code (G ₃ G ₂ G ₁ G ₀)
0000	0000
0001	0001
0010	0011
0011	0010
0100	0110
0101	0111
0110	0101
0111	0100
1000	1100
1001	1101
1010	1111
1011	1110
1100	1010
1101	1011
1110	1001
1111	1000

Binary to Gray Code

- ❑ A combinational logic circuit can be implemented using **XOR** gates for binary to gray code converter.



Gray to Binary Code

- ❑ A **Gray to Binary Code Converter** is a digital circuit that converts a **Gray code number** back to its **equivalent binary number**.
- ❑ **Logic for Gray to Binary Conversion**
- ❑ Each **binary bit** is derived from the **Gray code bit** using the following formula:
 - ❑ $B_n = G_n$
 - ❑ $B_{n-1} = B_n \oplus G_{n-1}$
 - ❑ $B_{n-2} = B_{n-1} \oplus G_{n-2}$
 - ❑ $B_0 = B_1 \oplus G_0$
- ❑ Where:
 - ❑ B_n = Binary bit
 - ❑ G_n = Gray code bit
 - ❑ $\oplus$ = XOR operation

Gray to Binary Code

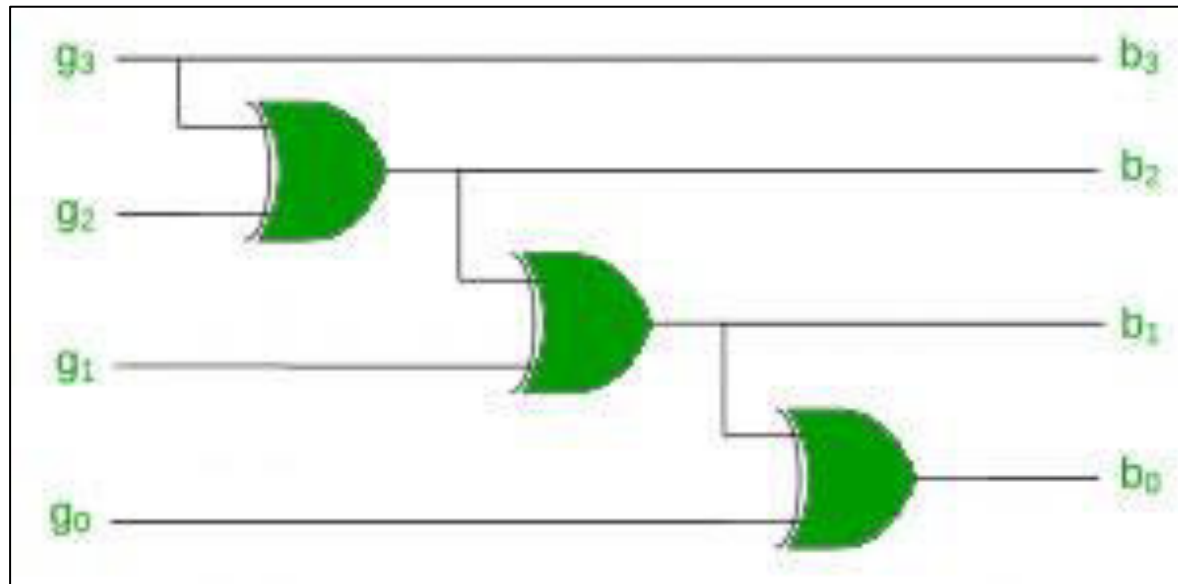
- ❑ For 4-bit Gray code input ($G_3G_2G_1G_0$), the corresponding Binary output ($B_3B_2B_1B_0$) is:
 - ❑ $B_3 = G_3$
 - ❑ $B_2 = B_3 \oplus G_2$
 - ❑ $B_1 = B_2 \oplus G_1$
 - ❑ $B_0 = B_1 \oplus G_0$

Gray to Binary Code

Gray Code (G ₃ G ₂ G ₁ G ₀)	Binary (B ₃ B ₂ B ₁ B ₀)
0000	0000
0001	0001
0011	0010
0010	0011
0110	0100
0111	0101
0101	0110
0100	0111
1100	1000
1101	1001
1111	1010
1110	1011
1010	1100
1011	1101
1001	1110
1000	1111

Gray to Binary Code

- ❑ A combinational logic circuit can be implemented using **XOR** gates for gray to binary code converter.



Gray to Binary Code

❑ Applications of Gray to Binary/Binary to Gray Code Converter

1. Communication Systems (Error Detection & Correction)
2. Digital Circuits (Minimizing Power Consumption)
3. Rotary Encoders (Used in Motion Sensing)
4. Analog-to-Digital Converters (ADC Processing)

Any Question?

Thank you

Digital Logic and Design

Lecture : 11

Department of Computer Science
GPGC SWABI (Khyber Pakhtunkhwa)

Spring 2025 Semester
BS Course: DLD

Course Instructor :
Mr. Ihtisham Ul Haq
Lecturer (CS)
GPGC Swabi

Contents

- ☐ Sequential Logic Circuits
- ☐ Latches
 - ☐ SR Latch
 - ☐ Gated SR Latch
 - ☐ D Latch
 - ☐ Gated D Latch
 - ☐ JK Latch
 - ☐ T Latch
- ☐ Advantages of Latches
- ☐ Disadvantages of Latches

Sequential Logic Circuits

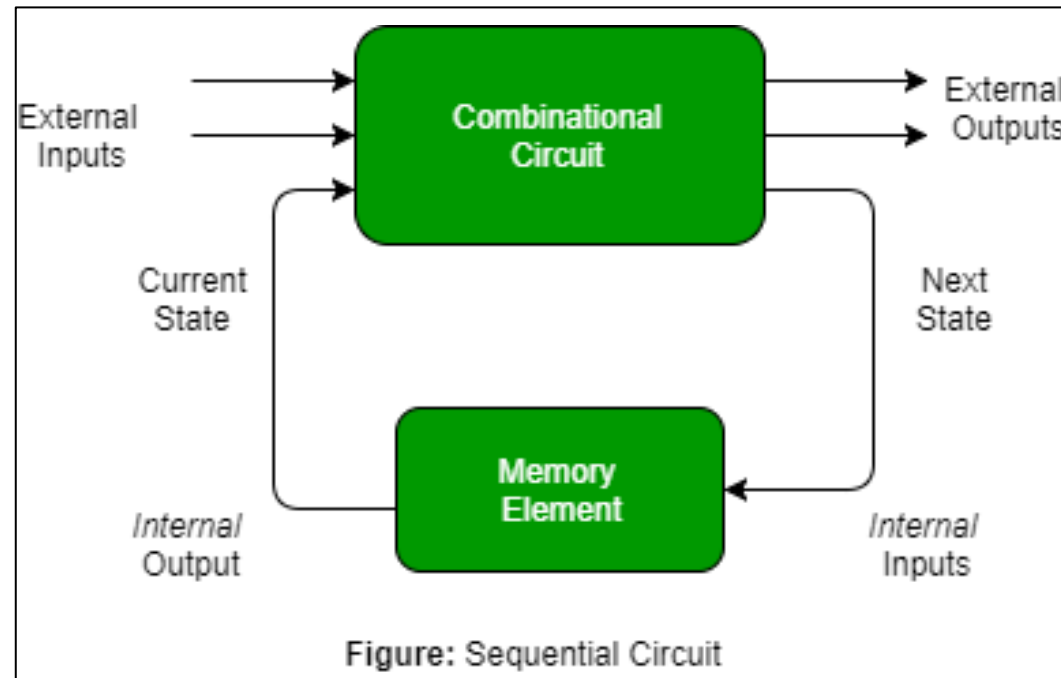
- ❑ Sequential logic circuits are digital circuits in which the output depends not only on the present inputs but also on past inputs or previous states.
- ❑ Unlike **combinational logic circuits**, which have outputs determined only by current inputs, sequential circuits use **memory elements** (flip-flops, latches) to store previous states.

Sequential Logic Circuits

- ❑ Sequential circuits are digital circuits that store and use previous state information to determine their next state.
- ❑ They are commonly used in digital systems to implement state machines, timers, counters, and memory elements and are essential components in digital systems design.
- ❑ The memory elements in sequential circuits can be implemented using flip-flops, which are circuits that store binary values and maintain their state even when the inputs change.

Sequential Logic Circuits

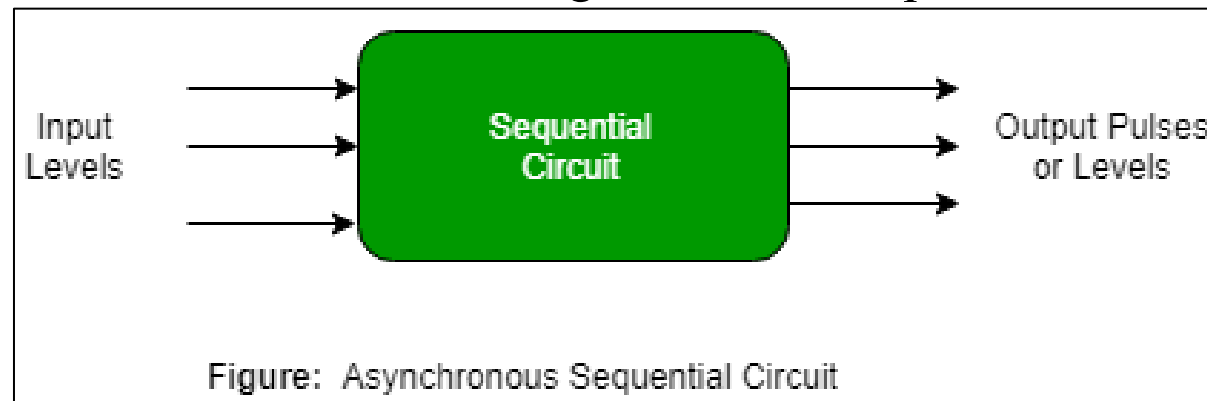
- ❑ As shown in the figure, there are two types of input to the combinational logic :
 - ❑ External inputs which are not controlled by the circuit.
 - ❑ Internal inputs, which are a function of a previous output state.



Types of Sequential Circuits

1) Asynchronous Sequential Circuit

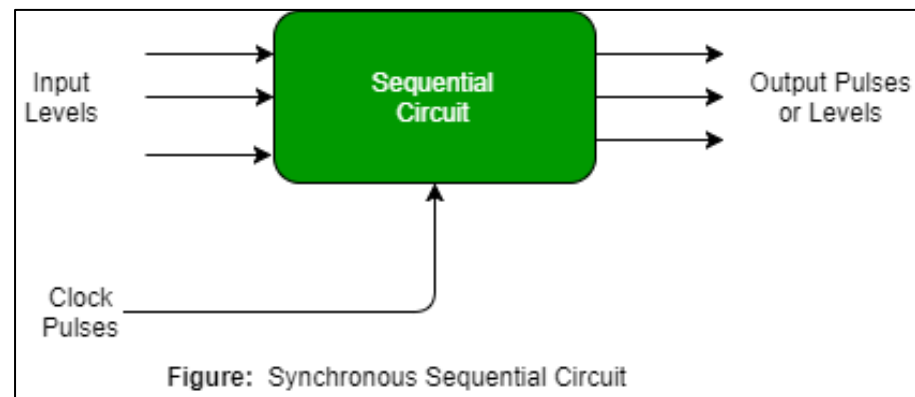
- ❑ These circuits do not use a clock signal but uses the pulses of the inputs.
- ❑ These circuits are faster than synchronous sequential circuits because there is no clock pulse and change their state immediately when there is a change in the input signal.
- ❑ We use asynchronous sequential circuits when speed of operation is important and independent of internal clock pulse.
- ❑ But these circuits are more difficult to design and their output is uncertain.



Types of Sequential Circuits

2) Synchronous Sequential Circuit

- ❑ These circuits use clock signal and level inputs (or pulsed).
- ❑ The output pulse is the same duration as the clock pulse for the clocked sequential circuits.
- ❑ Since they wait for the next clock pulse to arrive to perform the next operation, so these circuits are bit slower compared to asynchronous.
- ❑ Level output changes state at the start of an input pulse and remains in that until the next input or clock pulse.



Types of Sequential Circuits

- ❑ We use synchronous sequential circuit in synchronous counters, flip flops, and in the design of MOORE-MEALY state management machines.
- ❑ We use sequential circuits to design Counters, Registers, RAM, MOORE/MEALY Machine and other state retaining machines.

Applications of Sequential Logic Circuits

- ❑ Sequential circuits find application in virtually almost every digital system today because of their capacity to handle state information.
- ❑ Some common applications include:
 1. **Counters:** Appearing in commonly in digital clocks, frequency counters, and event counters.
 2. **Registers:** Found in microprocessors and digital systems as a storage medium, a transfer medium and a medium for manipulating data.
 3. **Memory Elements:** Used in RAM and other storage devices to keep data in a temporary hold.

Applications of Sequential Logic Circuits

- 4. **State Machines:** Made use in control systems , communication processes, and different digital devices for state control.
- 5. **Timers:** It is applied in time measurement, delay production, and scheduling functions in digital circuits.

Difference Between Combinational and Sequential Circuits

Feature	Combinational Logic	Sequential Logic
Memory	No memory, outputs depend only on current inputs	Has memory, stores previous states
Clock Requirement	No clock required	Uses a clock signal to synchronize
Examples	Adders, Multiplexers, Encoders, Decoders	Flip-Flops, Counters, Registers
Feedback	No feedback loop	Feedback present

Latches

- ❑ A latch is a basic memory element used in digital circuits to store one bit of data.
- ❑ It is a bistable circuit, meaning it has two stable states: 0 (low) and 1 (high).
- ❑ Latches are level-triggered, meaning they change their output based on the input as long as the enable signal is active.
- ❑ Latches are widely used in sequential circuits, such as registers, memory units, and flip-flops.

Types of Latches

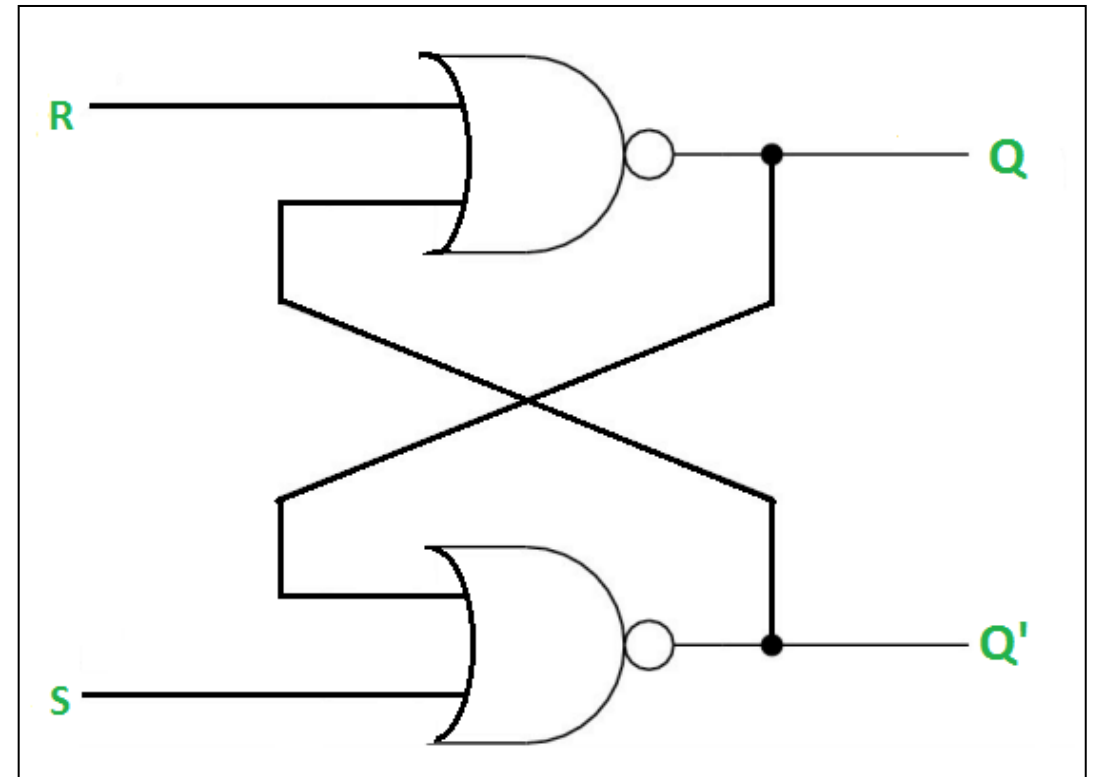
- ❑ In digital electronics different types of latches are:
 - ❑ SR Latches
 - ❑ Gated SR Latches
 - ❑ D Latches
 - ❑ Gated D Latches
 - ❑ JK Latches
 - ❑ T Latches

SR Latch

- ❑ S-R latches i.e., Set-Reset latches are the simplest form of latches and are implemented using two inputs: S (Set) and R (Reset).
- ❑ The S input sets the output to 1, while the R input resets the output to 0.
- ❑ When both S and R inputs are at 1, the latch is said to be in an “undefined” state.
- ❑ They are also known as preset and clear states.
- ❑ The SR latch forms the basic building blocks of all other types of flip-flops.

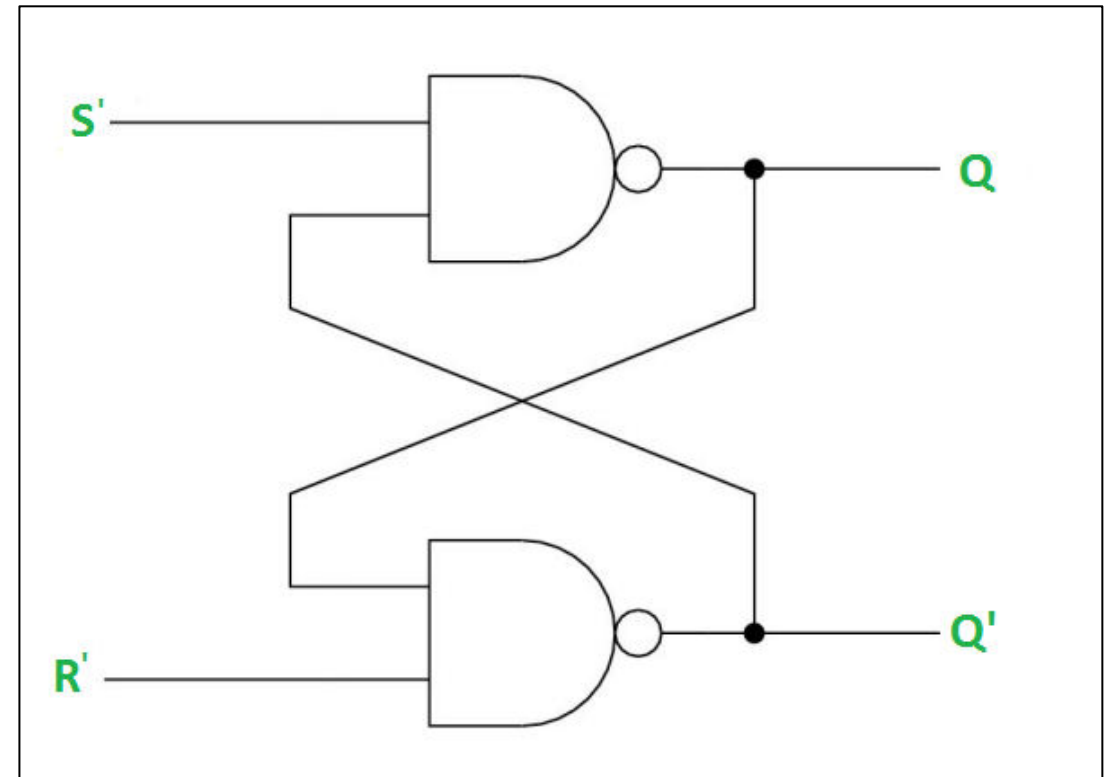
NOR Implementation

S	R	Q	Q'
0	0	No Change	No Change
0	1	0	1
1	0	1	0
1	1	X	X



NAND Implementation

S	R	S'	R'	Q	Q'
0	0	1	1	No Change	No Change
0	1	1	0	0	1
1	0	0	1	1	0
1	1	0	0	X	X



Different Cases of SR Latch

- ❑ The different cases of SR latch are discussed below.
- ❑ **Case 1: $S' = R' = 1$ ($S = R = 0$)**
- ❑ If $Q = 1$, Q and R' inputs for 2nd NAND gate are both 1.
- ❑ If $Q = 0$, Q and R' inputs for 2nd NAND gate are 0 and 1 respectively.

- ❑ **Case 2: $S' = 0, R' = 1$ ($S = 1, R = 0$)**
- ❑ As $S' = 0$, the output of 1st NAND gate, $Q = 1$ (SET state).
- ❑ In second NAND gate, as Q and R' inputs are 1, $Q' = 0$.

Different Cases of SR Latch

- ❑ **Case 3: $S' = 1, R' = 0$ ($S = 0, R = 1$)**
- ❑ As $R' = 0$, the output of 2nd NAND gate, $Q' = 1$.
- ❑ In first NAND gate, as Q and S' inputs are 1, $Q = 0$ (RESET state).

- ❑ **Case 4: $S' = R' = 0$ ($S = R = 1$)**
- ❑ When $S = R = 1$, both Q and Q' becomes 1 which is not allowed. So, the input condition is prohibited.

Gated SR Latch

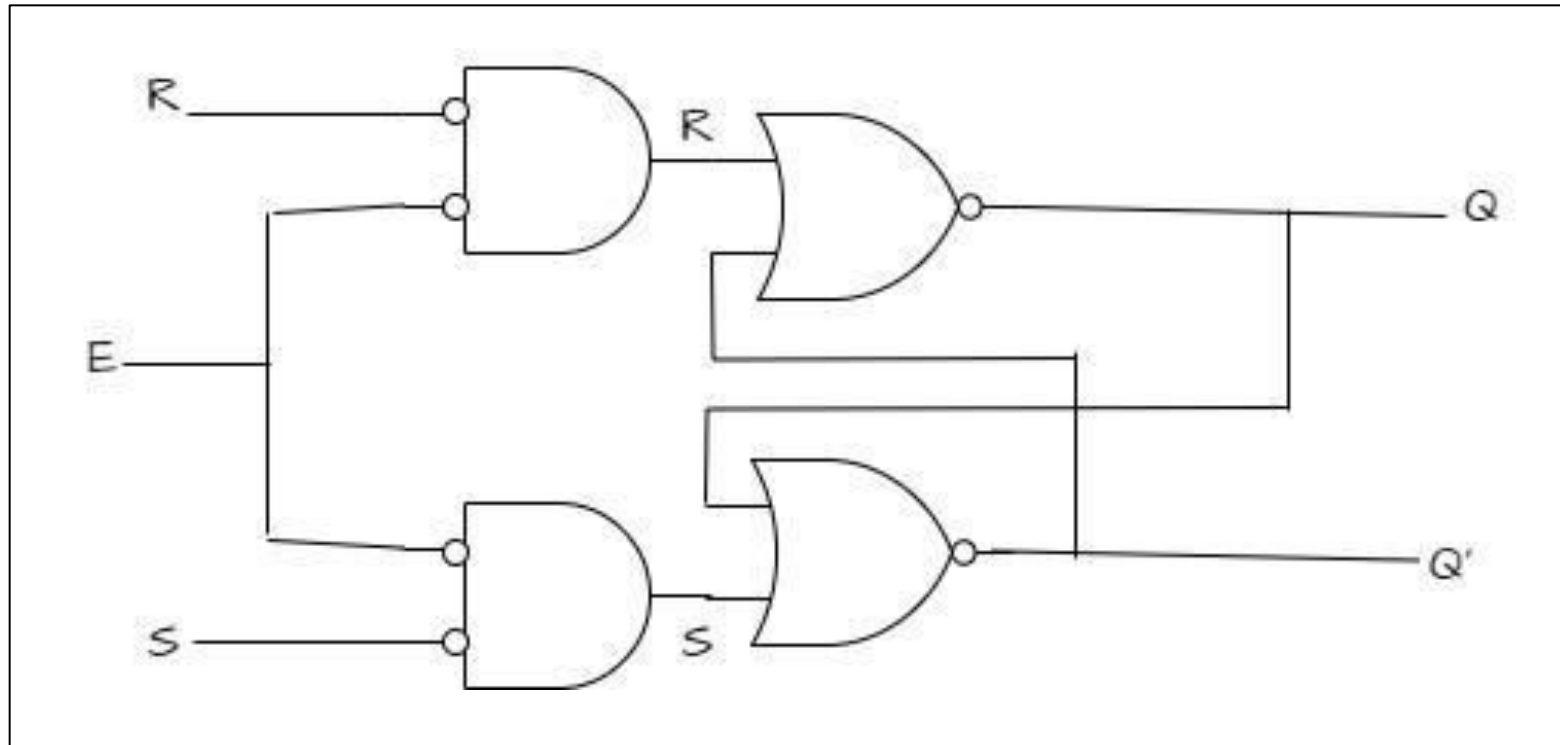
- ❑ A Gated SR latch is a SR latch with enable input which works when enable is 1 and retain the previous state when enable is 0.

Gated SR Latch Truth Table

Enable	S	R	Q_{n+1}
0	X	X	Q_n
1	0	0	Q_n
1	0	1	0
1	1	0	1
1	1	1	X

Gated SR Latch Logic Diagram

- ❑ The below logic diagram represents the gated SR latch.



D Latch

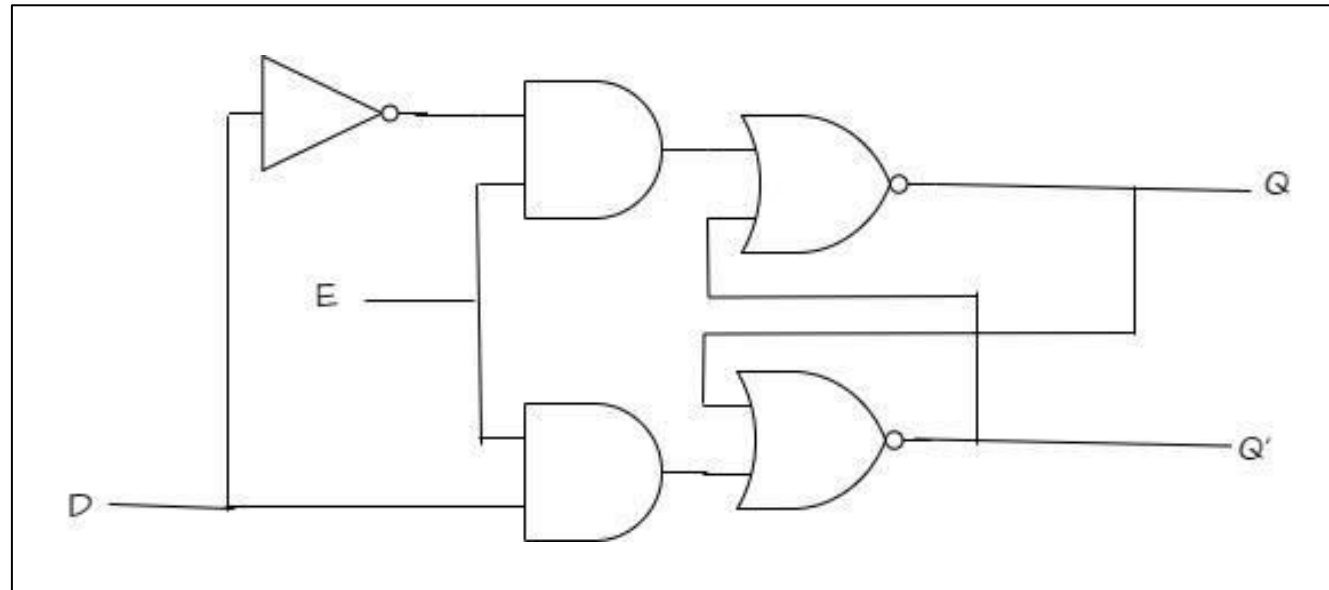
- ❑ D latches are also known as transparent latches and are implemented using two inputs: D (Data) and a clock signal.
- ❑ The output of the latch follows the input at the D terminal as long as the clock signal is high.
- ❑ When the clock signal goes low, the output of the latch is stored and held until the next rising edge of the clock.

D Latch Truth Table

E	D	Q	Q'
0	0	Latch	Latch
0	1	Latch	Latch
1	0	0	1
1	1	1	0

D Latch Logic Diagram

- ❑ The below logic diagram represents the D latch.



Gated D Latch

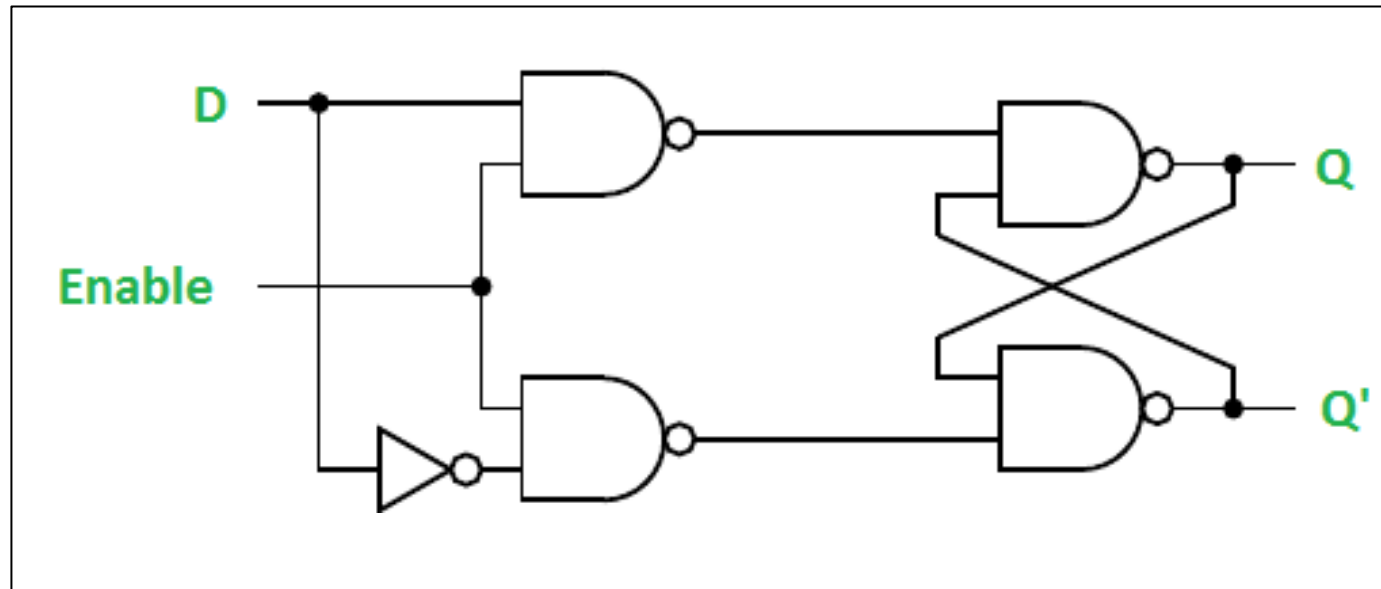
- ❑ D latch is similar to SR latch with some modifications made.
- ❑ Here, the inputs are complements of each other.
- ❑ The D latch stands for “data latch” as this latch stores single bit temporarily.

Gated D Latch Truth Table

Enable	D	Q_n	Q_{n+1}	STATE
1	0	x	0	RESET
1	1	x	1	SET
0	x	x	$Q(n)$	No Change
Characteristics Equation: $Q_{n+1} = EN.D + EN'.Q_n$				

Gated D Latch Logic Diagram

- ❑ The below logic diagram represents the gated D latch.



JK Latch

- ❑ JK latch has two inputs J and K.
- ❑ The output gets toggled when the J and K inputs are high.
- ❑ JK latch is just like SR latch, but it eliminates the undefined state of SR latch.

JK Latch

J	K	Q_{n+1}	Comment
0	0	Q	No change
0	1	0	Reset
1	0	1	Set
1	1	Q'	Toggle

JK Latch

- ❑ The below logic diagram represents the JK latch.

T Latch

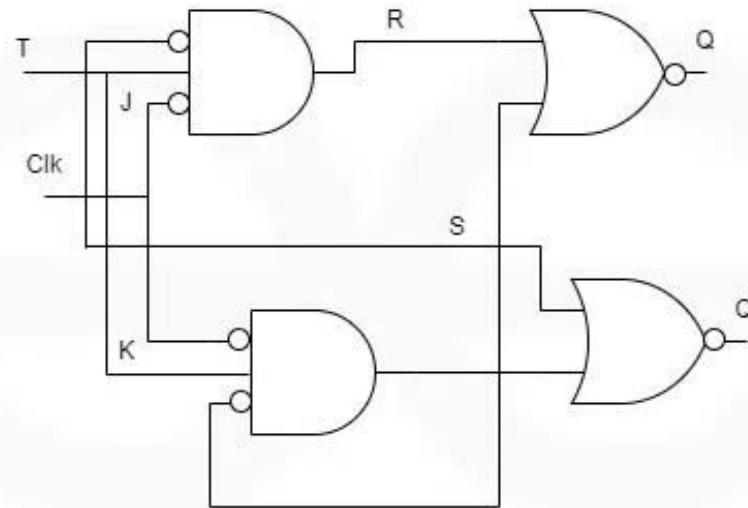
- ❑ When the J, K inputs of JK latch are shorted we get the T latch.
- ❑ In T latch the outputs are toggled when the inputs are high.

T Latch

T	Q (Next State)
0	No Change
1	Toggle Q

T Latch

- ❑ The below logic diagram represents the T latch.



Advantages of Latches

- ❑ Some of the advantages of latches are listed below.
- 1) **Easy to Implement:** Latches are simple digital circuits that can be easily implemented using basic digital logic gates.
- 2) **Low Power Consumption:** Latches consume less power compared to other sequential circuits such as flip-flops.
- 3) **High Speed:** Latches can operate at high speeds, making them suitable for use in high-speed digital systems.
- 4) **Low Cost:** Latches are inexpensive to manufacture and can be used in low-cost digital systems.
- 5) **Versatility:** Latches can be used for various applications, such as data storage, control circuits, and flip-flop circuits.

Disadvantages of Latches

- ❑ Some of the disadvantages of latches are listed below.
- 1) **No Clock:** Latches do not have a clock signal to synchronize their operations, making their behavior unpredictable.
- 2) **Unstable State:** Latches can sometimes enter into an unstable state when both inputs are at 1. This can result in unexpected behavior in the digital system.
- 3) **Complex Timing:** The timing of latches can be complex and difficult to specify, making them less suitable for real-time control applications.

Any Question?

Thank you

Digital Logic and Design

Lecture : 11

Department of Computer Science
GPGC SWABI (Khyber Pakhtunkhwa)

Spring 2025 Semester
BS Course: DLD

Course Instructor :
Mr. Ihtisham Ul Haq
Lecturer (CS)
GPGC Swabi

Contents

- ❑ Flip-Flops
 - ❑ SR Flip-Flop
 - ❑ JK Flip-Flop
 - ❑ D Flip-Flop
 - ❑ T Flip-Flop
- ❑ Advantages of Flips-Flops
- ❑ Disadvantages of Flip-Flops

Flip-Flops

- ❑ The flip-flop is a circuit that maintains a state until directed by input to change the state.
- ❑ A basic flip-flop can be constructed using four-NAND or four-NOR gates.
- ❑ Flip-flop is popularly known as the basic digital memory circuit.
- ❑ It has its two states as logic 1(High) and logic 0(low) states.
- ❑ A flip flop is a sequential circuit which consist of single binary state of information or data.
- ❑ The digital circuit is a flip flop which has two outputs and are of opposite states.
- ❑ It is also known as a Bistable Multivibrator.

Flip Flops

- ❑ In flip flops/any digital circuit the initial state is uncertain.
- ❑ It may come to set($Q=1$) or reset($Q=0$) state.
- ❑ In many applications, it is desired to initially set or reset the flip flop that is the initial state of the flip flop that needs to be assigned.
- ❑ This thing is accomplished by the preset(PR) and the clear(CLR).
- ❑ PR and CLR are active low (activated when equal to 0).
- ❑ After initialization, these signals are kept inactive, and the flip-flop operates under the control of the clock and data inputs (like J, K, D, or T).

Types of Flip-Flops

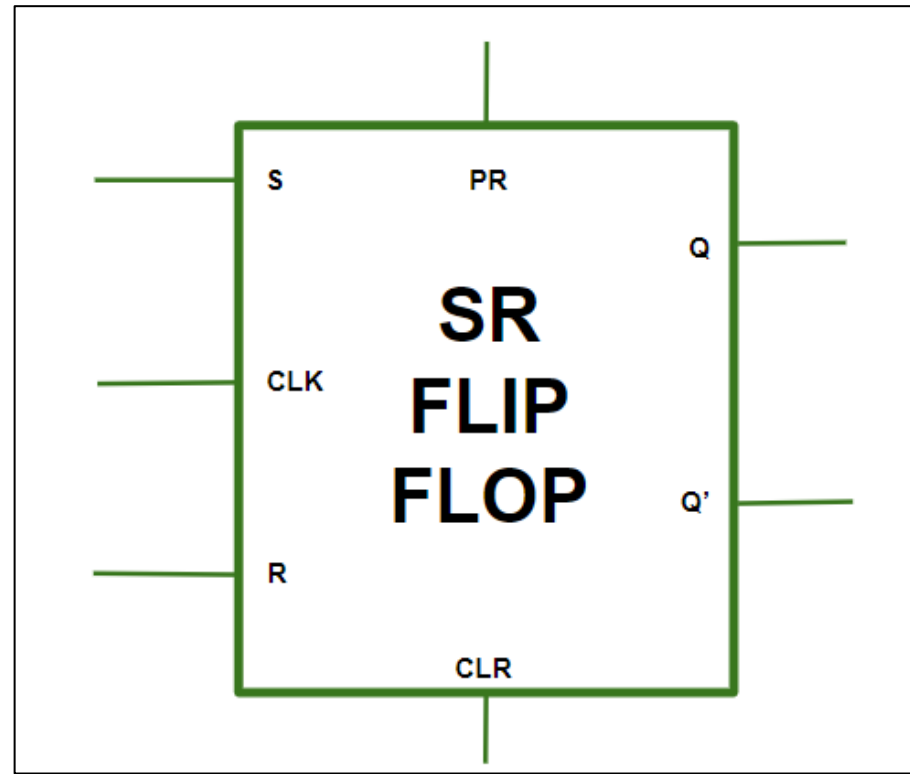
- ☐ Given Below are the Types of Flip-Flop.
 - ☐ SR Flip Flop
 - ☐ JK Flip Flop
 - ☐ D Flip Flop
 - ☐ T Flip Flop

S-R Flip Flop

- ❑ Two inputs S (Set) and Reset (R), that's why called SR Flip Flop.

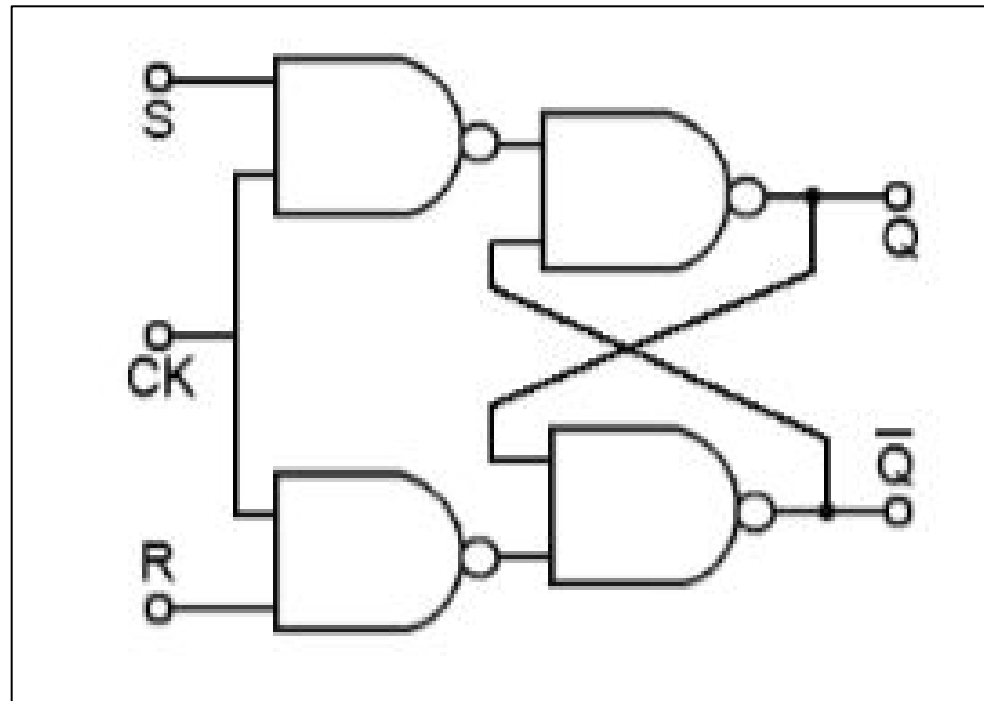
S-R Flip Flop Block Diagram

- ❑ Given Below is the Block Diagram of S-R Flip Flop.



S-R Flip Flop Logic Diagram

- ❑ Given Below is the Block Diagram of S-R Flip Flop.



S-R Flip Flop Truth Table

S (Set)	R (Reset)	Q (Next State)	Q' (Complement)
0	0	No Change	No Change
0	1	0	1
1	0	1	0
1	1	Invalid	Invalid

Operations of S-R Flip Flop

- ❑ Given Below are the Operations of S-R Flip Flop.
- ❑ **Case 1(PR=CLR=1):** The asynchronous inputs are inactive and the flip flop responds freely to the S,R and the CLK inputs in the normal way.
- ❑ **Case 2(PR=0 and CLR=1):** This is used when the Q is set to 1.
- ❑ **Case 3(PR=1 and CLR=0):** This is used when the Q' is set to 1.
- ❑ **Case 4(PR=CLR=0):** This is an invalid state.

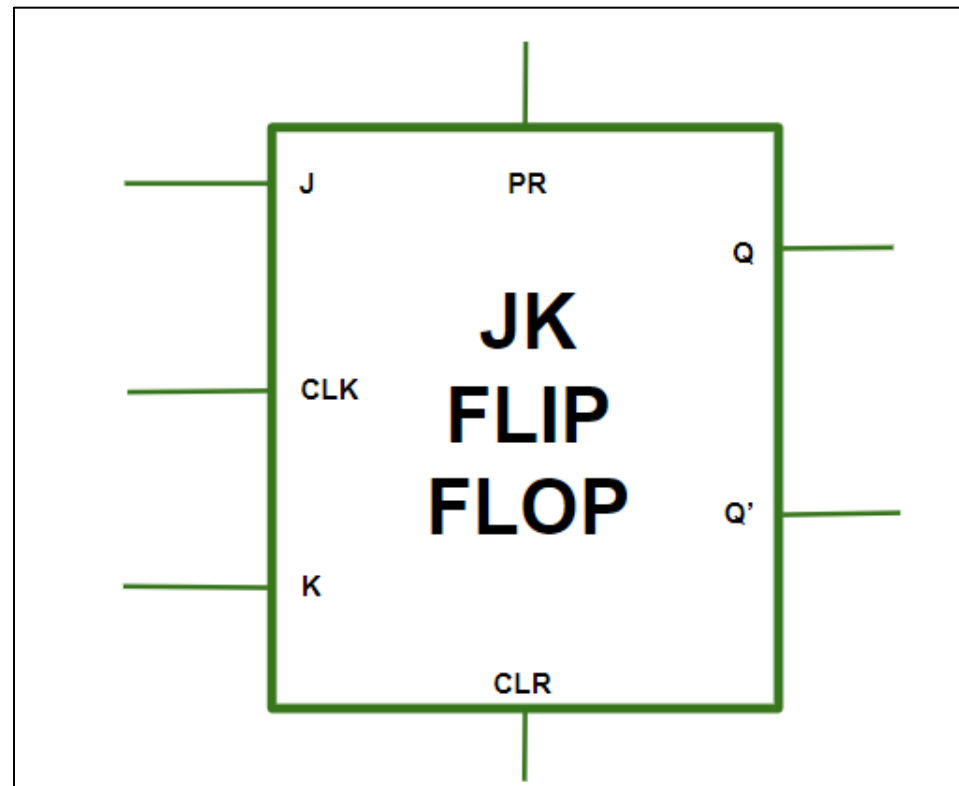
- ❑ **Characteristics Equation for SR Flip Flop**
- ❑ $Q_{N+1} = QNR' + SR'$

J-K Flip Flop

- ❑ In JK flip flops, The basic structure of the flip flop which consists of Clock (CLK), Clear (CLR), Preset (PR).

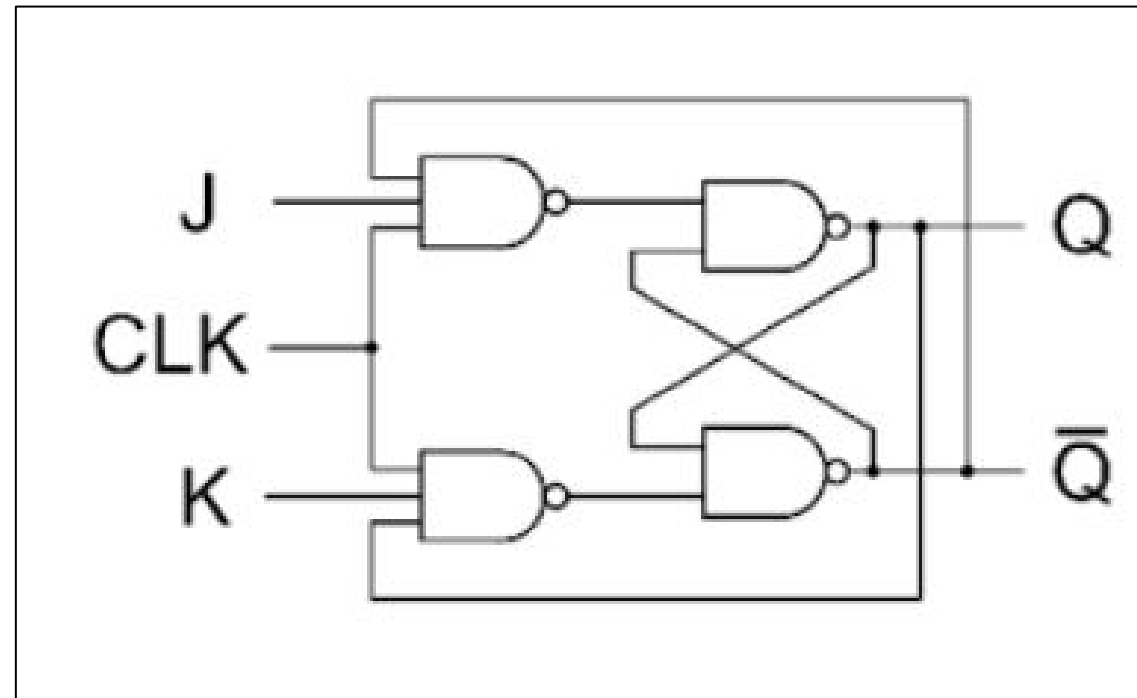
J-K Flip Flop Block Diagram

- ❑ Given Below is Block Diagram of J-K Flip Flop.



J-K Flip Flop Logic Diagram

- ❑ Given Below is Logic Diagram of J-K Flip Flop.



J-K Flip Flop Truth Table

J	K	Q (Next State)	Q' (Complement)
0	0	No Change	No Change
0	1	0	1
1	0	1	0
1	1	Toggle Q	Toggle Q'

Operations of J-K Flip Flop

- ❑ Given Below are the Operations of J-K Flip Flop.
- ❑ **Case 1 (PR=CLR=0):** This is an invalid state.
- ❑ **Case 2 (PR=0 and CLR=1):** The PR is activated which means the output in the Q is set to 1. Therefore, the flip flop is in the set state.
- ❑ **Case 3 (PR=1 and CLR=0):** The CLR is activated which means the output in the Q' is set to 1. Therefore, the flip flop is in the reset state.
- ❑ **Case 4 (PR=CLR=1):** In this condition the flip flop works in its normal way whereas the PR and CLR gets deactivated.

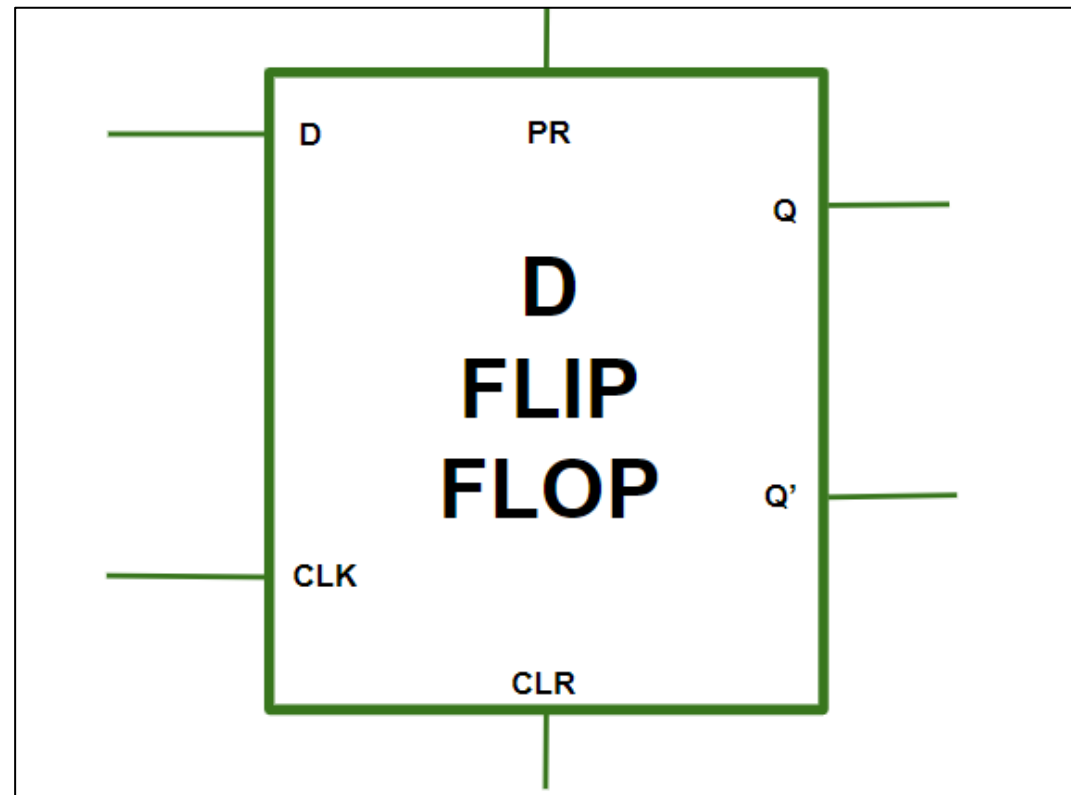
- ❑ **Characteristics Equation for JK Flip Flop**
- ❑ $Q_{n+1} = JQ'N + K'QN$

D Flip-Flop

- ❑ The D Flip Flop Consists a single data input(D), a clock input(CLK),and two outputs: Q and Q' (the complement of Q).

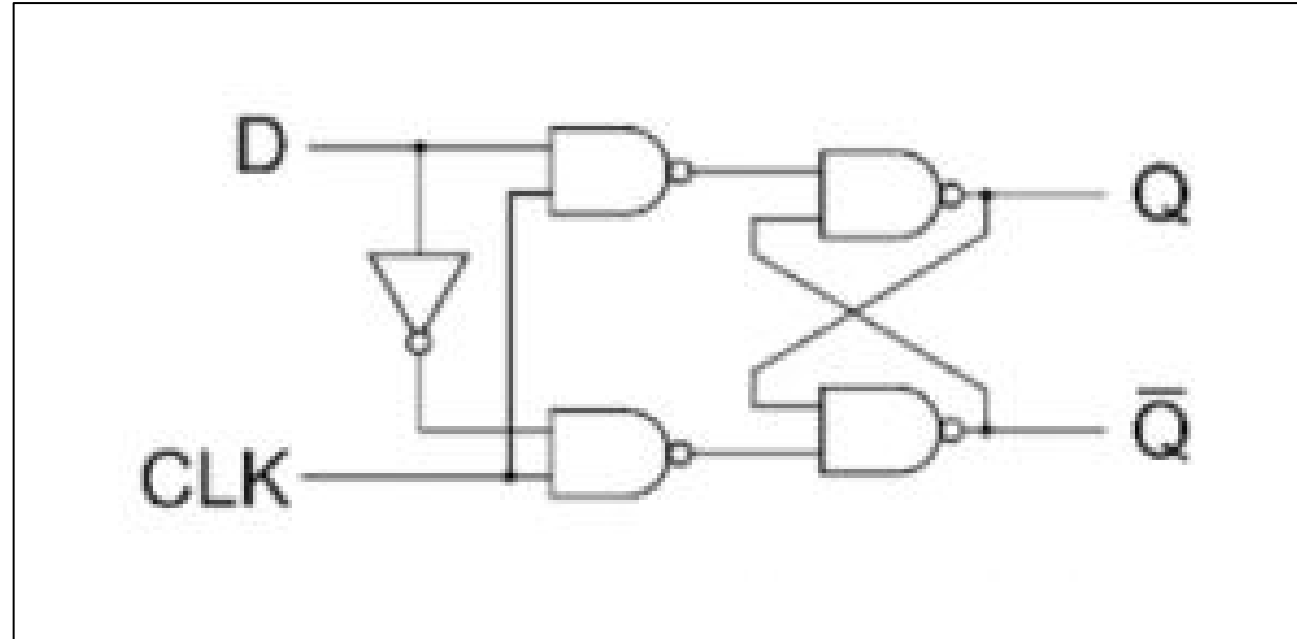
D Flip-Flop Block Diagram

- ❑ Given Below is the Block Diagram of D Flip Flop.



D Flip-Flop Logic Diagram

- ❑ Given Below is the Logic Diagram of D Flip Flop.



D Flip-Flop Truth Table

D	Q (Next State)	Q' (Complement)
0	0	1
1	1	0

Operations of the D Flip-Flop

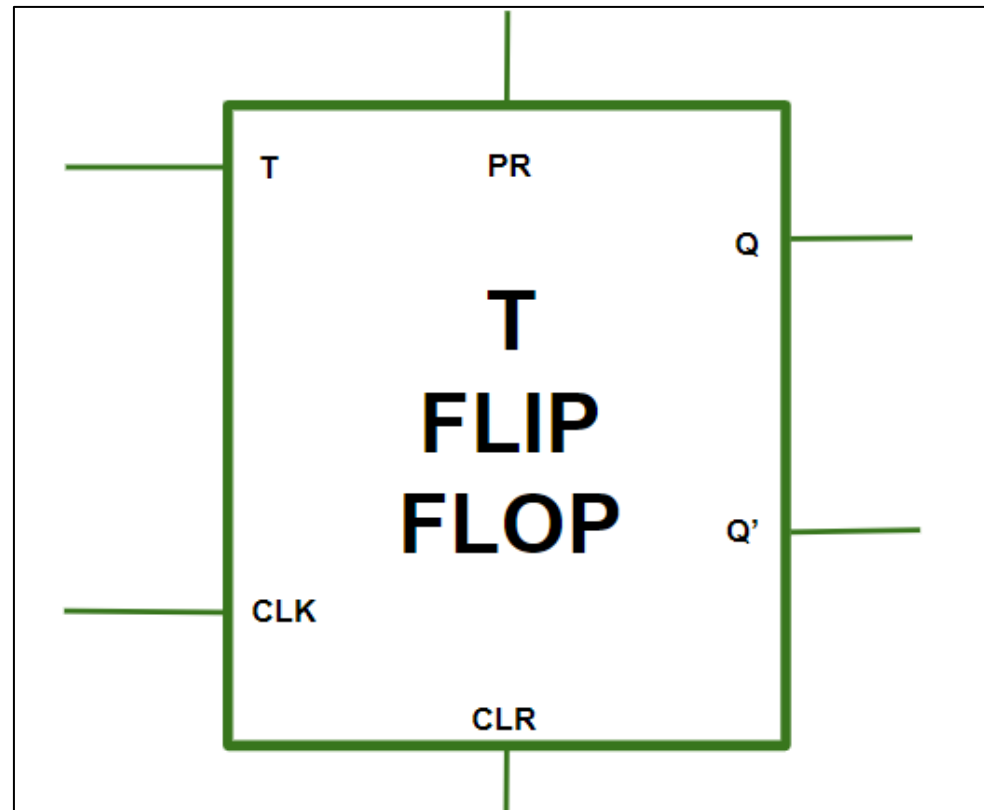
- ❑ Given Below are the operation of D Flip-Flop.
- ❑ **Case 1 (PR=CLR=0):** This condition represents an invalid state where both PR(present) and CLR(clear) inputs are inactive.
- ❑ **Case 2 (PR=0 and CLR=1):** This state is set state in which PR is inactive (0) and CLR is active(1) and the output Q is set to 1.
- ❑ **Case 3 (PR=1 and CLR=0):** This state is reset state in which PR is active (1) and CLR is inactive (0) and the complementary output Q' is set to 1.
- ❑ **Case 4 (PR=CLR=1):** In This state the flip flop behaves as normal, both PR and CLR inputs are active(1).
- ❑ **Characteristics Equation for D Flip Flop**
- ❑ $Q_{N+1} = D$

T Flip Flop

- ❑ The T Flip Flop consists of data input (T), a clock input (CLK), and two outputs: Q and Q' (the complement of Q).

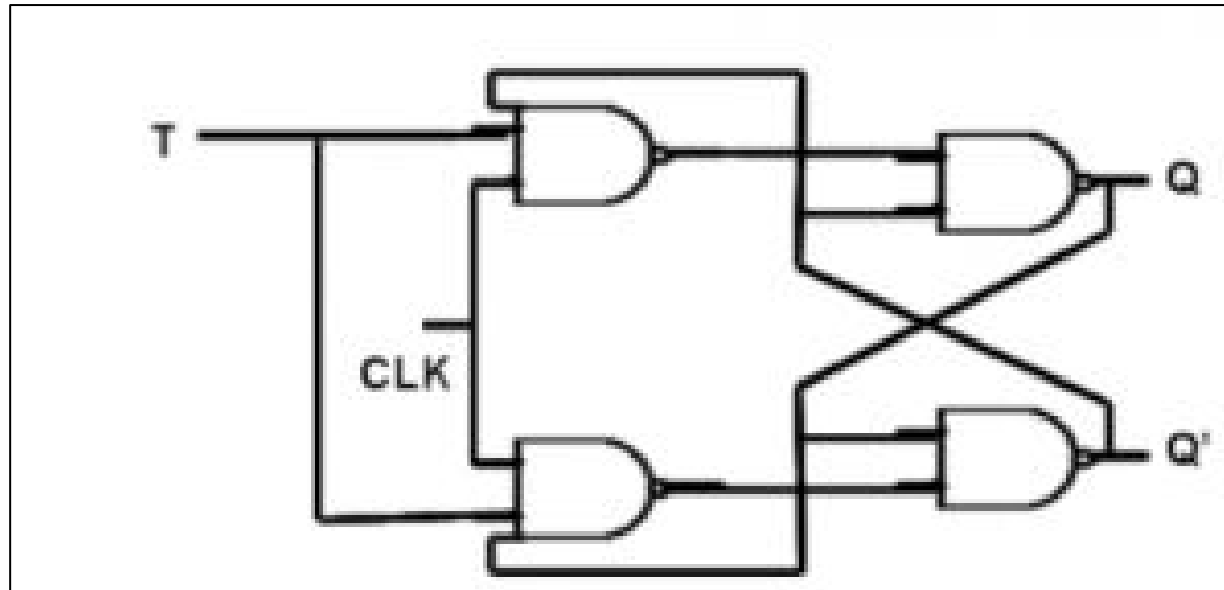
T Flip Flop Block Diagram

- ❑ Given Below is the Block Diagram of T Flip Flop.



T Flip Flop Logic Diagram

- ❑ Given Below is the Logic Diagram of T Flip Flop.



T Flip Flop Truth Table

T	Q (Next State)	Q' (Complement)
0	No Change	No Change
1	Toggle Q	Toggle Q'

Operations of the T Flip-Flop

- ❑ Given Below are the Operations of T Flip-Flop.
- ❑ **Case 1 (T=0):**In this condition the flip-flop remains in its current state regardless of clock input, Also the Output Q will remain unchanged until the value of T will not change.
- ❑ **Case 2 (T=1):**In this condition the flip flop's output will change. At each rising or falling edge of the clock signal the output Q will be in complementary state.

- ❑ **Characteristics Equation for T Flip Flop**
- ❑ $Q_{n+1} = Q'NT + QNT' = QN \text{ XOR } T$

Advantages of Flip-Flops

- 1) **Data Storage:** Flip-flops store one bit of data, making them essential for registers, memory units, and latches.
- 2) **Synchronous Operation:** Unlike latches, flip-flops operate on clock edges, reducing issues like race conditions.
- 3) **Sequential Logic Design:** Flip-flops are the building blocks of counters, shift registers, and state machines.
- 4) **Noise Immunity:** Because they change state only on clock edges, flip-flops are less affected by glitches compared to level-triggered latches.
- 5) **Toggle Functionality:** The T flip-flop is ideal for frequency division and binary counting applications.
- 6) **Reliable Data Synchronization:** Used in edge-triggered data transfers, ensuring accurate data flow in digital circuits.

Disadvantages of Flip-Flops

- 1) **Higher Power Consumption:** Flip-flops require a continuous clock signal, consuming more power than simple latches.
- 2) **Increased Circuit Complexity:** More gates and transistors are needed, increasing design complexity.
- 3) **Slower than Latches:** Since they update only on clock edges, flip-flops cannot respond as quickly as level-triggered latches.
- 4) **Requires More Hardware:** Implementing large-scale sequential circuits with flip-flops increases cost and chip area.
- 5) **Clock Dependency:** Flip-flops depend on the clock signal, which can introduce timing delays and synchronization issues.

Latches Vs Flip-Flops

- 1) Latches are level-triggered (respond continuously while enable is active).
- 2) Flip-flops are edge-triggered (respond only on clock edges).
- 3) Latches change state as long as the control signal (like Enable) is active.
- 4) Flip-flops change state only at the rising or falling edge of the clock.
- 5) Latches are generally simpler and faster.
- 6) Flip-flops are more robust and preferred in synchronous systems.

Any Question?

Thank you

Digital Logic and Design

Lecture : 13

Department of Computer Science
GPGC SWABI (Khyber Pakhtunkhwa)

Spring 2025 Semester
BS Course: DLD

Course Instructor :
Mr. Ihtisham Ul Haq
Lecturer (CS)
GPGC Swabi

Contents

- ☐ Counters
 - ☐ Asynchronous Counters
 - ☐ Synchronous Counters
 - ☐ MOD-N Counters (Decade Counters)
 - ☐ Applications of Counters
 - ☐ Advantages of Counters
 - ☐ Disadvantages of Counters
- ☐ Registers
 - ☐ Types of Registers
 - ☐ Applications of Registers
 - ☐ Advantages of Registers
 - ☐ Disadvantages of Registers

Counters

- ❑ A Counter is a device which stores (and sometimes displays) the number of times a particular event or process has occurred, often in relationship to a clock signal.
- ❑ A counter is a sequential circuit that counts pulses (clock signals) and keeps track of occurrences.
- ❑ It consists of flip-flops (D, T, or JK) and can be used in various applications like digital clocks, frequency measurement, and memory address sequencing.
- ❑ For example, in UP counter a counter increases count for every rising edge of clock.
- ❑ Not only counting, a counter can follow the certain sequence based on our design like any random sequence 0,1,3,2....

Counters

- ❑ They are used as frequency dividers where the frequency of given pulse waveform is divided.
- ❑ Counters are sequential circuit that count the number of pulses can be either in binary code or BCD form.
- ❑ The main properties of a counter are timing , sequencing , and counting.
- ❑ Counter works in two modes
 - ❑ Up counter
 - ❑ Down counter

Types of Counters

- ❑ Counters can be classified into two major types:
 1. **Asynchronous (Ripple) Counters**
 2. **Synchronous Counters**

Asynchronous (Ripple) Counter

- ☐ The clock is applied only to the first flip-flop, and the output of one flip-flop acts as the clock for the next.
- ☐ It is called a ripple counter because the clock pulse ripples through the flip-flops.
- ☐ Slower operation due to propagation delay.
- ☐ Simple design, requires fewer connections.

Synchronous Counter

- ❑ All flip-flops are triggered by the same clock signal at the same time.
- ❑ Faster and more efficient because there is no ripple effect.
- ❑ Requires additional combinational logic (AND gates) to control flip-flop toggling.

Decade Counter

- ❑ A decade counter counts ten different states and then reset to its initial states.
- ❑ A simple decade counter will count from 0 to 9 but we can also make the decade counters which can go through any ten states between 0 to 15(for 4 bit counter).

- ❑ **Important point:** Number of flip flops used in counter are always greater than equal to $(\log_2 n)$, where n =number of states in counter.

Applications of Counters

- ❑ **Digital Clocks** – Used to count seconds, minutes, and hours.
- ❑ **Frequency Counters** – Measures signal frequency in electronics.
- ❑ **Microprocessor Program Counter** – Tracks instruction execution.
- ❑ **Traffic Light Control** – Controls signal transitions at intersections.
- ❑ **Event Counting** – Used in industrial automation for product tracking.
- ❑ **Elevator Control** – Tracks floors for up/down movement.
- ❑ **Data Transmission** – Synchronizes bit transfer in communication systems.
- ❑ **Security Systems** – Generates one-time passwords (OTP).
- ❑ **Speedometers/Tachometers** – Measures vehicle/motor speed in RPM.
- ❑ **Pulse Generators** – Generates timing pulses in digital circuits.

Advantages of Counters

- 1. Simple and Cost-Effective** – Easy to design using flip-flops and logic gates.
- 2. Efficient in Counting Events** – Used in frequency measurement, event counting, and digital clocks.
- 3. Automation & Control** – Essential in industrial automation, traffic signals, and elevators.
- 4. Low Power Consumption** – Synchronous counters consume minimal power in digital circuits.
- 5. High Speed in Synchronous Counters** – No ripple effect, making them faster than asynchronous counters.
- 6. Memory Address Tracking** – Used in microprocessors for program counters and memory addressing.
- 7. Versatility** – Supports different counting modes (up, down, up/down, MOD-N).

Disadvantages of Counters

- 1. Propagation Delay in Ripple Counters** – Asynchronous counters are slow due to sequential clocking.
- 2. Complexity in Large Counters** – Requires additional logic gates for large counting ranges.
- 3. Limited Speed in Asynchronous Counters** – Slower than synchronous counters due to cumulative delays.
- 4. Power Consumption in Large Circuits** – More flip-flops increase power usage in larger designs.
- 5. Glitches in Asynchronous Counters** – Delay variations can cause incorrect counts.
- 6. Hardware Requirement** – More flip-flops and logic gates are needed for higher bit counters.

Registers

- ❑ A register is a group of flip-flops used to store binary data.
- ❑ It is a basic storage element in digital systems and is used for temporary data storage, transfer, and manipulation.
- ❑ Each flip-flop in a register stores one bit, so an n -bit register consists of n flip-flops.

Types of Registers

1. Buffer Register

- ☐ Stores temporary data for processing.
- ☐ Used for data transfer between components in a system.

2. Shift Register

- ☐ Shifts data left or right on each clock pulse.
- ☐ Used in serial data transfer and multiplication/division operations.

3. Counter Register

- ☐ Holds the count value for counters in CPUs and digital systems.
- ☐ Used in program counters and timers.

Types of Registers

4. Accumulator Register

- ❑ Stores intermediate results in **arithmetic and logic operations**.
- ❑ Commonly used in **ALU (Arithmetic Logic Unit)**.

5. General-Purpose Register (GPR)

- ❑ Temporary storage for **data and instructions** in microprocessors.
- ❑ Example: **AX, BX, CX, DX** registers in x86 architecture.

6. Special-Purpose Register (SPR)

- ❑ Stores control and status information.
- ❑ Example: **Instruction Register (IR), Stack Pointer (SP), Program Counter (PC)**.

Types of Shift Register

- ❑ **Serial-In Serial-Out (SISO)** – Data enters and exits serially.
- ❑ **Serial-In Parallel-Out (SIPO)** – Data enters serially and is available in parallel.
- ❑ **Parallel-In Serial-Out (PISO)** – Data enters in parallel and exits serially.
- ❑ **Parallel-In Parallel-Out (PIPO)** – Data enters and exits in parallel.

Applications of Registers

1. **Temporary Data Storage** – Holds data during CPU operations.
2. **Arithmetic and Logic Operations** – Stores operands and results in ALUs.
3. **Data Transfer** – Facilitates data movement between CPU, memory, and I/O devices.
4. **Instruction Execution** – Stores current instructions in the Instruction Register (IR).
5. **Program Execution Control** – Program Counter (PC) keeps track of the next instruction.
6. **Stack Operations** – Stack Pointer (SP) manages function calls and interrupts.
7. **Serial-to-Parallel and Parallel-to-Serial Conversion** – Used in communication systems.
8. **Counters and Timers** – Registers hold count values in counters and clocks.
9. **Memory Addressing** – Address Registers hold memory addresses for read/write operations.
10. **Shift Registers** – Used for bitwise operations, encryption, and signal processing.

Advantages of Registers

- 1. Fast Data Access** – Registers provide the fastest memory access compared to cache or RAM.
- 2. Efficient Data Processing** – Used in arithmetic operations, data storage, and control execution.
- 3. Low Power Consumption** – Registers use minimal power compared to larger memory units.
- 4. Data Transfer Optimization** – Shift registers enable smooth serial and parallel data transfer.
- 5. Instruction Execution Control** – Used in instruction fetching, decoding, and execution.
- 6. Memory Addressing** – Address registers speed up memory read/write operations.
- 7. Storage for Immediate Values** – Registers hold frequently used data, reducing memory access time.

Disadvantages of Registers

- 1. Limited Storage Capacity** – Registers store only a few bytes of data (typically 8-bit, 16-bit, 32-bit, or 64-bit).
- 2. High Cost per Bit** – Registers are made of flip-flops, which are expensive compared to RAM or storage.
- 3. Complex Hardware Design** – Requires more transistors and control logic in CPUs.
- 4. Volatility** – Registers lose data when the power is turned off.
- 5. Not Expandable** – Unlike RAM, register storage cannot be increased.
- 6. Dependency on Processor Architecture** – The number and type of registers vary based on CPU design.

Any Question?

Thank you